

Optimization by Simulated Annealing

S. Kirkpatrick, C. D. Gelatt Jr., and M. P. Vecchi

Summary

The paper introduces **simulated annealing** as a general-purpose method for solving large combinatorial optimization problems. Its main idea is that finding a low-cost configuration in optimization is similar to cooling a physical material until it reaches a low-energy state.

The authors adapt the **Metropolis algorithm** from statistical mechanics. In this analogy:

- A candidate solution is treated as a physical configuration.
- The objective or cost function is treated as energy.
- Temperature becomes a control parameter governing how freely the search explores different solutions.

Unlike ordinary iterative improvement, simulated annealing does not accept only better solutions. It may also accept a worse solution, especially when the temperature is high. This allows the search to escape local optima rather than becoming permanently trapped in them.

As the temperature gradually decreases, worse moves become less likely to be accepted. The algorithm therefore changes from broad exploration at high temperatures to detailed refinement at low temperatures.

Theoretical Basis

The paper begins by discussing combinatorial optimization problems, in which the number of possible solutions can grow extremely quickly as the problem becomes larger. Exact methods may therefore become computationally impractical.

A common alternative is iterative improvement. This begins with one solution and repeatedly applies changes that reduce the objective value. However, because it accepts only improvements, it often becomes trapped in a **local optimum**.

The authors compare this to rapidly cooling a physical material. If a material is cooled too quickly, its atoms may freeze into a defective or metastable arrangement rather than settling into the lowest-energy structure.

Slow physical annealing avoids this by allowing the material to remain mobile while its temperature decreases. Simulated annealing applies the same principle to optimization.

Under the Metropolis procedure:

- A proposed move that lowers the objective value is accepted.
- A move that raises the objective value may still be accepted.
- The probability of accepting a worse move decreases when the cost increase becomes larger.
- The probability also decreases as the temperature becomes lower.

This controlled acceptance of worse solutions allows the algorithm to move out of local optima and continue exploring.

Annealing Process

The algorithm begins at a high effective temperature, where many proposed moves are accepted. This is described as “melting” the optimization system.

The temperature is then lowered in stages. At every temperature, the search performs enough moves for the system to approach a stable or steady condition.

The combination of:

- the starting temperature,
- the sequence of temperature reductions,
- the number of moves attempted at each temperature,
- and the stopping condition

is called the **annealing schedule**.

At high temperatures, the algorithm determines broad structural features of the solution. At lower temperatures, it focuses on smaller and more detailed improvements. The authors describe this as a form of **adaptive divide-and-conquer**.

Statistical-Mechanics Concepts

The paper draws on several concepts from statistical mechanics.

Boltzmann Probability

The probability of accepting a worse solution is based on the Boltzmann distribution. Small cost increases are more likely to be accepted than large increases, particularly when the temperature is high.

Ground States

A ground state is the lowest-energy physical configuration. In optimization, this corresponds to an optimal or near-optimal solution.

Metastable States

A metastable state is a locally stable but nonoptimal physical arrangement. In optimization, it corresponds to a local optimum.

Entropy

Entropy represents how many configurations contribute to the system at a given temperature. As the temperature decreases, fewer configurations remain likely, and the search becomes more concentrated.

Specific Heat

The authors use a quantity similar to specific heat to identify temperatures where major structural changes occur. These points can indicate when slower cooling may be useful.

Frustration

Frustration occurs when different interactions or constraints conflict with one another, making it impossible to satisfy all preferences simultaneously. The authors compare difficult optimization problems to spin-glass systems, which can contain many different low-energy states.

This suggests that some optimization problems may contain many near-optimal solutions rather than one clearly superior solution.

Applications Examined

The paper applies simulated annealing to four main types of problems:

1. Circuit partitioning
2. Component placement
3. Wire routing
4. The Traveling Salesman Problem

Circuit Partitioning

The first application involves dividing circuits between two computer chips.

The objective balances two goals:

- minimizing the number of signals that cross between chips,
- and keeping the number of circuits assigned to each chip reasonably balanced.

The authors tested the method on an IBM microprocessor design containing approximately 5,000 primitive logic gates.

A random assignment would require around 3,000 pins on each chip. Simulated annealing produced much better partitions.

One balanced result required:

- 353 pins on one chip,
- 321 pins on the other.

Using a less strict balance penalty produced another result requiring:

- 271 pins on one chip,
- 183 pins on the other.

By comparison, rapid zero-temperature improvement produced solutions requiring approximately 677 to 730 pins. This showed that slow cooling found substantially better configurations than immediately allowing only improving moves.

Component Placement

The second application involves arranging 98 chips on a 10-by-10 grid within an IBM 3081 processor module.

The objectives were to:

- reduce estimated wire length,
- reduce wiring congestion,
- and satisfy physical placement constraints.

The algorithm generated moves by swapping two chips or swapping a chip with an empty location.

At high temperatures, chips moved freely and the layout became disordered. As the system cooled, functionally related groups of chips began forming clusters. At lower temperatures, smaller rearrangements refined the layout.

The final placement achieved approximately:

- a 30% reduction in peak wiring congestion,
- and a 10% reduction in estimated total wire length.

The experiment attempted about 250,000 chip interchanges and required approximately 12 minutes on an IBM 3033 computer.

The paper also observed two major temperature regions during cooling. One corresponded to the formation of large functional clusters, while the other corresponded to smaller local reductions in wire length.

Wire Routing

The third application involves selecting paths for wires connecting electronic components.

Random routing can create areas with excessive wire density. The authors examined different routing rearrangements, including L-shaped and Z-shaped moves.

Simulated annealing using Z-shaped moves produced the best reported result.

All tested optimization procedures reduced peak wire density by more than 45% compared with random routing. Simulated annealing with Z-shaped moves reduced the peak density by approximately 57% on average across the horizontal and vertical directions.

The method evaluates multiple wire routes collectively rather than processing each wire independently in a fixed sequence. This helps avoid the order-dependence found in conventional sequential routing procedures.

Traveling Salesman Problem

The Traveling Salesman Problem was used as a simpler and more standardized problem for studying simulated annealing.

A solution is represented as an ordered list of cities. A move reverses the direction of a selected section of the current route.

The experiments used Manhattan distance and included both:

- randomly distributed cities,
- and cities arranged in separate clusters.

The authors applied simulated annealing to instances containing up to approximately **6,000 cities**.

In the clustered 400-city problem, different types of route structure emerged at different temperatures:

- At high temperatures, inefficient long connections between distant regions began disappearing.
- At intermediate temperatures, the broad connections between clusters were organized.
- At lower temperatures, the route continued improving through local rearrangements within each cluster.

This supported the authors' argument that simulated annealing separates large-scale organization from small-scale refinement.

The paper also examined a simple greedy method that repeatedly visits the closest unvisited city. Although the method can perform very poorly in theoretical worst cases, its observed average performance was substantially better than the worst-case prediction.

The authors argue that for large practical problems, average behavior may sometimes be more useful than worst-case analysis.

Main Findings

The paper's principal findings are:

- Accepting occasional worse moves helps the search escape local optima.
- Slow cooling produces better solutions than rapid cooling or strict iterative improvement.
- Temperature controls the balance between exploration and refinement.
- Large structural changes tend to occur at high temperatures.
- Fine local improvements occur at low temperatures.
- Simulated annealing can be adapted to many different optimization problems.
- The method produced major improvements in partitioning, placement, and wire routing.
- It was able to process Traveling Salesman instances containing several thousand cities.
- The algorithm itself is relatively general, although designing effective moves and objective functions still requires knowledge of the problem.

Important Implementation Requirements

The authors identify four essential components of a simulated annealing implementation:

1. **Configuration representation**
A concise way to represent a candidate solution.
2. **Move generator**
A method for creating random neighboring solutions or rearrangements.
3. **Objective function**
A numerical function measuring solution quality and representing the necessary trade-offs.
4. **Annealing schedule**
A plan defining the temperatures, cooling rate, number of moves at each temperature, and stopping condition.

The paper states that the basic Metropolis mechanism is straightforward to implement. However, several problem-specific decisions may be difficult:

- selecting useful neighborhood moves,
- designing an appropriate objective function,
- choosing the initial temperature,
- determining how slowly to cool,
- and deciding when the system has frozen.

Conclusion

The authors conclude that simulated annealing provides a broader and more flexible search framework than ordinary iterative improvement.

Its temperature parameter allows the search to accept temporary deterioration, escape local optima, and address different levels of solution structure at different stages of cooling.

Their numerical experiments suggested that good solutions could be obtained with computational effort increasing approximately linearly or as a relatively small power of the problem size.

Because of its generality and ability to handle large problems, the authors predicted that simulated annealing would become a widely applicable heuristic optimization method.

Tabu Search—Part I

Fred Glover

Summary

The paper presents the foundational principles of **Tabu Search**, an adaptive strategy for solving combinatorial optimization problems. Tabu Search is designed to improve ordinary local-search methods by allowing the search to move beyond local optima while using memory to prevent cycling and guide future decisions.

Rather than treating optimization as a sequence of independent moves, the method uses information from the search history. Recent decisions, strong solutions, frequently used attributes, and previously explored regions can all influence which moves are allowed or preferred.

The paper develops the major components of Tabu Search, including:

- neighborhood-based movement,
- tabu restrictions,
- short-term memory,
- aspiration criteria,
- strategic oscillation,
- intermediate-term intensification,
- long-term diversification,
- probabilistic variants,
- and implementation structures for storing search information.

It also reviews early applications and computational findings involving scheduling, routing, graph coloring, circuit design, space planning, and other combinatorial problems.

Basic Optimization Framework

The paper represents a combinatorial optimization problem as the minimization of an objective function over a set of possible solutions.

A **move** transforms one trial solution into another. The set of moves available from the current solution forms its neighborhood.

Examples include:

- changing a binary variable from zero to one,

- following an alternating path in a graph,
- moving in a selected direction in nonlinear optimization,
- incrementing or decrementing an integer variable,
- exchanging elements between groups,
- or swapping edges in a route.

The specific meaning of a move depends on the problem being solved. Tabu Search is therefore not one fixed algorithm but a framework that controls and guides a problem-specific neighborhood search.

Limitation of Hill Climbing

The paper first describes a conventional hill-climbing procedure.

A hill-climbing method:

1. Starts with an initial solution.
2. Searches for a neighboring solution with a better objective value.
3. Moves to that solution.
4. Stops when no improving move exists.

Its major limitation is that it stops at a **local optimum**, which may not be globally optimal.

Tabu Search addresses this problem by continuing even when every available move makes the objective value temporarily worse. It selects the best admissible move, not necessarily an improving one.

This means that local optimality is not treated as a stopping condition. The search can move away from a local optimum and explore other regions.

Simple Tabu Search Procedure

The simplified procedure begins with:

- an initial solution,
- a record of the best solution found,
- an iteration counter,
- and an initially empty tabu set.

At each iteration:

1. The method identifies the currently available non-tabu moves.
2. It selects the best one according to an evaluation function.

3. It applies the selected move, even when it worsens the objective.
4. It updates the best-known solution if an improvement is found.
5. It updates the tabu restrictions.
6. It stops after a chosen number of iterations or after a specified period without improvement.

Three characteristics are emphasized:

- The tabu set constrains the search.
- The method does not stop merely because it reaches a local optimum.
- It selects the best admissible move rather than simply the first improving move.

If no improvement is possible, the method chooses the move producing the smallest deterioration.

Short-Term Memory and Tabu Restrictions

The core of the method is a short-term memory that records recent search information.

A common rule makes the reversal of a recently executed move temporarily tabu. For example, when an element has just been moved from one group to another, immediately moving it back may be prohibited.

This prevents the search from repeatedly cycling between the same solutions.

The period for which a move or attribute remains forbidden is called its **tabu tenure** or tabu-list length.

A large tenure reduces the chance of cycling but also restricts the search more heavily. A small tenure allows greater freedom but may permit the search to revisit previous states.

The paper reports that effective tenure values often exist within a relatively stable range, although this range can vary by problem. Later Traveling Salesman experiments showed that appropriate list size may depend on problem size and structure.

Strategic Forgetting

Tabu status is temporary rather than permanent.

Older restrictions are eventually removed, allowing the search to reconsider decisions after enough intervening exploration. This is described as **strategic forgetting**.

The search therefore remembers recent history strongly but gradually releases older information.

Move Attributes

The paper explains that storing complete moves or full solutions may require too much memory. Instead, Tabu Search can record selected **attributes** of moves.

An attribute may represent:

- the variable changed,
- an element's weight,
- the group receiving an element,
- an edge added to a route,
- an edge removed from a route,
- or an objective-function value.

Using attributes makes the tabu mechanism more flexible and memory-efficient.

It also means that one recorded attribute may classify several related moves as tabu rather than blocking only one exact move.

The amount of detail recorded matters. A very specific description blocks a narrow set of moves, while a broader description may prevent a larger family of related moves.

Partitioning Example

The paper presents an example involving the division of weighted elements among several sets.

Each set has a target total weight. The objective is to minimize the sum of deviations between:

- the actual weight assigned to each set,
- and that set's target weight.

The neighborhood consists of exchanging elements between sets. A null element with zero weight can be used so that an exchange may simply move one element without replacing it.

The best admissible swap is selected at each iteration, even when it temporarily worsens the objective.

Two data structures are introduced:

TABULIST

TABULIST stores attributes associated with recent moves. It is maintained as a circular list, so newly recorded attributes replace the oldest entries when the list reaches its capacity.

TABUSTATE

TABUSTATE allows the algorithm to determine efficiently whether a proposed move is tabu.

Instead of repeatedly searching the entire list, the algorithm consults the relevant TABUSTATE entries. Positive entries indicate that a particular attribute is currently forbidden.

The example illustrates how Tabu Search can encode recent search history without saving complete solutions.

Aspiration Criteria

A tabu restriction can sometimes block a highly desirable move. To prevent useful opportunities from being lost, the paper introduces **aspiration criteria**.

An aspiration criterion overrides tabu status when a move meets a sufficiently strong condition.

The most familiar case is allowing a tabu move when it produces a solution better than the best found so far. The paper develops more general aspiration-level functions based on:

- move attributes,
- objective values,
- historical performance,
- and the result previously obtained when similar moves were executed.

Aspiration criteria provide flexibility while preserving the anti-cycling role of tabu restrictions.

The paper describes tabu restrictions and aspiration criteria as complementary:

- Tabu restrictions make moves inadmissible.
- Aspiration criteria make otherwise tabu moves admissible.

Both can be organized within a common attribute-based framework.

Integration of Tabu Status and Aspiration

The paper explains that tabu status is often better assigned to move attributes than to complete moves.

For a Traveling Salesman Problem using 2-opt moves, attributes may include:

- the two edges removed,
- the two edges added,

- the lengths of those edges,
- or the objective value before and after the move.

Different attributes can receive:

- different tabu tenures,
- different aspiration levels,
- and different levels of importance.

For example, a short removed edge might remain tabu for less time than a longer one.

A move may be accepted when all its relevant attributes pass their aspiration tests, or when a sufficiently important attribute passes.

This allows a more refined decision than simply labeling an entire move as allowed or forbidden.

Search as a Directed Graph

The paper gives a graph-theoretic interpretation of Tabu Search.

- Every candidate solution is represented as a node.
- Every possible move from one solution to another is represented as a directed arc.
- A sequence of search iterations forms a path through this graph.

The tabu list temporarily blocks selected arcs, especially those that would reverse recent transitions.

This perspective helps explain cycling prevention and provides a possible foundation for theoretical analysis of the method.

Dominant, Equivalent, and Deficient Moves

The paper discusses how relationships among moves can influence tabu restrictions.

Dominant moves

A move dominates another when it is clearly preferable under the problem structure. If one move is prohibited, related dominated moves may also need to be prohibited.

Equivalent moves

Equivalent moves have the same meaningful effect. They may be treated similarly or ranked using a secondary rule.

Deficient moves

A deficient move may receive a favorable immediate evaluation but cannot lead to an improved solution.

Because Tabu Search can encounter nonimproving regions that ordinary hill climbing never visits, a heuristic evaluation rule may need modification when embedded inside Tabu Search. A move that appears best under a conventional local heuristic may need to be excluded because of its longer-term consequences.

Strategic Oscillation

Strategic oscillation deliberately moves the search back and forth across selected boundaries.

These boundaries may separate:

- feasible and infeasible solutions,
- constructive and destructive search,
- interior and exterior regions,
- primal and dual perspectives,
- or solutions with different levels of a controlling parameter.

Instead of remaining strictly inside the feasible region, the search may cross into infeasibility and later return from a different direction.

This can expose opportunities that are inaccessible when the search is restricted to one side of the boundary.

The paper describes applications where the search deliberately moves a specified number of steps beyond a boundary and then reverses direction. Multiple tabu lists may control movements in the two directions.

Strategic oscillation can:

- simplify the required neighborhood moves,
- reveal improved solutions near boundaries,
- explore slightly relaxed or tightened constraints,
- and prevent entrapment in a narrow region.

Intermediate-Term Memory and Intensification

Intermediate-term memory focuses the search more strongly on promising regions.

It records features shared by a collection of high-quality solutions, such as:

- frequently selected variables,
- commonly used edges,
- recurring assignments,
- or other structural patterns.

The search then encourages new solutions that retain these features. This process is called **intensification**.

For example, in a Traveling Salesman Problem, only a fraction of all possible edges may appear in good tours. After an initial search, unused edges can be temporarily discarded so that the algorithm concentrates on a smaller and more promising subproblem.

Because the reduced neighborhood can be evaluated more quickly, more iterations can be performed in the same amount of time.

The paper also proposes constructing regions from collections of good solutions and examining combinations or nearby variations of those solutions.

Long-Term Memory and Diversification

Long-term memory has the opposite purpose: it pushes the search toward regions unlike those already examined.

Rather than relying on random restarts, the method uses historical information to create a purposeful new direction.

Features that have appeared frequently are penalized, making less-used features more attractive.

In a Traveling Salesman Problem, the algorithm may count how often each edge has appeared. Frequently used edges then receive penalties so that new tours are encouraged to use different edges.

This is called **frequency-based memory**, in contrast with the recency-based memory of the ordinary tabu list.

Long-term diversification may:

- generate a new starting solution,
- continue directly from the current solution into a new region,
- or temporarily apply stronger tabu restrictions.

The paper argues that purposeful diversification is preferable to completely random restarting because it learns from the previous search.

Escaping Strong Local Optima

Some local optima act as strong attractors. Once the search enters them, ordinary moves may cause only a slow and irregular escape.

The paper suggests several accelerated-retreat strategies:

- applying stronger tabu restrictions,
- using the observed escape trajectory to predict a useful direction,
- and penalizing attributes that have appeared repeatedly.

These strategies can be relaxed once the search has moved far enough away.

The objective is to estimate the necessary escape distance from a local optimum and guide the search through that distance efficiently.

Probabilistic Tabu Search

The paper also proposes adding probabilistic elements to Tabu Search.

Randomization can produce diversity without requiring as much memory, but it creates a trade-off:

- Less record keeping may improve computational efficiency.
- Random movement can cause repeated work or unproductive wandering.

Three factors may determine the probability assigned to a move:

1. **Move attractiveness**
Better moves receive higher selection probability.
2. **Tabu status**
Recently forbidden moves receive lower probability.

3. **Aspiration level**

Strong historical performance may increase the probability or override the tabu penalty entirely.

The paper distinguishes this from simulated annealing.

In simulated annealing:

- all improving moves are accepted,
- the acceptance of worsening moves depends on temperature,
- and worsening moves become less likely over time.

In probabilistic Tabu Search:

- relative move quality remains important throughout the search,
- tabu history influences probability,
- and no cooling schedule is required.

The paper explains that probabilistic and deterministic components can be combined. Some recently recorded moves may remain completely forbidden, while older tabu moves receive gradually increasing probabilities of acceptance.

Under suitable assumptions, the probabilistic version can eventually reach an optimal solution with probability one if allowed to run indefinitely.

Practical Applications

The paper reviews several early applications.

Machine workload scheduling

A scheduling problem involving 300 to 500 integer variables and approximately 100 constraints was solved using strategic oscillation.

The reported objective values were less than half those obtained by the alternative methods tested. Runtime was approximately 10 to 20 seconds on a CDC 6600.

Computer channel balancing

This problem involved assigning loads to channels while minimizing deviations from target capacities.

Problems with approximately 106 to 1,000 variables were solved in under one minute. Tabu Search produced substantially better deviations than a commercial mathematical-programming system, while using much less computation time.

Space planning and clustering

A space-planning problem containing more than 25,000 variables and 50,000 constraints was solved in less than one minute on a minicomputer.

Employee scheduling

Large employee-scheduling problems involved:

- employee assignments,
- workstation requirements,
- duty rosters,
- employee availability,
- union rules,
- company policies,
- and time-period demand.

Instances containing approximately one to four million variables and thousands of constraints were solved to within 98% of optimality in roughly 22 to 24 minutes on an IBM personal computer.

Traveling Salesman Problem

The paper reports experiments using 2-opt moves, tabu restrictions, and aspiration criteria.

For smaller problems with known optima, Tabu Search found the optimal solutions.

For harder instances containing approximately 50 to 110 cities, the method usually improved upon the best previously reported solutions. In the 110-city case, the best solutions required between approximately four and twenty minutes, but each was superior to the previous best.

A notable result was that Tabu Search, using a relatively simple 2-opt neighborhood, outperformed earlier procedures based on the more powerful 3-opt heuristic.

The paper argues that effective search guidance can sometimes be more important than using a more sophisticated local move.

Reported Comparative Findings

The paper summarizes several early comparisons.

Reported studies found that Tabu Search outperformed simulated annealing in:

- graph coloring,
- integrated-circuit layout,
- the Traveling Salesman Problem,
- and maximum satisfiability.

These studies reported better solution quality and, in several cases, substantially lower computational effort.

For one reported Traveling Salesman comparison, Tabu Search found optimal solutions more frequently while using between roughly one-third and one-twenty-fifth of the effort required by simulated annealing.

Another scheduling comparison found that Tabu Search produced the best solutions among several methods in:

- approximately 80% of cases under a 12-minute limit,
- and approximately 90% when the time limit was doubled.

The paper cautions that the potential and limitations of Tabu Search were still not fully understood. It presents these results as promising early evidence rather than a complete proof of superiority.

Main Findings

The paper's main findings are:

- Memory allows local search to continue beyond local optima.
- Temporarily forbidding recent reversals helps prevent cycling.
- Selecting the best admissible move can be effective even when it worsens the current objective.
- Aspiration criteria prevent tabu restrictions from blocking exceptional opportunities.
- Attribute-based memory is more flexible and efficient than storing complete solutions.
- Short-term memory supports local movement and cycling prevention.
- Intermediate-term memory intensifies the search around promising patterns.
- Long-term memory diversifies the search toward unfamiliar regions.
- Strategic oscillation can exploit the boundary between feasible and infeasible regions.
- Probabilistic components may reduce memory requirements but can also create inefficient wandering.
- A relatively simple neighborhood can perform strongly when directed by effective memory and search controls.

- Early computational results showed strong performance across several large combinatorial problems.

Limitations and Future Directions

The author states that Tabu Search was still at an early stage of development.

Several issues remained open:

- identifying the best tabu tenure,
- determining which move attributes should be recorded,
- developing theoretical explanations for observed performance,
- deciding how aspiration criteria should be defined,
- coordinating short-, intermediate-, and long-term memory,
- measuring the appropriate escape distance from local optima,
- balancing deterministic control with randomization,
- and identifying the problem classes for which Tabu Search is best suited.

The paper also proposes further work involving:

- parallel processing,
- pattern recognition,
- neural networks,
- genetic algorithms,
- simulated annealing,
- and other hybrid search strategies.

Conclusion

The paper presents Tabu Search as a memory-based metastrategy rather than a single fixed algorithm.

Its essential principle is that search history should be used actively. Recent history prevents cycling, patterns from good solutions support intensification, and long-term frequencies support diversification.

By allowing nonimproving moves, controlling reversals, and overriding restrictions when strong opportunities arise, Tabu Search can move through regions that ordinary hill climbing cannot explore.

The early applications reviewed in the paper suggest that this framework can produce high-quality solutions with modest computational effort, including for very large combinatorial problems.

Particle Swarm Optimization: A Survey of Historical and Recent Developments with Hybridization Perspectives

Saptarshi Sengupta, Sanchita Basak, and Richard Alan Peters II

Summary

This paper is a broad review of **Particle Swarm Optimization (PSO)**, covering its historical origins, canonical operating mechanism, major parameter-control strategies, convergence behavior, neighborhood structures, discrete adaptations, parallel implementations, multiobjective extensions, hybrid algorithms, benchmarking practices, limitations, and future research directions.

Rather than proposing one new PSO algorithm, the article synthesizes a large body of previous research. It explains how PSO developed from models of collective animal behavior and how later researchers modified the basic method to address problems such as premature convergence, loss of diversity, parameter sensitivity, local-optimum stagnation, and difficulty with discrete or high-dimensional problems.

Historical Development

PSO is inspired by the coordinated movement of bird flocks and fish schools. The paper traces its intellectual development through several earlier computational models.

Particle Systems

An early influence was the use of particle systems to model natural phenomena such as:

- fire,
- clouds,
- water,
- and other irregular objects.

These systems represented natural structures using collections of moving particles rather than fixed geometric surfaces.

The Boids Model

A later influence was the Boids model, which simulated flocking through three simple rules:

1. **Separation** – avoid overcrowding nearby individuals.
2. **Alignment** – move in approximately the same direction as nearby individuals.
3. **Cohesion** – move toward the average position of neighboring individuals.

Complex group behavior emerged from these simple local interactions.

The paper notes that allowing every individual to communicate with every other individual creates approximately quadratic computational complexity. Restricting communication to local neighborhoods reduces this burden and creates a more scalable model.

Formal Introduction of PSO

PSO was formally introduced by James Kennedy and Russell Eberhart in 1995.

Early versions included mechanisms such as:

- nearest-neighbor velocity matching,
- random velocity perturbations,
- and flock-like behavioral rules.

Some of these mechanisms were later removed when researchers found that particles could still converge toward attractors using a simpler formulation based on personal and social experience.

Canonical PSO Mechanism

PSO uses a population called a **swarm**. Each member of the swarm is called a **particle**, and each particle represents one candidate solution.

A particle has:

- a current position,
- a current velocity,
- a personal best position,
- and access to a neighborhood or global best position.

At every iteration, the particle updates its velocity and position.

The velocity is influenced by three main components:

1. **Inertia component**
Preserves part of the particle's previous movement.
2. **Cognitive component**
Attracts the particle toward the best position it has personally discovered.
3. **Social component**
Attracts the particle toward the best position discovered by its neighborhood or the whole swarm.

The updated velocity is then added to the particle's current position.

If the new position has better fitness than the particle's previous personal best, that personal best is replaced. The global best is selected from the best personal positions found by the swarm.

The method uses no gradient information. It can therefore be applied to objective functions that are:

- nonlinear,
- nondifferentiable,
- discontinuous,
- poorly structured,
- or difficult to model mathematically.

Inertia Weight

The **inertia weight** controls how strongly a particle retains its previous velocity.

A high inertia weight generally promotes global exploration because particles travel farther and preserve more momentum. A low inertia weight promotes local exploitation because the social and cognitive attractors have greater control over movement.

The paper warns that:

- an excessively high inertia may cause particles to diverge,
- while an excessively low inertia may reduce exploration and cause early stagnation.

A commonly used strategy begins with a relatively high inertia weight and gradually lowers it. This allows broad exploration early in the search and more focused refinement later.

The paper reviews several inertia-weight strategies.

Random Selection

A new inertia value is randomly selected during each iteration, often from a predefined statistical distribution.

This introduces variability but requires care to maintain convergence.

Linear Time-Varying Inertia

The inertia weight decreases linearly from a maximum value to a minimum value over the search.

Frequently cited values are approximately:

- 0.9 initially,
- 0.4 near the end.

This is intended to move the swarm progressively from exploration to exploitation.

Nonlinear Time-Varying Inertia

The inertia weight decreases according to a nonlinear function.

Compared with linear reduction, this can give the algorithm more time in selected portions of the inertia range, often emphasizing local refinement later in the search.

Adaptive Inertia

The inertia value changes according to observed swarm performance, such as:

- relative fitness improvement,
- distance from local-best positions,
- swarm diversity,
- or iteration progress.

This attempts to respond to the current search condition rather than following a predetermined schedule.

Fuzzy-Adaptive Inertia

Fuzzy rules use information such as global-best fitness and current inertia to determine how the inertia weight should change.

The paper concludes that no one inertia strategy is universally best. Its effectiveness depends on the search space and objective function.

Velocity Clamping

Velocity clamping limits how far a particle can move during one iteration.

A large maximum velocity encourages exploration but may cause particles to overshoot good regions. A small maximum velocity promotes detailed search but may prevent the swarm from reaching distant areas.

The maximum velocity must therefore be chosen carefully.

The paper notes that inertia weight alone does not always eliminate the need for velocity limits.

Constriction Factor

A **constriction factor** is another mechanism for controlling velocity and promoting convergence.

It scales the complete velocity update according to mathematically derived conditions. Properly selected constriction parameters can cause particles to decelerate as the search progresses.

The constriction factor influences the exploration–exploitation balance:

- values favoring exploration may converge more slowly but potentially locate better solutions,
- values favoring exploitation may converge faster but produce more variable solution quality.

The paper reports that some studies found improved convergence when constriction was combined with velocity clamping.

Cognitive and Social Models

The paper examines reduced PSO models using only one of the two main attractors.

Cognitive-Only Model

Particles move based only on their own personal-best positions.

This encourages independent local search but provides weak information sharing. The reviewed studies found that this model:

- explores poorly beyond the initialization region,
- converges slowly,
- and may return suboptimal solutions.

Social-Only Model

Particles move only toward a global or neighborhood best.

This can produce rapid convergence. However, strong attraction toward one leader may cause premature convergence when that leader is located at a local rather than global optimum.

Combined Model

The canonical method combines cognitive and social attraction.

The paper argues that multimodal problems generally benefit from a balance between:

- individual experience,
- and collective knowledge.

Cognitive and Social Acceleration Coefficients

The cognitive and social coefficients determine how strongly particles are attracted toward personal-best and global-best positions.

The paper discusses commonly used equal values of approximately 2.05 for each coefficient. However, incorrect or extreme values may cause unstable or divergent behavior.

Alternative strategies include:

- decreasing the cognitive coefficient over time,
- increasing the social coefficient over time,
- adaptively changing both coefficients,
- and using fuzzy rules.

A time-varying strategy can encourage particles to rely more on personal exploration early in the search and more on collective convergence later.

Swarm Topologies

The swarm topology determines which particles exchange information.

Global-Best Topology

Every particle has access to the best solution found by the entire swarm.

Advantages include:

- rapid information spreading,
- and fast convergence.

Its main weakness is that the entire swarm may converge prematurely toward a poor global-best position.

Local-Best Topology

Each particle communicates only with a subset of the swarm.

This creates several local leaders rather than one global leader.

Advantages include:

- greater diversity,
- better global exploration,
- and lower risk of premature convergence.

The trade-off is slower convergence.

The article reviews several topology designs, including:

- ring structures,
- star structures,
- random topologies,
- Von Neumann neighborhoods,
- and toroidal arrangements.

It emphasizes that topology influences both information flow and convergence behavior.

Convergence Analysis

The paper reviews mathematical conditions under which particle trajectories converge toward an equilibrium point.

It notes an important distinction:

Convergence does not necessarily mean convergence to an optimum.

Particles may converge to a stochastic attractor determined by their personal-best and global-best positions. That point may be locally attractive but globally suboptimal.

This is one of the fundamental weaknesses of canonical PSO. A swarm may appear stable and converged even though it has failed to locate the true optimum.

Major Weaknesses of Canonical PSO

The paper identifies several recurring problems.

Premature Convergence

Particles may concentrate too quickly around a currently strong solution before other regions are properly explored.

Stagnation

Particle positions may stop improving, particularly when personal-best and global-best positions become similar.

Loss of Diversity

As particles converge, the swarm may lose the variety needed to investigate alternative regions.

Particle Explosion

Poor parameter choices may cause velocities to increase uncontrollably, making the swarm diverge.

Parameter Sensitivity

Different problems require different combinations of:

- inertia,
- velocity limits,
- acceleration coefficients,

- topology,
- population size,
- and stopping conditions.

Local-Optimum Convergence

The social nature of PSO can cause particles to follow an incorrect leader into a local optimum.

Much of the reviewed literature develops modifications intended to address these weaknesses.

Hybridization

A major portion of the paper surveys hybrid algorithms that combine PSO with other optimization methods.

The general purpose of hybridization is to combine PSO's population-based global exploration with another method's strengths, such as:

- local refinement,
- mutation,
- diversity preservation,
- constraint handling,
- or escape from local optima.

PSO and Genetic Algorithms

PSO has been combined with Genetic Algorithms through:

- sequential execution,
- parallel execution,
- exchanging high-fitness individuals,
- mutation,
- crossover,
- selection,
- and reproduction.

Some methods use PSO for rapid improvement and GA for diversity. Others run both algorithms simultaneously and exchange their best candidates.

Applications reviewed include:

- engineering design,

- neural-network design,
- electromagnetic optimization,
- power systems,
- clustering,
- investment portfolios,
- feature selection,
- classification,
- energy-demand forecasting,
- pavement inspection,
- engine optimization,
- and heliostat-field layout.

Reported findings commonly indicate faster convergence or better solution quality than using either PSO or GA alone, although results vary by problem.

PSO and Differential Evolution

Differential Evolution contributes difference-vector mutation and strong global exploration.

In hybrid DE–PSO methods:

- DE may relocate poorly performing particles,
- DE and PSO operators may alternate,
- particles may use DE-style difference vectors instead of cognitive attraction,
- or separate subpopulations may apply the two algorithms.

Applications include:

- benchmark optimization,
- image registration,
- filter design,
- reactive power dispatch,
- mechanics-parameter estimation,
- clustering,
- mobile-robot localization,
- photovoltaic maximum-power tracking,
- dynamic optimization,
- and transmission planning.

The reviewed studies frequently report improvements in:

- escape from local minima,
- convergence time,
- and final objective value.

Some hybrids, however, require more fitness evaluations.

PSO and Simulated Annealing

The paper surveys many combinations of PSO and Simulated Annealing.

The main reasoning is that:

- PSO supplies broad population-level exploration,
- while SA supplies strong local search and probabilistic escape from stagnation.

Some hybrids apply SA:

- whenever the global best stops improving,
- to selected particles,
- to all particles after a trigger,
- or as a second optimization phase.

Other methods use SA's Metropolis criterion to decide whether a new global-best candidate should replace the current one.

Applications include:

- partner selection,
- benchmark optimization,
- neural-network training,
- under-voltage load shedding,
- job-shop scheduling,
- Bayesian-network ordering,
- embedded-system energy reduction,
- tournament scheduling,
- electrical-power flow,
- process control,
- clustering,
- and hybrid-vehicle energy management.

The reviewed studies generally report that SA can help PSO avoid local stagnation, while PSO can accelerate the broader search.

PSO and Ant Colony Optimization

Ant Colony Optimization contributes pheromone-guided route construction and local refinement.

The reviewed hybrid methods use PSO to:

- tune ACO parameters,
- provide initial solutions,
- control pheromone behavior,
- identify promising global regions,
- or replace conventional pheromone updates.

ACO may then refine particle-generated solutions.

Applications include:

- continuous optimization,
- truss design,
- clustering,
- the Traveling Salesman Problem,
- vehicle routing,
- energy forecasting,
- phase-equilibrium estimation,
- container-truck routing,
- and communication-system detection.

Several route-oriented hybrids also include local procedures such as 3-opt or k-opt to improve tours.

PSO and Cuckoo Search

Cuckoo Search contributes Lévy-flight movement, which produces occasional long jumps through the search space.

This can improve PSO's ability to escape local optima.

PSO contributes:

- social communication,
- global-best guidance,
- and faster convergence.

Applications include:

- benchmark optimization,
- neural-network training,
- image classification,
- support-vector-machine parameter selection,

- maintenance optimization,
- and digital filter design.

The reviewed studies report that the combination can improve exploration while preserving PSO's convergence speed.

PSO and Artificial Bee Colony

Artificial Bee Colony methods contribute scout behavior and population diversification.

When PSO particles stagnate, bee-colony operators can introduce new candidate positions.

Hybrid methods may:

- run PSO and ABC in parallel,
- exchange best solutions,
- use ABC to update personal bests,
- select among PSO and bee phases according to stagnation,
- or combine PSO, ABC, and Differential Evolution.

Applications include:

- continuous optimization,
- medical classification,
- high-dimensional problems,
- and vehicle routing.

Several reviewed methods report improved diversity and faster convergence.

Other Hybridizations

The paper also discusses PSO combinations with:

- Artificial Immune Systems,
- Bat Algorithms,
- Firefly Algorithms,
- and Glowworm Swarm Optimization.

These combinations attempt to add:

- adaptive replacement of weak particles,
- alternative communication behavior,
- multi-peak detection,

- local refinement,
- and improved population diversity.

Applications include:

- complex layout design,
- transportation problems,
- medical-image registration,
- economic and environmental dispatch,
- and constrained global optimization.

Parallel PSO

PSO is naturally suitable for parallel execution because the fitness of many particles can be evaluated independently.

The paper distinguishes two common architectures.

Subswarm Parallelism

Different processors control separate subswarms. The subswarms periodically exchange information.

Master–Slave Parallelism

A master processor manages the algorithm and distributes fitness evaluations to several worker processors.

The reviewed studies report substantial acceleration, especially when fitness evaluation is computationally expensive.

Examples include:

- an eightfold speedup using ten processing nodes,
- approximately fortyfold GPU acceleration in a particle-filtering application,
- faster optimization of large graph-drawing problems,
- and clear GPU advantages as swarm size increased.

The paper emphasizes that parallel efficiency depends on:

- communication cost,
- load balance,
- topology,

- synchronization,
- and the cost of evaluating the objective function.

Niching and Multimodal Optimization

A multimodal problem contains several optima.

Canonical PSO may cause the entire swarm to converge to only one region. Niching methods divide the swarm into smaller groups so that different groups can locate different optima.

The paper reviews several strategies.

Fitness Stretching

Once an optimum is identified, the fitness landscape is modified to reduce its attraction and encourage discovery of other optima.

A limitation is that stretching can introduce artificial minima.

Deflection and Repulsion

Previously discovered optima repel particles or redirect the search toward new regions.

Neighborhood-Based Niches

Particles are grouped according to spatial proximity.

NichePSO

Subswarms are created when potential niches are detected.

Speciation

Particles within a threshold radius of a selected seed are treated as one species.

Adaptive Niching

The niche radius is estimated from distances between particles rather than fixed in advance.

Memory and Explorer Swarms

One swarm records good historical positions while another explores current positions.

The paper notes that niching methods can locate multiple optima but often introduce:

- additional parameters,
- high computational cost,
- quadratic interaction complexity,
- and poor scalability in high-dimensional spaces.

Dynamic Environments

In dynamic optimization, the objective landscape changes over time.

Optima may:

- move,
- change height,
- change shape,
- appear,
- or disappear.

Multiswarm and neighborhood-based PSO methods allow different groups to track separate moving optima.

However, dynamic niching remains difficult because of:

- frequent information exchange,
- parameter sensitivity,
- high-dimensional degradation,
- and the computational cost of maintaining several subpopulations.

Discrete Optimization

Canonical PSO is naturally continuous because positions and velocities are represented numerically.

The paper reviews several methods for adapting it to discrete problems.

Variable Rounding

Continuous positions are rounded to valid discrete values.

This is simple and may be fast, but poor rounding can send particles into infeasible or low-quality regions.

Binary PSO

Velocity is transformed through a sigmoid function into a probability that a binary variable becomes one.

The resulting position may represent:

- selected or unselected features,
- yes-or-no decisions,
- binary assignments,
- or graph choices.

Mutation and Boolean operations may be added to increase exploration.

Set-Based PSO

Solutions are represented as sets, while velocity represents possible changes to those sets.

Conventional arithmetic operators are replaced by set operations.

Applications reviewed include:

- the Traveling Salesman Problem,
- multidimensional knapsack problems,
- and vehicle routing with time windows.

Penalty Approaches

The objective function is modified to penalize positions far from permitted discrete values.

This makes a continuous search behave as though it were searching discrete options, but it can impose substantial computational costs.

Hybrid Discrete Approaches

PSO may be combined with:

- Branch and Bound,

- constraint-preserving procedures,
- nearest-vertex approximations,
- diversity mechanisms,
- or problem-specific decoding.

The reviewed applications include:

- maintenance scheduling,
- capacitor placement,
- unmanned-aerial-vehicle path planning,
- social-network influence maximization,
- project time–cost optimization,
- filter design,
- and network clustering.

The paper reports that some discrete PSO variants remain stable in high-dimensional cases where exact methods struggle, although success depends heavily on the chosen encoding.

Ensemble PSO

The article discusses ensemble methods in light of the **No Free Lunch theorem**, which states that no optimization algorithm is best across all possible objective functions.

An ensemble PSO maintains several:

- parameter strategies,
- movement rules,
- inertia schedules,
- or particle behaviors.

The algorithm selects or combines the strategy that appears most suitable during the search.

Examples include:

- Gaussian local search combined with differential mutation,
- heterogeneous particle behaviors,
- merit-based selection among PSO strategies,
- and ensembles of different inertia-weight schedules.

The reviewed studies generally find that heterogeneous or adaptive ensembles can outperform a uniform PSO, although they do not eliminate the basic weaknesses of the underlying method.

Benchmark Functions

The paper lists eight common benchmark functions:

1. Sphere
2. Schwefel 2.22
3. Schwefel 1.2
4. Rosenbrock
5. Schwefel 2.26
6. Rastrigin
7. Ackley
8. Griewank

These include:

- unimodal functions,
- simple multimodal functions,
- and unrotated multimodal functions.

The paper cautions that this set alone is insufficient for a comprehensive evaluation. A proper benchmark suite should also contain:

- rotated functions,
- composite functions,
- separable problems,
- nonseparable problems,
- different dimensions,
- and different landscape structures.

Reported Benchmark Results

The survey reproduces results from several PSO variants over the listed benchmark functions.

The values show that no one method clearly dominates every function. Hybrid variants may achieve extremely low errors on certain benchmarks but perform less strongly on others.

The paper stresses that tables of means should not be interpreted alone. Comparisons should include:

- standard deviation,
- repeated trials,
- statistical significance testing,
- and clearly stated computational budgets.

Some reviewed studies did not report standard deviations, limiting the conclusions that could be drawn from their results.

Performance-Comparison Practices

The paper discusses how deterministic and stochastic optimization methods should be compared.

Important concerns include:

- equal computational budgets,
- number of objective-function evaluations,
- multiple independent trials,
- local trial limits,
- global evaluation limits,
- varying convergence times,
- reinitialization strategies,
- and clearly defined success thresholds.

An algorithm may appear weak merely because the selected evaluation budget ends before its strongest phase. Conversely, a method may stagnate early and waste the remaining budget.

The paper describes comparison methods that use:

- a maximum budget for each individual run,
- a larger total budget across repeated runs,
- success probabilities,
- and aggregated performance regions.

It also refers to a large experiment involving:

- 134 constrained multimodal functions,
- 13 optimization methods,
- and more than 125,000 trials.

The survey argues that algorithm comparison should account for both efficiency and reliability rather than relying on one final fitness value.

Main Findings

The paper's major conclusions are:

- PSO is simple to understand and relatively easy to apply.
- It requires fewer adjustable components than many evolutionary algorithms.
- It can optimize functions without gradient information.

- It is applicable to continuous, constrained, unconstrained, and adapted discrete problems.
- Its social information exchange supports efficient global search.
- Its population structure makes it suitable for parallel computing.
- Performance depends strongly on inertia, velocity control, acceleration coefficients, topology, and population diversity.
- Canonical PSO is vulnerable to premature convergence, stagnation, local optima, and particle explosion.
- Convergence of particle positions does not guarantee convergence to the true optimum.
- Hybridization is widely used to compensate for PSO's weaknesses.
- Niching and multiswarm techniques improve multimodal search but become expensive and sensitive in high-dimensional problems.
- Discrete PSO requires specialized encoding or transformation.
- No PSO variant is universally superior across all functions.
- Fair evaluation requires diverse benchmarks, repeated trials, statistical testing, and controlled computational budgets.

Limitations Identified by the Authors

The survey highlights three major unresolved limitations.

Parameter Sensitivity

A parameter-setting strategy that succeeds on one problem may perform poorly on another.

PSO still requires substantial problem-specific adjustment.

Local-Optimum Convergence

Canonical PSO often converges prematurely in complex multimodal spaces unless it is significantly modified.

Weak High-Dimensional Multiobjective Performance

Niching and multiobjective techniques can work well at lower dimensions, but performance often deteriorates sharply as dimensionality increases.

The paper also identifies broader concerns:

- loss of diversity,
- unstable velocity growth,

- dependence on user-defined settings,
- inadequate theoretical unification,
- and lack of reliable self-adaptation.

Future Directions

The authors call for a more unified PSO framework that requires fewer manual adjustments.

Future research should focus on:

- intelligent self-adaptation,
- automatic parameter control,
- stronger diversity preservation,
- improved local-optimum escape,
- better high-dimensional scaling,
- theoretical treatment of particle explosion,
- robust convergence analysis,
- and methods that reduce problem-specific customization.

The authors view hybrid and ensemble methods as promising but argue that these methods often treat symptoms rather than fully resolving the weaknesses of canonical PSO.

Conclusion

The paper presents PSO as a flexible and influential global optimization paradigm whose appeal comes from its simplicity, cooperative population behavior, derivative-free operation, and broad applicability.

Its development has produced a large family of modifications involving:

- adaptive parameters,
- constriction,
- alternative topologies,
- parallel processing,
- niching,
- discrete encodings,
- hybrid methods,
- and ensemble strategies.

At the same time, the review concludes that PSO remains highly dependent on problem structure and parameter selection. Its tendency to lose diversity and converge prematurely continues to motivate further research.

The paper's overall position is that PSO is a powerful and adaptable optimization framework, but not a universally reliable method in its basic form.

Tabu Search: A Comparative Study

Harun Pirim, Engin Bayraktar, and Burak Eksioglu

Summary

This book chapter presents an introduction to **Tabu Search (TS)** and compares it with several other metaheuristic methods. Its main purposes are to:

- explain the basic concepts and operation of Tabu Search,
- introduce related optimization terminology,
- describe other major metaheuristics,
- review the problems to which Tabu Search has been applied,
- and compare its effectiveness and efficiency with alternative methods.

The chapter focuses particularly on difficult combinatorial optimization problems in areas such as:

- scheduling,
- vehicle routing,
- facility location,
- assignment,
- production planning,
- network design,
- and engineering design.

It is primarily a literature-based comparative review rather than a new experimental study. The authors examine approximately 150 publications, mainly from 2000 onward, and select representative studies showing different Tabu Search applications and comparisons.

Background: Difficult Optimization Problems

The chapter begins by explaining that many practical problems in scheduling, routing, and assignment are **NP-hard**. The number of possible solutions can grow so rapidly that complete enumeration becomes impractical.

For such problems, three broad approaches are available:

1. Use an exact enumerative method that guarantees an optimum.
2. Use a polynomial-time approximation method.

3. Use a heuristic or metaheuristic that seeks good solutions without guaranteeing optimality or a fixed runtime.

Local-search methods are widely used because they can find near-optimal solutions within reasonable amounts of time.

However, ordinary local search may stop at a poor local optimum. Metaheuristics such as Tabu Search and Simulated Annealing are designed to continue beyond the first local optimum.

Combinatorial Optimization

A combinatorial optimization problem involves selecting the best option from a finite or countably infinite set of discrete alternatives.

Candidate solutions may be represented as:

- sequences,
- permutations,
- graphs,
- partitions,
- assignments,
- or combinations of discrete decisions.

The chapter explains that many combinatorial problems can be viewed as searching for a particular permutation or structure.

The Traveling Salesman Problem is used as an example of rapid combinatorial growth. For a large number of cities, the number of possible tours becomes too large for complete examination.

Local and Global Optima

A **local optimum** is a solution that cannot be improved using the currently defined neighborhood moves.

A **global optimum** is the best solution in the entire solution space.

The chapter emphasizes that a local optimum can be far from the global optimum. The quality of local search therefore depends strongly on the neighborhood function.

A more powerful neighborhood may allow better solutions to be found, but it may also require substantially more computational time.

This creates a recurring trade-off between:

- neighborhood size,
- solution quality,
- and runtime.

Heuristics and Metaheuristics

A heuristic attempts to find a good solution but does not guarantee that the optimum will be found.

A metaheuristic is a more general framework that guides heuristic search and attempts to avoid becoming trapped at the first local optimum.

The chapter separates metaheuristics into two broad classes:

- **Single-trajectory methods**, which move through the solution space using one current solution, such as TS and SA.
- **Population-based methods**, which maintain several solutions simultaneously, such as GA, ACO, and PSO.

It also describes **hyperheuristics**, which select among several available heuristics during the optimization process.

Hill Climbing

Hill climbing, or iterative improvement, begins with an initial solution and repeatedly replaces it with a better neighbor.

The process stops when no better neighboring solution exists.

Its main weakness is local-optimum entrapment. The chapter identifies two possible responses:

- redesign the neighborhood,
- or allow nonimproving moves.

Tabu Search and Simulated Annealing both allow nonimproving moves, although they use very different mechanisms.

Neighborhood Functions

A neighborhood function defines which solutions are considered close to the current solution.

Common neighborhood operations include:

- swapping two elements,
- reversing part of a sequence,
- moving one element,
- replacing one decision,
- or displacing an item within a permutation.

The neighborhood structure must be designed according to the problem.

A large neighborhood may contain stronger improvements but require greater evaluation effort. In practice, algorithms often examine only a subset of the complete neighborhood.

Tabu Search Concepts

Short-Term Memory

Short-term memory stores recent search information for a limited number of iterations.

The tabu list is the main example. It records recent moves or attributes so that the search does not immediately reverse its decisions or revisit recently examined solutions.

Long-Term Memory

Long-term memory records broader historical information.

It supports:

- **intensification**, by directing the search toward patterns found in strong solutions,
- and **diversification**, by encouraging movement toward unexplored regions.

Tabu List

The tabu list records prohibited moves or move attributes.

A list that is too short may fail to prevent cycling. A list that is too long may block too many potentially useful moves.

The list length must therefore balance:

- cycling prevention,
- neighborhood freedom,
- and access to promising regions.

Tabu Tenure

Tabu tenure is the length of time for which a move or attribute remains tabu.

The chapter notes that appropriate tenure often depends on the size and structure of the problem.

Aspiration Criteria

An aspiration criterion allows a tabu move to be accepted when it is sufficiently desirable.

A common aspiration rule permits a tabu move when it produces a solution better than the best solution found so far.

This prevents the tabu mechanism from blocking exceptional improvements.

Recency-Based Memory

Recency memory records attributes modified during the recent past.

Its primary purpose is to prevent short cycles and immediate reversal of moves.

Frequency-Based Memory

Frequency memory records how often particular moves or attributes have been used.

Frequently used attributes can be penalized so that the search is encouraged to investigate different parts of the solution space.

This is usually associated with long-term diversification.

Quality-Based Memory

Quality memory distinguishes characteristics of good and bad solutions.

Attributes appearing regularly in strong solutions may be encouraged, while attributes associated with poor solutions may be penalized.

Explicit Memory

Explicit memory stores complete solutions, often elite solutions.

It can support intensification but may require substantial storage.

Attributive Memory

Attributive memory stores selected features rather than complete solutions.

It is more compact and can prohibit whole categories of moves through a single stored attribute.

Critical-Event Memory

Critical-event memory records significant events that occur during the search and creates an aggregated history of those events.

Strategic Oscillation

Strategic oscillation deliberately moves the search between different regions or conditions, such as:

- feasible and infeasible solutions,
- high and low values of a constraint,
- or intensified and diversified phases.

It is intended to expose opportunities that may not be found by remaining within one narrow region.

Path Relinking

Path relinking generates new solutions by exploring trajectories connecting high-quality solutions.

The method gradually transforms one elite solution toward another while examining intermediate possibilities.

Vocabulary Building

Vocabulary building identifies useful components or patterns from previous solutions and combines them into new candidate structures.

The chapter compares this concept with the generation of offspring in evolutionary algorithms.

Basic Tabu Search Procedure

A basic Tabu Search algorithm follows these steps:

1. Generate an initial solution.
2. Store it as the current and best-known solution.
3. Generate neighboring solutions.
4. Select the best admissible neighbor.
5. Allow a tabu move when it satisfies the aspiration criterion.
6. Replace the current solution.
7. Update the tabu list.
8. Repeat until a stopping condition is reached.

Unlike hill climbing, the selected neighbor does not have to improve the current solution.

Tabu Search may deliberately accept the least damaging worsening move to escape a local optimum.

The chapter describes TS as using:

- **adaptive memory**, because its decisions depend on search history,
- and **responsive exploration**, because the direction of the search changes according to the observed results.

Memory-Matrix Example

The chapter illustrates a matrix-based Tabu Search implementation for the Traveling Salesman Problem.

An upper triangular matrix stores **recency memory**, indicating how many more iterations particular city exchanges remain tabu.

A lower triangular matrix stores **frequency memory**, indicating how often each exchange has occurred.

This allows the algorithm to check prohibited exchanges and track commonly used moves efficiently.

The example shows how a solution can improve from one iteration to the next while the recency and frequency matrices are updated.

Simulated Annealing

The chapter describes Simulated Annealing as a randomized single-trajectory method inspired by the physical annealing of solids.

It accepts all improving moves and may accept a worsening move according to a probability determined by:

- the change in objective value,
- and a temperature parameter.

At high temperature, worse moves are accepted more readily, encouraging exploration.

As temperature decreases, the algorithm becomes more selective and concentrates on stronger solutions.

The **cooling schedule** controls how temperature changes and how long each temperature is maintained.

The chapter emphasizes that SA performance depends heavily on the cooling schedule.

Genetic Algorithms

Genetic Algorithms are population-based methods inspired by biological evolution.

Their main components include:

- chromosomes representing candidate solutions,
- a fitness function,
- selection,
- crossover,
- mutation,
- reproduction,
- and a termination rule.

A population is evaluated, stronger candidates are selected, and new generations are produced.

The chapter notes that crossover can generate diverse solutions but may also produce infeasible candidates.

GAs are applied in many fields, including:

- scheduling,
- routing,

- robotics,
- materials engineering,
- aerospace,
- biology,
- and chemistry.

Ant Colony Optimization

Ant Colony Optimization is inspired by the indirect communication of ants through pheromone trails.

Artificial ants construct solutions, and successful components receive stronger pheromone values.

The main stages are:

1. Generate solutions.
2. Update pheromone values.
3. Apply centralized or daemon actions.
4. Repeat until termination.

For routing problems, the probability of choosing the next city depends on:

- pheromone intensity,
- visibility or heuristic desirability,
- and parameters controlling their relative influence.

Pheromone evaporation prevents the search from concentrating permanently on one path.

GRASP

The Greedy Randomized Adaptive Search Procedure contains two phases:

1. **Construction phase**
A solution is built using a greedy randomized method. Strong candidate elements are placed in a restricted candidate list.
2. **Local-search phase**
The constructed solution is improved through neighborhood search.

GRASP repeats these phases and retains the best result.

The chapter mentions an example where GRASP generated good routes for a 70-city Traveling Salesman instance after evaluating thousands of solutions.

Particle Swarm Optimization

PSO is described as a population-based method inspired by collective animal behavior.

Each particle has:

- a position,
- a velocity,
- an individual best position,
- and a neighborhood best position.

Movement depends on:

- the particle's previous velocity,
- individual experience,
- and social communication.

The chapter notes applications of PSO to:

- Traveling Salesman problems,
- vehicle routing,
- flow-shop scheduling,
- and neural-network training.

Classification of the Metaheuristics

The chapter classifies the methods according to several features.

Tabu Search

- Adaptive memory
- Special neighborhood
- Primarily single-trajectory
- Can also be extended with population elements

Simulated Annealing

- Memoryless
- Random sampling
- Single-trajectory search

Genetic Algorithms

- Memoryless
- Randomized
- Population-based

Ant Colony Optimization

- Randomized
- Population-based
- Uses collective pheromone information

GRASP

- Randomized
- Single-solution construction and local search

Particle Swarm Optimization

- Randomized
- Population-based
- Uses social information exchange

The authors emphasize that implementation success depends on how effectively each general method is adapted to the specific problem.

Tabu Search Applications

The literature review covers a wide variety of applications.

Flow-Shop Scheduling

Several studies applied TS to flow-shop scheduling.

One comparison among TS, SA, and Variable Neighborhood Search used 20 problem instances. TS parameters were tuned, including:

- job-interchange neighborhoods,
- termination conditions,
- and tabu-list length.

Tuning improved TS performance by approximately 7%, but SA produced better schedules in less time in that particular study.

Other flow-shop research found that performance depended on:

- problem size,
- desired solution quality,
- and available runtime.

Reactive TS, which adjusts the tabu-list length dynamically, obtained strong results without extensive parameter tuning.

A changing-neighborhood TS using three neighborhood structures produced solutions comparable to neuro-TS and ACO. It required approximately half the computation time of neuro-TS and produced solutions closer to optimality than ACO.

A hybrid TS for re-entrant flow shops found optimal solutions for all small test problems and increasingly outperformed pure TS as problem size grew.

Job-Shop and Open-Shop Scheduling

The chapter reviews TS methods using:

- new neighborhood structures,
- shifting bottlenecks,
- hybrid genetic components,
- and specialized schedule representations.

One comparison found TS superior to GA in both solution quality and computation time.

A TS for two-machine preemptive open-shop scheduling found optimal solutions in most cases. In the remaining cases, the average deviation from the optimum was approximately 2%.

However, another mixed-model sequencing study reported that SA outperformed TS in most test cases.

Parallel-Machine Scheduling

TS was applied to:

- identical parallel-machine scheduling,
- tardiness minimization,
- and due-date-based categorization.

The chapter reports several successful applications but does not claim uniform superiority across every formulation.

Project Scheduling

Applications include:

- discrete–continuous project scheduling,
- resource-constrained project scheduling,
- discounted cash-flow scheduling,
- and schedule-dependent setup times.

TS generally produced results close to optimality and outperformed simpler search or random-sampling methods.

Timetabling and Rostering

A TS hyperheuristic produced more feasible schedules than GA, although GA produced lower costs in some cases.

This illustrates that the preferred method may depend on which evaluation criterion is prioritized.

Maintenance Scheduling

A hybrid memetic algorithm using TS improved solution quality but required slightly more runtime.

Vehicle Routing Applications

TS is widely applied to vehicle-routing problems.

Stochastic-Demand Vehicle Routing

A comparison among TS, SA, and threshold accepting used the same neighborhood structure.

The study found that:

- TS produced better solution quality,
- TS required less computation time than SA,
- and the advantage of TS increased with problem size.

The quality of the initial solution had a positive effect on TS but not the other methods.

Multi-Compartment Vehicle Routing

TS produced slightly better solutions than a memetic algorithm but required more computation time.

Open Vehicle Routing

A comparison of 11 methods found that adaptive large-neighborhood search, record-to-record travel, and TS all performed well.

Unmanned Aerial Vehicle Routing

TS generated feasible tours rapidly after assignment decisions had been made.

Its iterative design allowed the search to be stopped once an acceptable route was available.

Real-Time Material-Handling Routes

A parallel TS produced better routes than simpler procedures, although generally with greater computing time.

Vehicle Routing with Loading Constraints

TS was also applied to problems combining routing with two-dimensional loading restrictions.

Facility-Location Applications

The chapter reviews comparisons of TS, SA, and GA across several facility-location problems.

In one study:

- GA performed worst overall.
- TS produced better solutions than SA in 28 of 37 instances.
- TS and SA required approximately similar computation times.

However, the results changed according to the evaluation condition.

Time-Limited Comparison

TS performed best for capacitated and multiperiod facility-location problems.

It reached low-cost solutions faster than SA and GA.

Solution-Limited Comparison

When each algorithm was allowed to examine the same number of solutions, SA performed similarly to or better than TS.

Unrestricted Comparison

TS performed best for some facility-location classes, while SA performed best for another.

This demonstrates that conclusions depend on whether the comparison controls:

- time,
- number of evaluated solutions,
- or unrestricted convergence.

For one multicommodity problem, GA performed best because its representation reduced the number of candidate solutions requiring evaluation.

Arc Routing

The chapter reviews deterministic TS methods for capacitated arc-routing problems.

TS produced high-quality results within reasonable computation time.

One study preferred deterministic TS because its results were reproducible. Competing randomized algorithms returned different results on repeated runs.

The authors treat reproducibility as an advantage of deterministic Tabu Search.

Network Design

TS applications include:

- computer-network concentrator placement,
- self-healing ring design,
- subsystem expansion,
- and communication-network configuration.

For network-design problems containing more than 100 users, one TS method produced better results than CPLEX.

Production and Distribution

Applications include:

- integrated newspaper production and distribution,
- production planning with sequence-dependent setups,
- cutting and packing,
- cellular manufacturing,
- and production systems with multiple objectives.

A newspaper logistics application reduced the required number of delivery vehicles by approximately **18.18%**.

In cellular manufacturing, TS addressed:

- intercellular movement,
- cell-load variation,
- and combined objectives.

The method also allowed designers to specify limits on the number of cells and machines.

Engineering and Scientific Applications

The chapter lists many other areas, including:

- circuit partitioning,
- solenoid design,
- mobile communication cell planning,
- embedded processing systems,
- laminate design,
- electrical unit commitment,
- DNA sequencing,
- postal routing,
- groundwater modeling,
- graph coloring,
- redundancy allocation,
- and continuous-function optimization.

A circuit-partitioning study found that TS converged quickly and substantially reduced execution time compared with a random-move SA method while retaining good solution quality.

A reactive TS graph-coloring method produced good results on difficult benchmark graphs.

Comparative Findings for Scheduling

The reviewed scheduling studies do not show one universal winner.

Reported outcomes include:

- TS outperforming GA in job-shop scheduling,
- hybrid TS outperforming pure TS on larger re-entrant flow-shop problems,
- TS variants outperforming simpler search methods,
- SA outperforming TS in some two-machine and mixed-model scheduling problems,
- and PSO or ACO performing competitively in selected settings.

The chapter nevertheless concludes that pure and hybrid TS methods often perform strongly in:

- flow shops,
- job shops,
- project scheduling,
- and general scheduling.

Comparative Findings for Routing

In several routing studies, TS produced higher-quality solutions than SA and threshold-accepting methods.

However, better quality sometimes came with longer runtime.

The reviewed evidence suggests that TS becomes particularly competitive as problem size increases, although this is not universal across all routing formulations.

Why Tabu Search Can Perform Well

The authors identify several characteristics that may explain TS performance.

Aggressive Movement Toward Strong Regions

TS commonly selects the best admissible neighbor and moves quickly toward good local regions.

By contrast, SA relies on gradual probabilistic descent and may spend more time exploring weaker regions.

Adaptive Memory

TS remembers recent decisions and broader search patterns.

This reduces unnecessary revisiting and allows intensification and diversification to be performed strategically.

Flexible Constraint Handling

TS can incorporate problem-specific constraints into:

- neighborhood design,
- tabu restrictions,
- aspiration rules,
- and candidate lists.

This makes it useful for highly constrained problems.

Anytime Behavior

TS can be stopped after a feasible or acceptable solution is found.

The chapter argues that SA may be harder to interrupt meaningfully because its temperature should approach a sufficiently low level.

Deterministic Reproducibility

Deterministic TS variants can return the same result when run under the same conditions.

This contrasts with stochastic algorithms whose results vary across runs.

Fine-Tuning Capability

TS can work well as a local-refinement component within a hybrid method.

The chapter gives examples in which GA performs global exploration while TS fine-tunes selected solutions.

Weaknesses of Tabu Search

The chapter also identifies several disadvantages.

Parameter Tuning

TS may require careful selection of:

- tabu-list length,
- neighborhood structure,
- stopping conditions,
- candidate-list size,
- aspiration criteria,
- penalty coefficients,
- and intensification or diversification rules.

The same parameter settings do not necessarily work across problems.

Neighborhood-Evaluation Cost

TS may evaluate many neighboring solutions during each iteration.

This can make it slower than SA, which may generate only one candidate per iteration.

Dependence on Problem Design

A successful TS requires an effective representation and neighborhood structure.

Poorly designed tabu restrictions may prevent useful moves without eliminating harmful cycling.

Lack of Universal Benchmarks

The authors note that optimization algorithms are often tested on benchmarks that fail to capture the full structure of real-world problems.

This makes broad claims of superiority difficult to justify.

Iterative Development

The first implementation of a TS method should not be expected to be ideal.

Effective designs often require repeated adjustment according to computational results.

Comparison with Simulated Annealing

The chapter contrasts the methods in several ways.

Search Decisions

- TS selects the best admissible move from a neighborhood.
- SA usually generates one candidate and accepts it according to a probability.

Memory

- TS uses adaptive memory.
- Standard SA is memoryless.

Nonimproving Moves

- TS permits worsening moves strategically.
- SA permits them probabilistically according to temperature.

Runtime

TS can often be stopped at any point with the best solution retained.

SA generally depends on completing a meaningful cooling schedule.

Variability

Deterministic TS may be reproducible.

SA results vary because of random move selection and probabilistic acceptance.

Computational Cost

TS may require more work per iteration because it evaluates a neighborhood.

SA may perform cheaper iterations but require many iterations and careful cooling.

The reviewed studies contain examples of each method outperforming the other.

Comparison with Genetic Algorithms

GA maintains and evaluates a population, which can create substantial computational cost for complex objective functions.

TS works from one current solution and its neighborhood.

The authors suggest that:

- GA can provide strong global exploration,
- TS can perform effective fine-tuning,
- and combining them may outperform either method alone.

GA can also perform well where little structural information is available or where an efficient population representation reduces the number of candidate solutions.

Comparison with Particle Swarm Optimization

The chapter reports that TS and TS hybrids produced better results than PSO in a no-wait flow-shop scheduling study.

However, the comparison is based on reviewed applications rather than one universal theoretical result.

PSO differs fundamentally because it is:

- population-based,
- stochastic,
- and driven by individual and social experience.

TS instead follows a memory-guided trajectory through a neighborhood.

Comparison with Ant Colony Optimization

Several scheduling comparisons found TS and ACO to require similar computation time.

In one changing-neighborhood flow-shop study, TS produced solutions closer to the optimum.

ACO may be particularly natural for route-construction problems, while TS can refine solutions using problem-specific neighborhood moves.

Hybrid Methods

The chapter emphasizes the increasing use of hybrid metaheuristics.

Examples include:

- TS with GA,
- TS with memetic algorithms,
- TS with Branch and Bound,
- TS with constraint programming,
- TS with neural methods,
- and TS with problem-specific constructive heuristics.

A Tabu Genetic Algorithm incorporated TS aspiration principles into GA selection. It reportedly outperformed:

- GA,
- TS,
- and conventional alternating GA–TS hybrids

in both solution quality and convergence speed.

The authors suggest that hybrid methods often outperform pure algorithms because they combine:

- global exploration,
- local refinement,
- constraint handling,
- and memory.

Main Findings

The chapter's main findings are:

- Tabu Search is a flexible memory-based framework for difficult optimization problems.
- It escapes local optima by allowing nonimproving moves.

- Short-term memory prevents cycling.
- Long-term memory supports intensification and diversification.
- Aspiration criteria prevent valuable moves from being unnecessarily prohibited.
- Performance depends heavily on neighborhood design and parameter tuning.
- TS has been applied successfully to scheduling, routing, location, network, production, and engineering problems.
- Many studies report TS outperforming SA, GA, PSO, or simpler heuristics.
- Other studies report SA or GA performing better under certain objectives and resource limits.
- Algorithm rankings change according to problem size, runtime limits, solution-evaluation limits, and required quality.
- Deterministic TS offers reproducibility, but may require costly neighborhood evaluation.
- Hybrid TS methods frequently outperform pure TS.
- No single metaheuristic is uniformly superior across every problem.

Important Limitations of the Chapter

The chapter is a broad literature review, not a controlled experiment.

The studies summarized use different:

- datasets,
- hardware,
- neighborhoods,
- stopping conditions,
- parameter settings,
- representations,
- and evaluation measures.

Consequently, the reported comparisons cannot always be treated as directly equivalent.

The chapter generally favors Tabu Search and sometimes draws broad conclusions from heterogeneous studies. Nevertheless, it also acknowledges several cases where SA, GA, or another method performed better.

Conclusion

The authors conclude that Tabu Search is one of the most useful metaheuristics for difficult combinatorial optimization problems.

Its defining strength is adaptive memory, which allows it to:

- avoid cycles,
- learn from previous solutions,
- concentrate on promising regions,
- and deliberately explore unfamiliar areas.

The literature reviewed shows particularly extensive use in production, scheduling, and logistics.

However, strong performance depends on problem-specific design. Tabu Search must be adapted through suitable:

- neighborhoods,
- tabu tenures,
- aspiration criteria,
- candidate lists,
- and hybrid components.

The authors ultimately support continued research into hybrid metaheuristics and more sophisticated models of memory, including ideas inspired by how human memory contributes to problem solving.

Empirical Analysis of Metaheuristic Search Techniques for the Parameterized Dynamic Slope Scaling Procedure

Weili Zhang and Charles D. Nicholson

Summary

This paper examines how three metaheuristic search methods—**Simulated Annealing**, **Tabu Search**, and **Particle Swarm Optimization**—can be used to improve a parameterized approximation procedure for the **Fixed-Charge Network Flow problem**.

The paper does not use the three metaheuristics to construct network-flow solutions directly. Instead, they are used to search for a good value of a numerical parameter, denoted by ψ , that changes how the underlying Dynamic Slope Scaling Procedure behaves.

The authors compare the methods according to:

- final solution quality,
- improvement over the original procedure,
- computation time,
- number of parameter evaluations,
- improvement obtained per iteration,
- performance across different problem sizes,
- and sensitivity to network characteristics.

The central finding is that all three metaheuristics eventually obtain similar-quality solutions, but **Particle Swarm Optimization is generally more efficient**, particularly for larger network instances.

Fixed-Charge Network Flow Problem

The Fixed-Charge Network Flow problem, or FCNF, involves moving commodities through a network at minimum cost.

The network contains:

- nodes representing supply, demand, or intermediate transfer locations,
- directed arcs connecting the nodes,
- variable costs depending on the amount of flow,

- and fixed costs incurred whenever an arc is used.

The optimization problem must determine:

- which arcs should be opened,
- and how much flow should be sent through each open arc.

The total cost consists of:

- variable transportation costs,
- plus the fixed costs of using selected arcs.

The fixed-charge component requires binary decisions, making FCNF a mixed-integer and NP-hard problem.

The paper notes that FCNF includes or relates to many practical problems, such as:

- transportation,
- facility location,
- lot sizing,
- production planning,
- supply-chain network design,
- and distribution systems.

Exact and Approximate Solution Methods

Branch and Bound is commonly used to obtain exact solutions to mixed-integer problems.

It repeatedly solves relaxed versions of the problem and maintains:

- lower bounds from relaxations,
- and upper bounds from feasible integer solutions.

The method approaches optimality as the gap between these bounds decreases.

However, Branch and Bound can become inefficient when the relaxations are weak, particularly when fixed costs are high. This motivates the use of approximation and heuristic methods.

Dynamic Slope Scaling Procedure

The **Dynamic Slope Scaling Procedure**, or DSSP, is an approximation method designed for FCNF.

Its main idea is to remove the binary arc-opening variables and replace the original fixed and variable costs with an iteratively updated linear cost coefficient.

The fixed cost of an arc is divided by its flow from the previous iteration. This produces an approximate per-unit cost.

The algorithm repeatedly:

1. Calculates linearized arc costs using the previous flow.
2. Solves the resulting linear network-flow problem.
3. Updates the linearized costs.
4. Evaluates the resulting solution using the original FCNF objective.
5. Repeats until the flow values stop changing or no further improvement occurs.

The best solution found across all DSSP iterations is returned, which is not necessarily the solution from the final iteration.

Parameterized Dynamic Slope Scaling Procedure

The paper studies a modified method called the **Parameterized Dynamic Slope Scaling Procedure**, written as Ψ -DSSP.

This method introduces an additional multiplier, ψ , into the linearized fixed-cost term.

The parameter changes the influence of fixed cost during each DSSP iteration. Even a small change in ψ can alter:

- the estimated arc costs,
- the selected network paths,
- the search trajectory,
- and the final solution.

Previous research tested only a few fixed ψ values, including:

- 0.25,
- 0.50,
- 0.75,
- 1.00,
- and 1.25.

Those tests produced improvements of up to approximately 30% over ordinary DSSP.

However, the best ψ value was different for different network instances, and there was no simple pattern for choosing it in advance.

The current paper therefore treats ψ selection as a continuous optimization problem.

Research Objectives

The paper identifies three principal contributions.

More Precise Parameter Search

The authors investigate whether searching a finer range of ψ values can improve solution quality beyond what was achieved using only a few predetermined settings.

Comparison of Metaheuristics

The study compares:

- Simulated Annealing,
- Tabu Search,
- and Particle Swarm Optimization

as search methods for selecting ψ .

Evaluation Across Difficult Network Instances

The methods are tested on FCNF instances with different:

- numbers of nodes,
- numbers of arcs,
- network densities,
- supply and demand structures,
- and cost characteristics.

The authors analyze both solution quality and search efficiency.

Simulated Annealing Implementation

The paper uses two forms of Simulated Annealing:

1. **Boltzmann Simulated Annealing**
2. **Very Fast Annealing**

The algorithm begins with:

- an initial ψ value,
- the corresponding Ψ -DSSP solution,
- and an initial temperature.

At each iteration, a nearby candidate ψ value is generated.

If the candidate gives a lower network cost, it is accepted.

If the candidate is worse, it may still be accepted according to a probability determined by:

- its percentage deterioration,
- and the current temperature.

The use of percentage deterioration rather than an absolute cost difference makes the acceptance behavior more comparable across network problems with different cost magnitudes.

Boltzmann Cooling

The Boltzmann temperature decreases according to a logarithmic function.

This produces relatively slow cooling and therefore a gradual reduction in exploration.

Candidate values are generated partly through a normally distributed random variable and scaled according to temperature.

Very Fast Annealing

Very Fast Annealing uses an exponential cooling rule.

Its neighborhood-generation rule produces candidate movements whose range also changes with temperature.

The two versions therefore differ in both:

- how temperature decreases,
- and how candidate ψ values are sampled.

Dwell Parameter

If a candidate is rejected, SA may evaluate several additional candidates before lowering the temperature.

The maximum number of candidate attempts at one temperature is controlled by a dwell parameter.

In the experiment, this was set to three.

Stopping Conditions

The SA procedures stop when they reach:

- the maximum number of iterations,
- the maximum runtime,
- or an early-termination condition caused by lack of improvement.

Tabu Search Implementation

Tabu Search is adapted to a **one-dimensional continuous search space** because ψ is a real-valued number.

Instead of using ordinary discrete moves, the method divides the neighborhood around the current ψ value into several concentric distance bands.

For example, one band contains candidates close to the current value, while other bands contain increasingly distant candidates.

At each iteration:

1. Several distance bands are created around the current ψ .
2. One candidate is randomly selected from each band.
3. Candidates too close to tabu values are excluded.
4. Ψ -DSSP is evaluated for each admissible candidate.
5. The candidate with the lowest network cost becomes the next current value.
6. The chosen value is added to the tabu list.
7. The oldest tabu value is removed when the list becomes too long.

This structure allows the search to examine both:

- local candidates,
- and more distant candidates

during the same iteration.

The implementation uses a tabu list of recently selected ψ values rather than complete network-flow solutions.

Particle Swarm Optimization Implementation

The PSO implementation uses a population of particles moving along the one-dimensional ψ interval.

Each particle has:

- a current ψ position,
- a velocity,
- its own historically best position,
- and access to the swarm's globally best position.

At every iteration:

1. Each particle's current ψ value is evaluated using Ψ -DSSP.
2. Its personal best is updated when appropriate.
3. The global best is updated.
4. Velocity is adjusted using inertia, personal-best attraction, and global-best attraction.
5. Position is updated by adding the new velocity.

Inertia Weight

The inertia weight decreases linearly from a maximum to a minimum value.

The intention is to encourage:

- broader search early in the procedure,
- and stronger convergence later.

Acceleration Coefficients

Two acceleration coefficients represent:

- attraction toward the particle's own best value,
- and attraction toward the swarm's best value.

Random multipliers create stochastic variation in these movements.

Initial Population

Particle positions are generated uniformly within the ψ interval from 0 to 2.

Initial velocities are randomly generated within a maximum velocity range.

Experimental Dataset

The study tests the methods on **180 generated FCNF instances**.

The instances are divided into three network sizes:

- 25 nodes,
- 50 nodes,
- and 100 nodes.

For each network size, 60 feasible instances are generated.

The number of arcs varies randomly, producing networks with different densities.

Each undirected connection is represented by two directed arcs.

Network Structure

Nodes are assigned as:

- supply nodes,
- demand nodes,
- or transshipment nodes.

Approximate probabilities are:

- 20% supply,
- 20% demand,
- and 60% transshipment.

Adjustments are made when necessary to ensure feasibility.

Variable arc costs are generated from a relatively small numerical range, while fixed costs are much larger.

This produces difficult instances with a high fixed-to-variable cost ratio, which are known to be challenging for ordinary DSSP.

Supply quantities are randomly assigned and then distributed among the demand nodes.

The linear subproblems are solved using Gurobi.

Network Characteristics Measured

The authors calculate several network descriptors.

Density

Density represents the proportion of possible arcs that are present.

The test networks range from very sparse to very dense.

Percentage of Supply Nodes

This measures the proportion of nodes providing commodity flow.

Percentage of Demand Nodes

This measures the proportion receiving flow.

Average Supply

This is the average amount provided by each supply node.

Fixed-to-Variable Cost Ratio

This compares the overall magnitude of fixed costs with variable costs.

The average ratio is approximately 4,000, reflecting the deliberately high fixed-cost structure.

Gamma Characteristic

A combined measure, γ , compares average supply with the fixed-to-variable cost ratio.

A high value indicates that variable costs are likely to have greater influence. A low value suggests stronger fixed-cost influence.

The generated instances mostly have relatively low γ values, which is consistent with fixed costs strongly influencing the problem.

Parameter Settings

The authors attempt to use reasonable and comparable settings rather than exhaustively tuning every possible configuration.

Simulated Annealing

- Maximum candidate attempts per temperature: 3
- Initial temperature: 0.25

Tabu Search

- Minimum candidate separation: 0.01
- Maximum neighborhood distance: 0.20
- Tabu-list length: 5
- Number of candidate neighbors per iteration: 5

Particle Swarm Optimization

- Number of particles: 10
- Maximum velocity: 1
- Minimum inertia: 0.4
- Maximum inertia: 0.9
- Personal acceleration coefficient: 2
- Social acceleration coefficient: 2

Stopping Rules

All methods use three possible stopping conditions.

Maximum Runtime

Each method is permitted to run for up to one hour.

Maximum Iterations

The iteration limit is scaled according to the time required to solve an ordinary DSSP instance.

Early Termination

The algorithm stops when the objective value has not improved for a specified number of iterations.

The authors note that early termination is useful only when the resulting solution is strong. Quick convergence to a poor local optimum should not be regarded as efficient.

Distribution of Best ψ Values

The best ψ values found by each method span nearly the entire interval from 0 to 2.

The histograms show that no single value or narrow range dominates across all network instances.

Some concentration occurs near lower ψ values, particularly for PSO, but strong solutions also occur throughout the interval.

This confirms that a fixed universal ψ value would not perform best for all FCNF instances.

Improvement over Ordinary DSSP

The parameterized procedure guided by metaheuristic search improves upon ordinary DSSP in approximately **94% to 95% of cases**.

Across all methods and experiments, the reported overall average improvement is approximately **10% to 12%**.

The paper reports:

- an overall mean improvement of approximately 11.83% in one aggregate calculation,
- average per-method improvements of roughly 10%,
- and a maximum improvement of approximately 24.2%.

The amount of improvement ranges from approximately:

- 0.2% at the low end,
- to 24.2% at the high end.

Results by Network Size

The percentage of instances improved by Ψ -DSSP increases as network size grows.

25 Nodes

Approximately 86% of the instances are improved.

Average improvement is around 10% to 11%.

50 Nodes

Approximately 98.3% of the instances are improved.

Average improvement remains around 10%.

100 Nodes

All tested instances are improved.

Average improvement remains approximately 9% to 10%.

Although improvements become more consistently present in larger networks, the size of the improvement does not significantly differ across the three node levels.

The reported p-value for the difference in improvement magnitude by network size is 0.91, indicating no statistically significant effect.

Results by Metaheuristic

The final solution-quality results are extremely similar.

Boltzmann Simulated Annealing

- Improvement in approximately 95% of instances
- Mean improvement of approximately 10.2%
- Maximum improvement of approximately 23.9%

Very Fast Annealing

- Improvement in approximately 95% of instances
- Mean improvement of approximately 10.1%
- Maximum improvement of approximately 24.2%

Tabu Search

- Improvement in approximately 95% of instances
- Mean improvement of approximately 10.1%
- Maximum improvement of approximately 24.2%

Particle Swarm Optimization

- Improvement in approximately 94.4% of instances
- Mean improvement of approximately 10.3%
- Maximum improvement of approximately 23.9%

The differences in average final solution quality are not statistically significant.

Thus, none of the four implementations clearly dominates in terms of the final cost obtained.

Correlation with Network Characteristics

The authors test whether the amount of improvement is related to network structure.

Network Density

Density is the only characteristic with a statistically significant correlation with improvement.

The correlation is positive for every search method.

This means that as the network becomes denser and more complex, the parameterized procedure tends to outperform ordinary DSSP by a larger amount.

The correlation is moderate rather than extremely strong, with coefficients approximately between 0.23 and 0.31.

Other Characteristics

No statistically significant relationship is found between improvement and:

- percentage of supply nodes,

- percentage of demand nodes,
- average supply,
- fixed-to-variable cost ratio,
- or the γ characteristic.

The benefits of metaheuristic parameter search are therefore relatively robust across the tested ranges of these network properties.

Relationship Between ψ and Improvement

The authors find essentially no linear correlation between the numerical ψ value and the amount of improvement.

The reported Pearson correlation is approximately **0.004**.

The scatterplot shows strong and weak improvements throughout nearly the entire ψ range.

This means that:

- larger ψ values are not consistently better,
- smaller ψ values are not consistently better,
- and the best parameter cannot be predicted using a simple monotonic rule.

Efficiency Measures

The authors compare efficiency using:

- CPU time,
- number of evaluated Ψ -DSSP solutions,
- and improvement per iteration.

The improvement-per-iteration measure divides the percentage cost improvement by the number of parameter evaluations.

This attempts to distinguish genuinely efficient search from a method that obtains good results only by conducting many evaluations.

Results for 25-Node Networks

For the smallest problems, average runtimes are approximately:

- 14 seconds for Boltzmann SA,
- 16 seconds for Very Fast Annealing,
- 11 seconds for TS,
- and 7 seconds for PSO.

PSO is significantly faster than TS, which is significantly faster than both SA versions.

PSO also has the highest improvement per iteration.

The efficiency ordering is approximately:

1. PSO
2. TS
3. Boltzmann SA and Very Fast Annealing

Results for 50-Node Networks

Average runtimes are approximately:

- 249 seconds for Boltzmann SA,
- 282 seconds for Very Fast Annealing,
- 166 seconds for TS,
- and 249 seconds for PSO.

However, the statistical tests find no significant difference among the methods for either:

- runtime,
- or improvement per iteration.

For these medium-sized networks, all four approaches perform similarly from an efficiency perspective.

Results for 100-Node Networks

For the largest problems, average runtimes are approximately:

- 2,889 seconds for Boltzmann SA,
- 4,114 seconds for Very Fast Annealing,
- 1,934 seconds for TS,
- and 1,597 seconds for PSO.

PSO and TS converge after approximately 30 minutes on average.

The SA versions take substantially longer:

- approximately 48 minutes for Boltzmann SA,
- and approximately 1 hour and 9 minutes for Very Fast Annealing.

PSO and TS are statistically faster than the SA implementations.

PSO also produces the highest improvement per iteration.

Statistical Analysis

The paper uses:

- Pearson correlation analysis,
- Analysis of Variance,
- and Tukey Honest Significant Difference tests.

These tests are used to determine whether observed differences are statistically meaningful rather than simply numerical fluctuations.

The main statistical conclusions are:

- no significant difference in final solution quality across the metaheuristics,
- no significant effect of network size on the magnitude of improvement,
- a significant positive relationship between network density and improvement,
- and significant efficiency advantages for PSO in small and large instances.

Main Findings

The paper's major findings are:

- Fine-tuning the ψ parameter substantially improves Ψ -DSSP solutions.
- Approximately 95% of the tested FCNF instances benefit from parameter search.
- Average solution improvement is approximately 10%.
- Maximum improvement reaches approximately 24%.
- The best ψ value differs greatly between problem instances.
- There is no simple relationship between ψ and solution improvement.
- SA, TS, and PSO ultimately find solutions of statistically similar quality.
- PSO generally reaches those solutions more efficiently.
- TS also performs efficiently on the largest instances.
- The SA implementations require considerably more time on large networks.
- Network density is positively associated with the benefit of parameter tuning.

- Other network characteristics do not significantly predict improvement.
- The metaheuristic-guided procedure remains effective across a broad range of network structures.

Important Limitations

The authors do not test every possible parameter setting for the metaheuristics.

Their SA, TS, and PSO configurations are based on experimentation and values suggested by previous literature, but a more extensive tuning process could alter the results.

The comparison also focuses on:

- one numerical parameter,
- one underlying approximation procedure,
- and one family of network-flow problems.

The results therefore do not establish that PSO is always more efficient than SA or TS in other optimization contexts.

The best solution found by each method is also evaluated through Ψ -DSSP rather than compared directly with the proven optimum for every instance.

The study primarily measures improvement over ordinary DSSP, not absolute optimality gaps.

Conclusion

The authors conclude that metaheuristic parameter search is an effective way to improve the Parameterized Dynamic Slope Scaling Procedure.

All three methods—Simulated Annealing, Tabu Search, and Particle Swarm Optimization—find similarly strong ψ values and produce approximately equal final solution quality.

The principal difference is efficiency.

Particle Swarm Optimization:

- converges faster on average,
- requires fewer effective evaluations,
- and produces the highest improvement per iteration.

Tabu Search also performs strongly, especially on large network instances.

The two Simulated Annealing versions eventually reach comparable solution quality but generally require more time.

The paper therefore recommends the use of a metaheuristic-guided Ψ -DSSP procedure, with particular support for PSO when search efficiency is important.

Hybrid of Particle Swarm Optimization, Simulated Annealing and Tabu Search for the Reconstruction of Two-Dimensional Targets from Laboratory-Controlled Data

B. Mhamdi, K. Grayaa, and T. Aguil

Summary

This paper proposes a hybrid optimization algorithm that combines **Particle Swarm Optimization, Simulated Annealing, and Tabu Search** for reconstructing two-dimensional dielectric objects from experimentally measured microwave-scattering data.

The method is called **PSO-SA-TS**.

The authors address a weakness of standard Particle Swarm Optimization: although PSO is relatively simple and often converges quickly, particles can gather around a local optimum and lose the ability to explore other regions. The proposed method uses Simulated Annealing and Tabu Search as local-improvement procedures within the PSO population.

The paper evaluates the hybrid algorithm using laboratory-controlled measurements of dielectric objects and compares it with:

- standard PSO,
- PSO combined only with Tabu Search,
- and PSO combined only with Simulated Annealing.

The reported results show that the full PSO-SA-TS hybrid produces:

- lower reconstruction error,
- faster convergence,
- fewer cost-function evaluations,
- shorter computation time in one reported experiment,
- and better robustness against measurement noise.

Microwave Imaging Problem

The objective of microwave imaging is to reconstruct the physical and geometrical properties of an object that cannot be examined directly.

A known electromagnetic field is directed toward the object. Sensors then measure the field scattered by the object.

From these measurements, the reconstruction procedure attempts to estimate properties such as:

- shape,
- location,
- relative dielectric permittivity,
- and electrical conductivity.

Possible applications include:

- biomedical imaging,
- breast-cancer detection,
- nondestructive testing,
- detection of buried objects,
- pipeline inspection,
- and geoscientific investigation.

Difficulty of the Inverse-Scattering Problem

The paper explains that electromagnetic inverse scattering is:

- nonlinear,
- ill-posed,
- and sensitive to noise.

The forward problem is relatively straightforward: given the object's electromagnetic properties, calculate the scattered field.

The inverse problem is much harder: given the measured scattered field, infer the unknown object properties.

A small error or amount of noise in the measurements can cause a large difference in the reconstructed object.

Because of this, the inverse problem is reformulated as a global optimization problem.

The algorithm searches for an object profile whose calculated scattered field closely matches the experimentally measured field.

Existing Solution Approaches

The paper divides existing approaches into two broad groups.

Gradient-Based Methods

Examples include:

- Newton-type methods,
- modified gradient methods,
- Levenberg–Marquardt methods,
- and contrast-source inversion.

These methods may converge efficiently when their initial assumptions are favorable.

However, they can:

- become trapped in local extrema,
- diverge,
- or fail when the initial estimate is poor.

Global Stochastic Methods

Examples include:

- Genetic Algorithms,
- Simulated Annealing,
- and Particle Swarm Optimization.

These methods are attractive because they explore the search space more broadly and do not rely entirely on local gradient information.

The paper notes that PSO has become popular in microwave and antenna applications because it is comparatively easy to implement and tune.

Main Limitation of Standard PSO

In standard PSO, particles are influenced by:

- their own best-known position,
- and the best-known position of the swarm.

This can produce rapid convergence.

However, when the global-best solution is only a local optimum, the particles may gather around it. Their velocities then become very small, and the swarm may no longer explore distant regions.

The authors describe this as **premature convergence**.

The problem becomes particularly serious in:

- high-dimensional spaces,
- complex objective landscapes,
- and problems containing many local minima.

Reason for Hybridization

The paper refers to the No Free Lunch principle, which states that no optimization method performs best across all possible objective functions.

The authors therefore combine methods with different strengths.

Role of Particle Swarm Optimization

PSO performs population-based global exploration and rapidly spreads information about strong solutions.

Role of Simulated Annealing

SA performs local search and can accept worse solutions probabilistically.

This helps it escape local optima.

Role of Tabu Search

TS searches local neighborhoods while using memory to prevent repeated movement through recently explored regions.

It also uses aspiration rules to override tabu restrictions when necessary.

The hybrid is intended to preserve PSO's convergence speed while improving local search and resistance to stagnation.

Electromagnetic Problem Formulation

The physical target is modeled as a two-dimensional cross-section of an infinitely long cylindrical object.

The object is placed inside an investigation domain and illuminated by several incident electromagnetic waves.

Receivers outside the domain measure the scattered field.

The material properties are represented through a contrast function containing:

- relative permittivity,
- and conductivity.

The paper assumes materials that are:

- stationary,
- linear,
- isotropic,
- and nondispersive.

Because the incident field is polarized along one axis and the object is assumed uniform along that axis, the original three-dimensional electromagnetic problem is reduced to two dimensions.

Forward Model

The forward problem computes the scattered field from a known object profile.

The authors formulate this using coupled integral equations:

- an observation equation connecting the object's induced sources to the measured scattered field,
- and a state equation connecting the internal total field to the incident field and object contrast.

The equations are discretized using the **Method of Moments**.

The investigation area is divided into small square cells. Within every cell, the field and material properties are assumed to be constant.

The discretized equations are represented in matrix form.

Inverse Model

The unknown solution vector contains the electromagnetic properties of every cell.

This includes:

- the relative permittivity of each cell,
- and the conductivity of each cell.

A candidate solution is evaluated by:

1. Using the candidate profile to calculate a synthetic scattered field.
2. Comparing the synthetic field with the measured experimental field.
3. Calculating a normalized error.

The optimization objective is to minimize this discrepancy.

A lower objective value indicates that the candidate object profile produces electromagnetic behavior closer to the actual measured object.

Particle Swarm Optimization Component

Each particle represents one possible object profile.

A particle contains:

- a position vector,
- a velocity vector,
- and a personal-best position.

The swarm also records the global-best position.

At every generation, velocity is updated using:

- inertia,
- attraction toward the personal best,
- attraction toward the global best,
- and random coefficients.

The new velocity is then added to the current position.

The inertia weight decreases over time to balance:

- broad exploration at the beginning,

- and local refinement near the end.

Simulated Annealing Component

The SA procedure begins from a current solution and generates a nearby candidate.

If the candidate has a lower objective value, it is accepted.

A worse candidate may also be accepted according to the Metropolis probability.

This probability depends on:

- the difference between the new and current objective values,
- and the current temperature.

At high temperature, worsening moves are accepted more often.

As temperature decreases, the algorithm becomes more selective.

The cooling rule is geometric:

$$T(n+1) = \beta T(n) \quad T(n+1) = \beta T(n)$$

where the cooling factor is between zero and one.

The initial temperature is calculated using the maximum and minimum objective values in the initial swarm.

This causes many worsening moves to be acceptable early in the process.

Tabu Search Component

The TS procedure begins with a candidate solution and evaluates its neighbors.

Its two main mechanisms are:

Tabu List

The tabu list records previously explored local choices and temporarily prevents them from being selected again.

This discourages cycling and repeated examination of the same region.

Aspiration Rule

A tabu solution may be released when the search would otherwise have no admissible move.

The paper uses an aspiration rule that frees the tabu move associated with the smallest increase in the current objective value.

This allows the local search to continue even when all available candidates are classified as tabu.

Structure of the PSO-SA-TS Hybrid

The hybrid keeps the three optimization mechanisms active throughout the search.

It does not simply run one algorithm after another.

The procedure begins with a randomly generated particle population.

During every generation, the population is divided into two groups:

- one group is locally improved using Simulated Annealing,
- the other group is locally improved using Tabu Search.

The locally improved particle solutions are then used to update:

- personal-best positions,
- and the global-best position.

Afterward, ordinary PSO velocity and position updates are applied to the whole swarm.

This combines local improvement with global population movement.

Hybridization Coefficient

The allocation between SA and TS is controlled by a **hybridization coefficient**, or HC.

Its interpretation is:

- **HC = 0**: all local improvement uses TS, producing PSO-TS.
- **HC = 1**: all local improvement uses SA, producing PSO-SA.
- **$0 < HC < 1$** : part of the swarm uses SA and the remainder uses TS.

For example, $HC = 0.5$ means that approximately half the population is locally improved through SA and the other half through TS.

The coefficient controls the balance between the two local-search methods.

Hybrid Algorithm Procedure

The complete process is:

1. Randomly initialize the particle population.
2. Evaluate the objective value of every particle.
3. Randomly select the fraction of particles assigned to SA.
4. Apply several SA local-search iterations to those particles.
5. Update the SA temperature.
6. Apply TS local search to the remaining particles.
7. Update the tabu list.
8. Compare the local-best solution of each particle with its historical personal best.
9. Update the swarm's global best.
10. Update particle velocities and positions using PSO.
11. Repeat until a target solution or maximum-generation limit is reached.

Experimental Data

The algorithm is evaluated using experimental data from the Institut Fresnel in Marseille, France.

Measurements are collected from actual dielectric targets.

The experimental arrangement uses:

- 241 receiver positions,
- receivers placed around a circular measurement boundary,
- multiple emitter positions,
- and a frequency of 4 GHz.

The investigation region measures:

- 0.15 meters by 0.15 meters.

It is divided into:

- 20 by 20 cells,
- for a total of 400 cells.

The optimization therefore estimates the material properties across a 400-cell spatial grid.

Algorithm Parameters

The reported PSO configuration uses:

- 300 particles,
- a maximum of 400 iterations,
- cognitive coefficient $c1=2c_1 = 2c1=2$,
- social coefficient $c2=2c_2 = 2c2=2$,
- and a time-varying inertia weight decreasing from 0.9 to 0.4.

For Tabu Search:

- the tabu-list size is 5.

For Simulated Annealing:

- the cooling factor is 0.9.

These parameter values are based on earlier literature and heuristic testing.

Reconstruction Error

The quality of a reconstructed object is evaluated using a relative permittivity error.

This compares:

- the actual relative permittivity in every cell,
- with the reconstructed relative permittivity.

The errors are normalized and aggregated across the entire 400-cell domain.

A lower error means the reconstructed dielectric profile is closer to the known original object.

First Experimental Object

The first object, called **FoamDieIExt**, consists of two nearby dielectric cylinders.

Foam Cylinder

- Diameter: 80 millimeters
- Relative permittivity: approximately 1.45, with an uncertainty of ± 0.15

Plastic Cylinder

- Diameter: 31 millimeters
- Relative permittivity: approximately 3, with an uncertainty of ± 0.3

The two objects are placed close to one another.

Selection of the Hybridization Coefficient

The authors test several HC values between zero and one.

The reconstruction-error curve is U-shaped.

The error is relatively high when:

- HC is close to zero, meaning mostly TS,
- or HC is close to one, meaning mostly SA.

The minimum error occurs around:

$$HC=0.5$$

The authors therefore use $HC = 0.5$ as the standard setting for the full PSO-SA-TS method.

This means SA and TS are applied to approximately equal portions of the swarm.

First-Object Reconstruction Results

The first target is reconstructed using:

- standard PSO,
- PSO-TS,
- PSO-SA,
- and PSO-SA-TS.

The reported relative permittivity errors are:

Method	Reconstruction error
--------	-------------------------

Standard PSO	0.105
PSO-TS	0.047
PSO-SA	0.024
PSO-SA-TS	0.003

The full hybrid produces the lowest error by a substantial margin.

The reconstructed image produced by PSO-SA-TS is visually much closer to the original object shape and permittivity distribution.

The recovered conductivity is nearly uniform and close to zero, matching the fact that the targets are treated as purely dielectric.

Convergence Results

The convergence graph compares objective values across generations.

The ordering is clear:

- standard PSO converges most slowly,
- PSO-TS improves on standard PSO,
- PSO-SA converges more rapidly,
- and PSO-SA-TS reaches a very low objective value fastest.

The hybrid using all three techniques experiences a steep early decline in error and approaches near-zero error much earlier than the other methods.

The authors conclude that combining SA and TS with PSO improves both:

- convergence speed,
- and final reconstruction accuracy.

Second Experimental Object

The second target, called **FoamTwinDiel**, contains:

- two plastic cylinders,
- and one foam cylinder.

The materials have characteristics similar to those in the first experiment but are arranged differently.

The reconstruction is more complex because the method must distinguish three nearby targets and their different permittivity values.

Second-Object Reconstruction Accuracy

The reported relative permittivity errors are:

- **Standard PSO:** approximately 5.62
- **PSO-SA-TS:** approximately 0.64

The hybrid reconstruction more clearly identifies:

- the shape of the foam cylinder,
- the two plastic cylinders,
- their relative positions,
- and the differences in permittivity.

A cross-sectional profile through the reconstructed image shows that PSO-SA-TS closely follows the original piecewise permittivity pattern.

The standard PSO profile deviates more strongly from the true values.

Computation Time

For the second experiment, the reported computation times are:

- **Standard PSO:** 72 minutes and 23 seconds
- **PSO-SA-TS:** 26 minutes and 43 seconds

Although the hybrid performs additional SA and TS local-search operations, it completes the reconstruction much faster in the reported test.

This is attributed to faster convergence and fewer required objective-function evaluations.

Noise-Robustness Experiment

The authors add zero-mean white Gaussian noise to the measured electromagnetic fields.

Several signal-to-noise ratios are tested.

Both methods perform poorly at very low signal-to-noise levels, but PSO-SA-TS generally produces lower reconstruction error.

As signal quality improves, the hybrid error falls toward zero, while standard PSO retains a noticeable residual error.

The paper concludes that PSO-SA-TS is more robust against measurement noise than standard PSO.

Main Findings

The paper's major findings are:

- Standard PSO can converge prematurely when particles gather around a local optimum.
- SA and TS can improve local search and help particles escape stagnation.
- A hybrid that keeps all three algorithms active throughout the run performs better than using PSO alone.
- Combining PSO with only SA or only TS also improves reconstruction.
- The full PSO-SA-TS method produces the best reconstruction among the tested variants.
- An approximately equal division between SA and TS, represented by $HC = 0.5$, produces the lowest error.
- The hybrid reaches lower objective values in fewer generations.
- It reconstructs object shape and permittivity more accurately.
- It substantially reduces error for both tested target configurations.
- It requires less computation time than standard PSO in the second reported experiment.
- It is more resistant to additive measurement noise.

Important Limitations

The study compares the hybrid primarily against standard PSO and two partial hybrids.

It does not report independent comparisons against:

- pure standalone SA,
- or pure standalone TS

using the same complete reconstruction task.

Therefore, the paper demonstrates the benefit of adding SA and TS to PSO but does not establish which of the three algorithms would perform best independently.

The evaluation uses only two principal dielectric-object arrangements.

The experiments are also limited to:

- two-dimensional reconstruction,
- one operating frequency,
- one experimental measurement system,
- and a fixed grid resolution of 20 by 20 cells.

The parameter settings appear to be selected from literature guidance and heuristic testing rather than through a comprehensive parameter-optimization study.

The hybrid also contains many interacting parameters, including:

- PSO swarm size,
- maximum generations,
- inertia schedule,
- acceleration coefficients,
- SA cooling factor,
- tabu-list size,
- local-search length,
- and hybridization coefficient.

Conclusion

The authors conclude that PSO-SA-TS is an effective method for solving nonlinear microwave inverse-scattering problems.

PSO provides broad global exploration and fast population-based movement. Simulated Annealing and Tabu Search strengthen local refinement and help the swarm move away from local optima.

The laboratory-data experiments show that the hybrid achieves:

- higher reconstruction accuracy,
- faster convergence,
- fewer objective-function evaluations,
- improved runtime,
- and greater robustness to noise

than standard PSO.

The authors propose future work involving:

- three-dimensional simulations,
- and further evaluation using experimental data.

Linking Search Space Structure, Run-Time Dynamics, and Problem Difficulty: A Step Toward Demystifying Tabu Search

Jean-Paul Watson, L. Darrell Whitley, and Adele E. Howe

Summary

This paper investigates **why Tabu Search performs effectively** on the Job-Shop Scheduling Problem and what characteristics make individual scheduling instances easy or difficult.

Instead of proposing a new scheduling algorithm, the authors develop a mathematical and empirical model of the behavior of an existing Tabu Search method. Their central claim is that the algorithm's apparently complicated search behavior can be modeled with high accuracy as a **biased one-dimensional random walk**.

The paper examines:

- the region of the solution space actually explored by Tabu Search,
- the relationship between search-space structure and problem difficulty,
- the influence of short-term tabu memory,
- the effect of problem structure,
- the effect of the neighborhood operator,
- the importance of the initial solution,
- and the effect of tabu tenure.

The authors find that the search usually remains close to local optima, moves through this restricted region in a predictable way, and spends longer solving instances whose effective search spaces are larger.

Job-Shop Scheduling Problem

The paper studies the classical static and deterministic **Job-Shop Scheduling Problem**, or JSP.

A JSP contains:

- nnn jobs,
- mmm machines,
- a fixed sequence of machines for each job,

- and a processing time for every operation.

Each job must be processed exactly once on every machine. The machine order for a job is predetermined.

The problem includes several constraints:

- an operation cannot begin until the preceding operation of the same job is complete,
- each machine can process only one operation at a time,
- and operations cannot be interrupted once processing begins.

The objective is to minimize the **makespan**, which is the completion time of the final operation.

Makespan minimization for the JSP is NP-hard.

Random and Structured Scheduling Instances

The authors distinguish among three types of JSP instances.

Random JSPs

Processing times are selected randomly, and each job follows a randomly generated machine order.

These form a large portion of commonly used benchmark instances.

Workflow JSPs

Machines are divided into groups or workflow partitions.

Every job must complete all machines in one partition before moving to the next partition, although the order within each group can vary.

Flow-Shop JSPs

All jobs visit all machines in the same order.

Flow-shop instances therefore have the greatest routing structure among the three categories.

The paper observes that added structure does not necessarily make these scheduling problems easier.

For fixed values of n and m :

- random JSPs tend to be the easiest,
- workflow JSPs tend to be more difficult,
- and flow-shop JSPs tend to be the most difficult.

Problem Instances

The experiments use several collections of scheduling problems.

The main sets include:

- 1,000 random 6×6 JSPs,
- 1,000 random 6×6 JSPs,
- corresponding workflow and flow-shop instances,
- 100 random 10×10 JSPs,
- additional 10×10 workflow and flow-shop problems,
- and established OR Library benchmark instances.

Processing times are generally sampled uniformly between 1 and 99.

For the smaller instances, the authors calculate:

- the optimal makespan,
- and the complete set of optimal solutions.

These known optima are needed because much of the study measures how far the search is from the nearest optimal solution.

Tabu Search Algorithm Examined

The main algorithm is a modified version of Taillard's Tabu Search method. The authors call it **TSN1**.

It is intentionally simpler than the strongest Tabu Search algorithms available at the time. This allows the authors to study its underlying behavior without too many additional mechanisms.

TSN1 uses:

- a steepest-descent search bias,

- the N1 neighborhood operator,
- short-term tabu memory,
- dynamic tabu tenure,
- and a simple aspiration rule.

It does not use a long-term memory mechanism such as reintensification.

N1 Neighborhood Operator

The N1 operator changes a schedule by swapping two adjacent operations that belong to a critical block.

A critical block is a consecutive sequence of operations on a critical path whose timing directly affects the makespan.

At every iteration:

1. All eligible N1 neighbors are generated.
2. Their makespans are calculated.
3. The admissible neighbor with the lowest makespan is selected.
4. Ties are broken randomly.

The chosen move may worsen the current schedule. The algorithm therefore continues beyond local optima.

Tabu Restriction

When two adjacent operations are exchanged, the reverse ordering becomes tabu for several iterations.

This prevents the algorithm from immediately undoing the move and cycling between the same schedules.

The length of time for which the reversal remains prohibited is the **tabu tenure**.

The tenure is dynamically sampled from a predefined interval rather than kept fixed throughout the search.

Aspiration Criterion

A tabu move is allowed when it produces a schedule better than the best schedule found during the current trial.

This prevents the tabu restriction from blocking a new best solution.

When both a non-tabu move and a tabu-but-aspired move produce the same minimum makespan, the algorithm favors the non-tabu move.

Search Cost

The paper defines search cost as the number of Tabu Search iterations required to locate a globally optimal solution.

Search cost varies between independent runs because:

- initial solutions may differ,
- tabu tenure is dynamically sampled,
- and ties are broken randomly.

The authors therefore report both:

- median search cost,
- and mean search cost.

They estimate these values using 1,000 independent trials per problem instance.

Distance Between Schedules

The authors use a **disjunctive graph distance** to measure the difference between two schedules.

It counts how many pairwise machine-processing order relationships differ between the schedules.

Informally, two schedules are farther apart when they place many pairs of jobs in different orders on the machines.

This distance is also a lower bound on the number of N1 moves required to transform one schedule into another.

Local Optima

A random local optimum is constructed by:

1. Generating a random feasible schedule.
2. Applying steepest-descent local search.
3. Stopping when no N1 move can reduce the makespan.

Earlier work by the authors examined the mean distance between random local optima and the nearest global optimum.

This measure was strongly related to problem difficulty, but it had important weaknesses:

- it was expensive to calculate,
- it became less accurate for difficult instances,
- it did not scale well,
- and it did not explain the actual run-time behavior of Tabu Search.

The current paper develops a more detailed model to address these limitations.

Region of the Solution Space Explored

The authors hypothesize that TSN1 does not explore the complete solution space.

Instead, it operates mainly within a smaller subspace containing:

- local optima,
- and solutions located only a small number of moves away from local optima.

They call this subspace $S_{\text{lopt}+}$.

To test the idea, they run TSN1 for one million iterations on selected JSP instances and measure how far each visited solution lies from a local optimum.

The results show that the search usually remains only **one or two N1 moves away from a local optimum**, regardless of problem size.

By comparison, randomly generated schedules are much farther from local optima.

For example, while TSN1 solutions often have mean descent distances near two, random schedules can have mean distances ranging from approximately 12 to more than 70 moves, depending on problem size.

This indicates that Tabu Search ignores a large part of the total feasible solution space.

Weak Local-Optimum Basins

The paper finds that local optima in the JSP generally have weak attraction basins.

The search can often escape a local optimum by accepting only one or two worsening moves and then returning to steepest descent.

This explains why TSN1 remains close to local optima without becoming permanently trapped in them.

The algorithm repeatedly moves:

- into a local optimum,
- slightly away from it,
- and toward another local optimum.

Effective Search-Space Size

The authors argue that problem difficulty depends primarily on the **effective size of the search space actually explored**, rather than the complete number of feasible schedules.

This effective space is influenced by:

- the number of local and near-local-optimal solutions,
- the number of global optima,
- and the distribution of those global optima within the explored region.

A larger effective search space means that the algorithm must move through more candidate states before encountering an optimal solution.

Static Cost Models

The paper reviews earlier attempts to predict search difficulty using fixed properties of the fitness landscape.

These include:

- number of optimal solutions,
- backbone size,
- average distance between local optima,
- fitness-distance correlation,
- landscape correlation length,

- and mean distance from random local optima to the nearest optimum.

Most conventional landscape measures explain only a limited portion of the variation in Tabu Search cost.

Reported coefficients of determination for common measures range from approximately:

- 0.22 to 0.54.

The mean distance from local optima to the nearest optimum performs better, explaining:

- approximately 83% of the variation for 6×46 times 46×4 problems,
- and approximately 65% for 6×66 times 66×6 problems.

However, this model still performs poorly for some difficult or structured instances.

Dynamic Random-Walk Model

The paper's main contribution is a dynamic model of the actual search process.

A state in the model represents:

- the current distance from the nearest optimal solution,
- and the recent search direction.

The recent direction can indicate that the search was moving:

- closer to the optimum,
- farther from the optimum,
- or remaining at the same distance.

The optimal solution is represented as an absorbing state. Once the random walk reaches distance zero, the simulated search ends.

Two Main Search Biases

The model identifies two forms of bias.

Distance-Dependent Bias

The probability of moving closer to an optimum depends on the current distance.

When the search is far from an optimum, it becomes increasingly likely to move closer.

When the search is close to an optimum, it becomes easier for it to move away again.

This creates a restoring tendency toward a middle region of the effective search space.

Search Momentum

The direction of the previous move affects the next move.

When the search has recently moved closer to an optimum, it is more likely to continue moving closer.

When it has recently moved farther away, it is more likely to continue moving farther away.

The authors describe this as momentum or trending behavior.

This momentum is caused primarily by short-term tabu memory.

Interpretation of the Random Walk

The search behaves somewhat like a diffusion process with a restoring force.

It does not move uniformly through the search space.

Instead, it tends to hover around a characteristic intermediate distance between:

- an optimal solution,
- and the farthest region that the algorithm commonly explores.

The maximum commonly observed distance is called $D_{\max}$.

The average distance maintained by the search is approximately:

$$D_{\max}/2$$

The value of $D_{\max}$ therefore acts as an important measure of effective search-space size.

Accuracy of the Dynamic Model

The dynamic random-walk model predicts the mean cost of locating optimal solutions with very high accuracy.

For the 6×46 and 6×66 random JSP sets, the model explains approximately:

- **96% of the variability in mean search cost.**

For the selected 10×1010 random JSPs, it explains approximately:

- **97% of the variability.**

For most instances, the actual mean search cost is within a factor of two of the model's prediction.

The accuracy does not decline noticeably as the problem size increases from the smaller instances to the 10×1010 problems.

The model also predicts the cost of locating suboptimal solutions with similarly high accuracy.

Run-Length Distributions

Search cost is not a fixed value but a distribution across repeated trials.

The paper refers to this as the **run-length distribution**.

For most moderate and difficult JSP instances, the number of iterations required to find an optimum is approximately exponentially distributed.

This means that:

- many trials end relatively quickly,
- but some require much longer,
- and the probability of success during the next interval remains approximately constant.

For roughly 10% of the tested 10×1010 instances, the exponential model is statistically rejected.

These exceptions are generally easier instances and contain fewer very short successful runs than a perfect exponential distribution would predict.

The authors note that Weibull distributions may fit some easier instances more accurately.

Predicted Versus Actual Run-Length Distributions

Although the dynamic model predicts average search cost accurately, its full predicted distributions are often statistically distinguishable from the observed distributions.

For most of the analyzed instances, statistical tests reject the hypothesis that the predicted and actual run-length distributions are identical.

Nevertheless, the distributions are generally similar in shape.

For difficult instances, both predicted and actual distributions are close to exponential. Much of the difference can be explained by small shifts in the predicted mean rather than a fundamentally incorrect distributional structure.

Difficulty of Structured JSPs

The authors compare random, workflow, and flow-shop instances.

For the 6×66 \times 66×6 sets, median search costs are approximately:

- 280 iterations for random JSPs,
- 3,137 iterations for workflow JSPs,
- and 12,127 iterations for flow-shop JSPs.

For the 10×1010 \times 1010×10 sets, the mean or median costs grow much more dramatically:

- approximately 315,000 for random instances,
- approximately 43.5 million for workflow instances,
- and approximately 262 million for flow-shop instances.

One particularly difficult 10×1010 \times 1010×10 flow-shop problem required an average of approximately **900 million iterations** to locate an optimum.

Why Structured Problems Are Harder

The transition probabilities for structured JSPs are more irregular than those for random JSPs.

However, the dynamic random-walk model remains highly accurate.

It explains approximately:

- 95% to 97% of the variability

in the cost of solving workflow and flow-shop instances.

The main difference is that structured instances have larger values of $D_{\max}$.

For example, average $D_{\max}$ values for 6×6 problems are approximately:

- 26.67 for random JSPs,
- 47.80 for workflow JSPs,
- and 68.86 for flow-shop JSPs.

The authors conclude that structured problems are harder mainly because their effective search spaces are wider.

Alternative N5 Neighborhood Operator

The paper also studies an algorithm using the more selective **N5 move operator**, called TSN5.

N5 excludes many N1 moves that cannot immediately improve the makespan.

This reduces the neighborhood size and lowers the cost of evaluating each iteration.

However, the N5 search space can become disconnected. Some portions may form traps from which an N5-only search cannot escape.

The authors add a mechanism that detects these traps and escapes through a short sequence of N1 moves.

N1 Versus N5 Performance

The number of iterations needed under TSN1 and TSN5 can differ substantially.

Differences by a factor of ten are common, and some approach a factor of one hundred.

In some instances, TSN5 actually requires more iterations to find the optimum.

However, N5 neighborhoods are often more than ten times smaller than N1 neighborhoods.

Because each TSN5 iteration is much cheaper, TSN5 generally achieves lower total runtime despite sometimes requiring more iterations.

Random-Walk Model for N5

The dynamic model remains highly accurate for TSN5.

It explains approximately:

- 96% of the variation in mean search cost.

This means that replacing the neighborhood operator changes:

- the transition probabilities,
- the maximum explored distance,
- and the number of iterations,

but does not fundamentally change the high-level search process.

TSN5 still behaves as a biased random walk among local and near-local-optimal solutions.

Effect of Initial Solutions

The authors use the model to predict whether better initialization methods improve Tabu Search performance.

They examine several schedule-construction methods, including:

- First-Come, First-Serve,
- Longest Remaining Work,
- Most Work Remaining,
- Shortest Processing Time,
- random active schedules,
- random non-delay schedules,
- Taillard's lexicographic method,
- the Werner–Winkler insertion method,
- and random semi-active schedules.

Some methods generate local optima that are significantly closer to global optima than the random baseline.

However, even the improved starting solutions remain too far from an optimum to produce a meaningful performance advantage on difficult 10×1010 times 1010×10 instances.

Initialization Results

Across the tested initialization methods, the worst difference in median search cost is less than approximately 3%.

The best observed improvement is around:

- **2.79%**, using the Werner–Winkler method.

None of the differences is statistically significant.

The paper concludes that the initial solution has little effect on TSN1 performance unless it begins extremely close to an optimal solution.

A solution with a good makespan is not necessarily close to an optimum in search-space distance. Therefore, initial fitness alone is not a reliable indicator of how useful a starting solution will be.

The authors acknowledge that initialization may matter more for:

- easier problems,
- searches targeting suboptimal rather than optimal solutions,
- or algorithms using reintensification.

Effect of Tabu Tenure

The authors examine three tabu-tenure intervals:

- small: [5,10][5,10][5,10],
- medium: [8,14][8,14][8,14],
- large: approximately [10,18][10,18][10,18].

The small interval is close to the minimum required to avoid stagnation and permanent trapping.

As tenure increases, the search is forced farther away from recently visited regions.

This increases $D_{\max}$ and expands the effective search space.

Tabu-Tenure Results

Compared with the small tenure interval:

- the medium interval produces $D_{\max}$ values approximately 6% larger,
- and mean search costs approximately 95% larger.

Compared with the medium interval:

- the large interval produces $D_{\max}$ values approximately 5% larger,

- and mean search costs approximately 60% larger.

The authors also observe continued growth in difficulty for still larger tenure values.

Tabu-Tenure Recommendation

The paper recommends selecting tabu tenure:

as small as possible while still preventing search stagnation.

A tenure below the necessary minimum may cause:

- cycling,
- trapping,
- or failure to escape local optima.

However, a tenure larger than necessary forces the search to explore an unnecessarily large region and substantially increases solution time.

This provides a theoretical explanation for why excessive short-term memory can reduce Tabu Search performance.

Role of Short-Term Memory

Short-term memory is not uniformly beneficial.

When the search is moving toward an optimum, memory creates momentum that may help it continue in the correct direction.

When the search is moving away, the same mechanism encourages it to continue moving away.

Therefore, memory both:

- supports escape from local optima,
- and risks carrying the search unnecessarily far from promising regions.

Its benefit depends on using only enough memory to prevent stagnation.

Main Findings

The paper's principal findings are:

- Tabu Search explores only a restricted portion of the complete scheduling solution space.
- The search usually remains within one or two moves of local optima.
- Local-optimum basins in the JSP are generally weak and easy to escape.
- The high-level behavior of the studied Tabu Search algorithm resembles a biased one-dimensional random walk.
- Search direction exhibits momentum because of short-term tabu memory.
- Problem difficulty depends heavily on the effective width of the explored search space.
- The dynamic model explains approximately 95% to 97% of the variation in search cost.
- The model remains accurate across random, workflow, and flow-shop scheduling problems.
- The model also applies to a stronger N5 neighborhood operator.
- Structured JSPs are harder because their effective search spaces are wider.
- Starting-solution methods have little influence on difficult instances unless they place the search extremely close to an optimum.
- Larger tabu tenure expands the effective search space and substantially increases search cost.
- Tabu tenure should be kept at the smallest value that still prevents stagnation.
- Short-term memory can help or hurt depending on the direction in which the search is moving.

Important Limitations

The paper concentrates on the Job-Shop Scheduling Problem.

It is uncertain whether the same random-walk behavior applies to Tabu Search in other optimization problems, such as:

- quadratic assignment,
- permutation flow-shop scheduling,
- vehicle routing,
- or satisfiability problems.

The model also focuses mainly on short-term memory.

State-of-the-art Tabu Search algorithms often use additional mechanisms such as:

- long-term memory,
- reintensification,
- diversification,
- and path relinking.

The effect of these mechanisms on the random-walk structure remains unresolved.

The model is mainly constructed after observing the algorithm's behavior. Estimating its transition probabilities requires substantial computation, so it is not yet a cheap method for predicting the difficulty of a previously unseen instance.

The authors also do not provide a complete causal explanation for all observed biases. For example, they do not fully explain why the search avoids large portions of the local-optimum region or why it is attracted toward a middle distance from optimal solutions.

Conclusion

The paper concludes that the behavior of Tabu Search on the Job-Shop Scheduling Problem is less mysterious than its strong performance might initially suggest.

At a high level, the algorithm repeatedly moves among local and near-local-optimal schedules in a manner resembling a biased random walk.

Its search cost is governed largely by:

- the size of the effective search region,
- the distribution of optimal solutions within that region,
- and the directional momentum created by short-term memory.

The dynamic model accurately explains both optimal and suboptimal search costs across different problem sizes, structures, and neighborhood operators.

The study also produces two practical conclusions:

- sophisticated starting solutions provide little benefit unless they begin extremely close to an optimum,
- and tabu tenure should be only large enough to prevent stagnation, since additional memory can sharply reduce performance.

The authors present the model as a foundation for studying more advanced Tabu Search methods and for determining whether similar behavior occurs in other NP-hard optimization problems.

List-Based Simulated Annealing Algorithm for Traveling Salesman Problem

Shi-hua Zhan, Juan Lin, Ze-jun Zhang, and Yi-wen Zhong

Summary

This paper proposes a modified Simulated Annealing method called **List-Based Simulated Annealing**, or **LBSA**, for solving the Traveling Salesman Problem.

The main purpose of LBSA is to reduce one of the practical weaknesses of standard Simulated Annealing: its sensitivity to parameter settings, particularly the initial temperature and cooling schedule.

Instead of reducing temperature using a fixed geometric, arithmetic, or logarithmic formula, LBSA stores several temperature values in a list. The algorithm repeatedly uses the largest temperature in the list and then updates the list according to information collected during the search.

The authors evaluate LBSA on benchmark Traveling Salesman Problem instances containing from 48 to 85,900 cities. They examine:

- convergence behavior,
- cooling behavior,
- sensitivity to parameter values,
- different temperature-list update rules,
- different neighborhood operators,
- solution quality,
- computational time,
- and performance relative to several other optimization algorithms.

The experiments show that LBSA is relatively insensitive to its main parameters and produces competitive or superior results across a wide range of TSP sizes.

Traveling Salesman Problem

The Traveling Salesman Problem asks for the shortest possible tour that:

- visits every city exactly once,
- returns to the starting city,
- and minimizes total travel distance.

For n cities, a solution can be represented as a permutation:

$$\pi = (\pi(1), \pi(2), \dots, \pi(n))$$

The tour length is calculated by summing the distances between consecutive cities and adding the distance from the final city back to the first.

The paper distinguishes between two versions of TSP.

Symmetric TSP

The distance from city i to city j is the same as the distance from city j to city i .

This forms an undirected graph.

Asymmetric TSP

The cost or availability of travel may differ depending on direction.

Examples include:

- one-way streets,
- differing arrival and departure fees,
- and routes that exist in only one direction.

The TSP is NP-hard, meaning that no known polynomial-time algorithm guarantees an optimal solution for every sufficiently large instance.

Standard Simulated Annealing

Simulated Annealing begins with an initial solution and repeatedly generates neighboring solutions.

When a new solution is better than the current one, it is accepted.

When it is worse, it may still be accepted according to the Metropolis probability:

$$p = e^{-\frac{f(y) - f(x)}{t}}$$

where:

- x is the current solution,
- y is the candidate solution,

- $f(x)f(x)f(x)$ and $f(y)f(y)f(y)$ are their objective values,
- and t is the current temperature.

At high temperatures, worse solutions have a relatively high probability of acceptance. This encourages broad exploration.

As temperature decreases, worse solutions are accepted less frequently, and the search becomes increasingly focused on local improvement.

Cooling Schedule

A complete cooling schedule normally specifies:

- the starting temperature,
- the temperature-reduction function,
- the Markov-chain length,
- and the stopping condition.

A commonly used schedule is geometric cooling:

$$t_{k+1} = \alpha t_k$$

where α is a cooling coefficient, often between approximately 0.8 and 0.99.

The authors argue that even in a simplified SA implementation, both the initial temperature and cooling coefficient significantly affect performance.

Finding good values is difficult because:

- different problems require different settings,
- poor values may produce premature convergence,
- overly high temperatures may waste time accepting unproductive solutions,
- and overly slow cooling may require excessive computation.

Adaptive cooling can reduce this sensitivity, but many adaptive procedures also increase algorithmic complexity.

Purpose of List-Based Cooling

The authors aim to create a cooling method that is:

- adaptive,
- simple,

- robust to parameter choices,
- and easy to implement.

Their approach is inspired by list-based threshold-accepting algorithms, which maintain a list of acceptance thresholds instead of relying on one fixed threshold-reduction rule.

In LBSA, the list stores temperature values rather than thresholds.

Temperature List

The temperature list is organized as a priority queue.

Higher temperatures have higher priority.

During each outer iteration:

1. The algorithm retrieves the largest temperature in the list.
2. It uses that temperature in the Metropolis acceptance rule.
3. It performs a fixed number of neighborhood evaluations.
4. It calculates new temperature information from the worsening solutions accepted during that period.
5. It removes the previous maximum temperature.
6. It inserts a new, generally lower temperature into the list.

Because the inserted temperature is lower than the removed maximum, the overall temperature decreases as the search proceeds.

Creating the Initial Temperature List

The initial temperature values are generated using a selected initial acceptance probability p_0 .

The algorithm first generates:

- a current solution xxx ,
- and a neighboring solution yyy .

The difference in their objective values is used to calculate a temperature:

$$t = -\frac{|f(y) - f(x)|}{\ln(p_0)}$$

This process is repeated until the temperature list reaches its desired length.

The list therefore reflects actual cost differences found in the problem's neighborhood rather than being filled with arbitrarily selected temperature values.

Updating the Temperature List

During one temperature stage, the algorithm records the worsening solutions that are accepted.

Suppose a worse solution has a cost difference d_i , and it was accepted because a generated random number r_i was below the Metropolis probability.

The corresponding temperature value is calculated as:

$$t_i = -d_i \ln(r_i) \quad t_j = -\frac{d_j}{\ln(r_j)} \quad t_i = -\ln(r_i) d_i$$

The average of these calculated values is then inserted into the temperature list after the previous maximum is removed.

The authors choose the average because it produces a balanced rate of temperature decrease.

If no worse solution is accepted during the current temperature stage, the list is not updated in the normal way.

Adaptive Cooling Behavior

Unlike standard geometric cooling, LBSA does not apply the same cooling coefficient throughout the entire search.

Its effective cooling rate changes according to the behavior of accepted moves.

The authors describe the method as similar to geometric cooling with a variable cooling coefficient.

The experimental cooling curves show that LBSA:

- reduces temperature quickly during the early stages,
- and reduces temperature more slowly during the later stages.

This behavior allows the algorithm to avoid spending excessive time at very high temperatures while preserving more time for detailed search at low temperatures.

Computational Complexity of the Temperature List

The authors suggest implementing the list using a maximum heap.

Heap insertion and retrieval require logarithmic time.

They argue that maintaining the temperature list therefore does not change the overall time-complexity class of the Simulated Annealing procedure.

Solution Representation and Neighborhood Operators

A TSP solution is represented as a permutation of cities.

The paper examines three basic neighborhood operators.

Inverse Operator

A section of the route between two selected positions is reversed.

This normally replaces two edges in a symmetric TSP.

Insert Operator

One city is removed from its current position and inserted into another position.

This generally replaces three edges.

Swap Operator

Two cities exchange positions.

This generally replaces four edges.

Greedy Hybrid Neighborhood Operator

LBSA uses a hybrid neighborhood procedure.

After selecting two route positions, the algorithm creates three candidates:

- one using inversion,
- one using insertion,
- and one using swapping.

The candidate with the shortest tour is selected as the proposed neighboring solution.

This expands the effective neighborhood because three types of movement are evaluated for the same selected positions.

Experimental Design

The authors perform five main groups of experiments.

Cooling-Schedule Effectiveness

LBSA is compared with:

- geometric cooling,
- and arithmetic cooling.

Parameter Sensitivity

The authors vary:

- temperature-list length,
- and initial acceptance probability.

Temperature-Update Method

The list is updated using:

- the average,
- maximum,
- or minimum

of the calculated temperature values.

Neighborhood Structure

The inverse, insert, swap, and hybrid operators are compared.

Competitiveness

LBSA is compared with several previously published algorithms on TSPLIB and VLSI benchmark instances.

Cooling-Schedule Experiments

The first set of experiments uses four benchmark instances:

- BCL380 with 380 cities,
- XQL662 with 662 cities,
- XIT1083 with 1,083 cities,
- and XSC6880 with 6,880 cities.

Their best-known integer tour lengths are:

- 1,621,
- 2,513,
- 3,558,
- and 21,537.

The algorithms use:

- 1,000 outer iterations,
- and a Markov-chain length equal to the number of cities.

Temperature Behavior

The temperature graphs show that arithmetic cooling decreases very slowly and approximately linearly.

Geometric cooling falls more rapidly.

LBSA initially falls even faster than geometric cooling but later decreases more gradually.

The authors explain that high-temperature stages may accept nearly every candidate, including many unproductive moves. Rapidly leaving these stages can therefore save computation.

Slower cooling later allows more effort to be spent examining promising low-temperature regions.

Convergence Behavior

The tour-length graphs show that LBSA improves solutions more quickly than both geometric and arithmetic cooling.

Across all four benchmark problems, LBSA:

- has faster early convergence,
- reaches low tour lengths sooner,
- and maintains good long-term improvement.

Arithmetic cooling performs particularly poorly during much of the run because it remains at relatively high temperatures for too long.

Geometric cooling performs better than arithmetic cooling but generally converges more slowly than LBSA.

Temperature-List Length Sensitivity

The authors test 30 list lengths:

- from 10 to 300,
- increasing in steps of 10.

Each configuration is run 50 times on the four VLSI instances.

Performance is measured using the percentage difference between:

- the mean tour length,
- and the best-known tour length.

The results show that LBSA performs similarly over a broad range of list lengths.

However, list length is more sensitive for larger instances.

Small Instances

A wide range of list lengths produces similar results.

Large Instances

Very large lists significantly reduce performance.

A large list causes temperature to decrease slowly. If the Markov-chain length is not long enough for the search to reach equilibrium at each temperature, the algorithm spends too much time at high temperatures and accepts too many unproductive moves.

The authors select a fixed list length of:

120120120

for subsequent experiments.

Initial Temperature Sensitivity

The initial temperatures are generated using 30 different initial acceptance probabilities.

The tested values range from extremely small probabilities, around 10^{-20} , to 0.9.

Each configuration is run 50 times.

The results indicate that LBSA is relatively insensitive to the initial temperature.

The preferred direction differs by problem size:

- for smaller problems, the initial temperature should not be too low,
- for larger problems, it should not be too high.

However, the variation in performance across the tested range remains relatively small.

The authors attribute this robustness to the adaptive temperature-list mechanism.

Comparison of Temperature-Update Methods

The authors compare three choices for replacing the maximum list temperature:

- average calculated temperature,
- maximum calculated temperature,
- minimum calculated temperature.

The average update performs best on all four benchmark instances.

The reported mean tour lengths are:

Instance	Best known	Average update	Maximum update	Minimum update
BCL380	1,621	1,630.2	2,357.4	1,651.9
XQL662	2,513	2,527.2	4,117.8	2,565.8
XIT1083	3,558	3,592.9	6,663.0	3,674.6
XSC6880	21,537	21,920.0	63,472.2	22,649.8

Why the Average Update Works Best

Using the maximum calculated temperature causes the temperature to decline too slowly.

The algorithm then continues accepting many worse solutions and behaves almost like a random search.

Using the minimum calculated temperature causes temperature to decline too sharply.

The algorithm then loses its ability to escape local minima and begins behaving like ordinary greedy local search.

The average provides a middle course between excessive exploration and premature freezing.

Neighborhood-Operator Results

The hybrid operator performs best across all four benchmark instances.

The reported mean tour lengths are:

Instance	Inverse	Insert	Swap	Hybrid
BCL380	1,631.9 6	1,776.2 4	2,666.4 4	1,630.8 4
XQL662	2,535.2	2,836.5 2	4,437.2	2,527.2
XIT1083	3,603.4 8	4,118.72	6,760.8 4	3,592.0 8
XSC6880	22,115.6	29,048	64,263. 4	21,930. 6

Among the individual operators:

1. Inversion performs best.
2. Insertion performs second best.
3. Swapping performs worst.

Operator-Selection Frequencies

When the hybrid operator is used on BCL380, the selected candidates come from:

- inversion: 65%,
- insertion: 31%,
- swapping: 4%.

Similar percentages are observed on the other problems.

The authors explain that inversion performs well because it provides a fine-grained neighborhood change, replacing only two edges in a symmetric TSP.

The hybrid method performs slightly better because it searches a larger combined neighborhood and selects the best of the three generated candidates.

Comparison with GSAACS and MSA-IBS

LBSA is compared with:

- Genetic Simulated Annealing Ant Colony System, or GSAACS,
- and Multiagent Simulated Annealing with Instance-Based Sampling, or MSA-IBS.

The comparison uses 24 TSPLIB instances containing:

- 51 to 1,655 cities.

Each LBSA and MSA-IBS configuration uses:

- a population size of 30,
- 1,000 outer iterations,
- and a Markov-chain length equal to twice the number of cities.

Each algorithm is run 25 times per instance.

The average percentage errors are:

- GSAACS: 1.62%,
- MSA-IBS: 0.25%,
- LBSA: 0.15%.

Both LBSA and MSA-IBS outperform GSAACS on all 24 instances.

LBSA performs slightly better than MSA-IBS on average.

Comparison with GA-PSO-ACO and MSA-IBS

A second comparison uses 35 TSPLIB instances containing:

- 48 to 33,810 cities.

LBSA is compared with:

- a hybrid Genetic Algorithm–Particle Swarm Optimization–Ant Colony Optimization method,
- and MSA-IBS.

LBSA and MSA-IBS use:

- a population size of 10,
- 1,000 outer iterations,
- and a Markov-chain length equal to the number of cities.

Each method is run 20 times per instance.

The reported average percentage errors are:

- GA-PSO-ACO: 2.43%,
- MSA-IBS: 0.53%,
- LBSA: 0.49%.

LBSA and MSA-IBS outperform GA-PSO-ACO on all tested instances.

LBSA again performs slightly better than MSA-IBS on average.

Comparison with ASA-GS and MSA-IBS

The third large comparison uses 40 benchmark instances containing:

- 151 to 85,900 cities.

LBSA is compared with:

- Adaptive Simulated Annealing with Greedy Search, or ASA-GS,
- and MSA-IBS.

The algorithms are run 25 times on each instance.

The reported averages are:

Method	Average percentage error	Average CPU time
ASA-GS	1.87%	650.31
MSA-IBS	0.81%	283.52

LBSA	0.75%	282.49
------	-------	--------

LBSA produces:

- a lower average error than ASA-GS,
- less than half the average CPU time of ASA-GS,
- and slightly better average error and runtime than MSA-IBS.

Scalability

The largest instance contains 85,900 cities.

For this problem, the reported average percentage errors are:

- ASA-GS: 9.63%,
- MSA-IBS: 2.89%,
- LBSA: 2.21%.

The CPU times are approximately:

- ASA-GS: 8,855.13,
- MSA-IBS: 7,596.18,
- LBSA: 7,586.60.

LBSA therefore produces the strongest reported average solution quality on the largest problem while using slightly less time than MSA-IBS and substantially less than ASA-GS.

Main Findings

The paper's main findings are:

- Standard SA performance depends strongly on its cooling schedule and temperature parameters.
- A temperature list can replace a fixed cooling formula.
- LBSA adapts temperature according to objective-value differences observed during the search.
- Its temperature decreases rapidly early and more slowly later.
- LBSA converges faster than geometric and arithmetic cooling in the tested TSP instances.
- Its performance is relatively robust across a broad range of list lengths.

- It is also relatively insensitive to initial temperature values.
- Excessively long temperature lists can harm performance on large problems.
- Updating the list with average calculated temperature performs much better than using the maximum or minimum.
- Inversion is the best individual neighborhood operator among those tested.
- A hybrid of inversion, insertion, and swapping produces the strongest overall results.
- LBSA obtains low average errors across small and very large TSPLIB problems.
- It outperforms GSAACS, GA-PSO-ACO, and ASA-GS in the reported comparisons.
- It performs slightly better than MSA-IBS on average.
- It remains competitive on instances containing tens of thousands of cities.

Important Limitations

The study does not compare LBSA with every major TSP algorithm or exact optimization method.

Most comparisons are made using numerical results reported in earlier publications rather than by implementing every competing algorithm on identical hardware.

This creates possible differences involving:

- processors,
- programming languages,
- implementation quality,
- parameter tuning,
- stopping conditions,
- and computational budgets.

Some comparisons adjust the LBSA population size so that its CPU time remains below that of the competing method. This makes runtime comparisons less fully controlled.

The hybrid neighborhood evaluates three candidate moves and chooses the best one. Its improvement may therefore arise partly from additional neighborhood evaluations rather than from the cooling schedule alone.

The paper also uses fixed outer-loop limits and Markov-chain lengths, so the theoretically slow convergence requirements of classical SA are not examined.

Although the authors describe LBSA as parameter-insensitive, list length still becomes important for large instances. Very large lists can cause serious performance deterioration.

Conclusion

The authors conclude that List-Based Simulated Annealing provides a simple and effective adaptive cooling method for the Traveling Salesman Problem.

Its principal advantage is reduced sensitivity to manually selected temperature parameters.

The method treats its temperature list as a geometric-like schedule with a variable cooling coefficient. Because this coefficient responds to the search process, LBSA:

- leaves unproductive high-temperature stages quickly,
- spends more effort at promising lower temperatures,
- preserves the ability to escape local minima,
- and avoids the abrupt freezing associated with overly aggressive cooling.

The experimental results indicate that LBSA is:

- effective,
- computationally competitive,
- robust over broad parameter ranges,
- relatively easy to implement,
- and scalable to very large TSP instances.

The authors suggest that its parameter robustness could allow the same cooling strategy to be applied to other optimization problems with less manual tuning.

Vehicle Logistics Intermodal Route Optimization Based on Tabu Search Algorithm

Siyuan Ru

Summary

This paper proposes a vehicle-logistics route optimization method based on the **Tabu Search algorithm**. Its main objective is to reduce transportation costs and increase the profits of vehicle-logistics companies by combining several transportation modes instead of depending mainly on road transport.

The proposed system considers:

- road transportation,
- railway transportation,
- waterway transportation,
- manufacturers,
- distribution centers,
- transport stations and ports,
- and vehicle dealers.

The complete method has three major stages:

1. Determine suitable distribution-center locations.
2. Construct a multimodal transportation network and eliminate infeasible routes.
3. Use Tabu Search to optimize the remaining routes according to profit, cost, time, and capacity.

The study compares Tabu Search with:

- a Genetic Algorithm,
- Simulated Annealing,
- a Genetic Algorithm–Simulated Annealing hybrid,
- and local search.

The reported results indicate that Tabu Search produced lower transportation costs, lower route errors, higher accuracy, and larger route profits. However, one experiment also found that Tabu Search required substantially more computation time than the standalone Genetic Algorithm and Simulated Annealing methods.

Research Motivation

The paper explains that automotive logistics companies face increasingly narrow profit margins.

Vehicle transport has traditionally depended heavily on roads. However, changes in regulations have reduced the number of vehicles that may be carried in a single road shipment, increasing the cost per transported vehicle.

Other available transportation modes include:

- railway transportation,
- waterway transportation,
- and short-distance road connections.

A multimodal transportation system can potentially reduce costs by using the strengths of each mode.

However, simply adding more transportation modes does not guarantee improvement. A poorly designed combination may increase:

- total distance,
- transportation time,
- transfer costs,
- and handling requirements.

The study therefore seeks to determine appropriate combinations of transportation modes and routes.

Vehicle Logistics Characteristics

The paper identifies three important characteristics of vehicle logistics.

High Transportation Frequency and Small Batches

Vehicle shipments may occur frequently, but the number of vehicles in each shipment may be relatively small.

Complex Distribution Routes

Vehicles move through networks containing manufacturers, distribution centers, dealers, railway stations, ports, and transfer points.

High Safety Requirements

Finished vehicles are high-value products. Damage during transport can cause substantial financial losses.

The route-planning system must therefore consider more than distance alone.

Vehicle-Logistics Structure

Automotive logistics is divided into:

- parts logistics,
- and finished-vehicle logistics.

Parts logistics includes activities such as:

- procurement,
- production,
- sales,
- recycling,
- import,
- and export.

Finished-vehicle logistics concerns the movement of completed vehicles from manufacturers through distribution centers to dealers.

The paper's model focuses on the transportation of finished vehicles.

Distribution-Center Location

Before optimizing transportation routes, the paper first determines suitable distribution-center locations.

Because manufacturers and dealers already have fixed locations, the distribution center is treated as the location decision.

The paper initially uses a center-of-gravity method.

In plain text, the distribution-center coordinates are calculated as:

$$x = \text{sum}(w_j \times X_j) / \text{sum}(w_j)$$

$$y = \text{sum}(w_j \times Y_j) / \text{sum}(w_j)$$

Where:

- X_j and Y_j are the coordinates of dealer j .
- w_j is the demand associated with dealer j .
- x and y are the estimated distribution-center coordinates.

The weighted center places the distribution center closer to dealers with greater demand.

P-Median Model

The study also uses a **P-median model**.

The P-median problem selects a specified number of facility locations so that the total distance or cost between demand points and their assigned facilities is minimized.

The model uses binary encoding:

- 1 means that a candidate city is selected as a distribution center.
- 0 means that it is not selected.

For example, when $P = 3$, exactly three candidate locations must be selected.

Each dealer is assigned to one selected distribution center.

Improved Two-Level Location Model

The authors state that the traditional P-median model considers only transportation costs between dealers and distribution centers.

It does not fully account for:

- construction costs,
- manufacturer production capacity,
- vehicle type,
- manufacturer-to-distribution-center transportation,
- dealer demand,
- or distribution-center capacity.

The paper therefore proposes an improved two-level model.

Its cost objective contains five broad components:

1. Transportation of small vehicles from distribution centers to dealers
2. Transportation of large vehicles from distribution centers to dealers
3. Transportation of small vehicles from manufacturers to distribution centers
4. Transportation of large vehicles from manufacturers to distribution centers
5. Distribution-center construction costs

In simplified plain text:

Total location cost = dealer transport costs + manufacturer transport costs + distribution-center construction costs

Location-Model Constraints

The model includes constraints for:

- satisfying each dealer's demand for small vehicles,
- satisfying each dealer's demand for large vehicles,
- not exceeding manufacturers' small-vehicle production capacity,
- not exceeding manufacturers' large-vehicle production capacity,
- balancing vehicles shipped from manufacturers with vehicles distributed to dealers,
- and not exceeding distribution-center capacity.

Examples of the constraints in copy-safe notation are:

sum(sij across all distribution centers i) = small-vehicle demand of dealer j

sum(bij across all distribution centers i) = large-vehicle demand of dealer j

sum(wbik across all distribution centers i) <= large-vehicle capacity of manufacturer k

sum(wsik across all distribution centers i) <= small-vehicle capacity of manufacturer k

The notation describes how many small and large vehicles move among manufacturers, distribution centers, and dealers.

Greedy Removal Heuristic

The improved location model cannot be solved quickly through direct calculation.

The paper therefore uses a greedy removal heuristic.

The procedure is:

1. Initially treat all candidate sites as selected distribution centers.
2. Assign manufacturers and dealers to the candidate centers while satisfying the constraints.
3. Select one distribution center for possible removal.
4. Remove it when the remaining assignment still satisfies the constraints.
5. Repeat until only the required number of distribution centers remains.

This procedure gradually reduces the candidate set instead of searching through every possible combination.

Multimodal Transportation Network

After locating distribution centers, the study builds a multimodal logistics network.

The network contains three basic element types:

Nodes

Nodes include:

- manufacturers,
- distribution centers,
- railway stations,
- water ports,
- and dealers.

Edges

Edges represent:

- road transportation,
- railway transportation,
- and waterway transportation.

Weights

Edge weights include:

- transportation capacity,
- transportation time,
- transportation distance,
- and transportation cost.

Expanded Transportation Nodes

A physical location may support more than one transportation mode.

The authors argue that representing it as one ordinary node would incorrectly imply that all transportation modes share identical characteristics.

The model therefore expands one physical transportation location into separate mode-specific nodes, such as:

- a road node,
- a railway node,
- and a waterway node.

Each expanded node can have different:

- capacities,
- transportation costs,
- handling costs,
- and travel times.

The destination node is not expanded in the same way.

Transportation Assumptions

The optimization model makes several assumptions:

- Each dealer's demand is supplied by only one manufacturer.
- Manufacturer, dealer, station, port, and distribution-center locations are known.
- Distances between nodes are known.
- Node capacities are known.
- Transportation costs are known.
- Dealer demand is known.

- Large transport vehicles are not explicitly modeled as transported products.
- Vehicle transportation uses combinations of road, railway, waterway, and short-distance connecting transport.

Route Cost Model

The cost of a route includes several categories:

- road mileage cost,
- railway mileage cost,
- waterway mileage cost,
- short-distance connection cost,
- manufacturer pickup and transfer cost,
- railway-station handling cost,
- port handling cost,
- and distribution-center handling cost.

In simplified notation:

`Route cost = road cost + railway cost + waterway cost + connection cost + transfer and handling costs`

For the entire transportation system:

`Total network cost = sum of all route costs`

Revenue Model

The revenue model contains corresponding income components for:

- road or mileage-based vehicle transport,
- railway transport,
- waterway transport,
- short-distance connections,
- manufacturer transfers,
- railway-station transfers,
- port transfers,
- and distribution-center transfers.

In simplified notation:

`Route revenue = transport income + pickup income + transfer income`

For the complete system:

Total network revenue = sum of all route revenues

Profit Objective

The main objective is to maximize transportation-network profit.

The formula can be written safely as:

Maximum profit = Total network revenue - Total network cost

The model therefore does not simply seek the route with the shortest distance. A longer multimodal route may be selected when it produces greater overall profit while satisfying all constraints.

Capacity and Time Constraints

Routes must satisfy two major categories of constraints.

Capacity Constraints

These include the capacities of:

- distribution centers,
- railway stations,
- and water ports.

Time Constraints

Each dealer has a maximum acceptable transportation time.

A route is excluded when it exceeds this limit, even when the route could otherwise produce high profit.

The algorithm therefore first removes infeasible routes before performing profit optimization.

Restricted Graph-Traversal Algorithm

The paper proposes a restricted graph-traversal procedure for finding feasible routes.

The process is:

1. Input the transportation data and time limits.
2. Set the starting node as the current node.
3. Visit each neighboring node.
4. Calculate the accumulated travel time.
5. Continue only when the accumulated time remains within the limit.
6. Record the path when the destination is reached.
7. Backtrack to the previous node.
8. Continue until all permitted paths have been considered.

This process removes routes that violate time constraints.

The remaining feasible paths become the search space used by Tabu Search.

Tabu Search Procedure

The initial solution is the logistics company's existing transportation plan.

The paper argues that a good initial solution can improve the speed and quality of Tabu Search.

The procedure is:

1. Input the existing transportation routes as the initial solution.
2. Use all feasible routes as the search space.
3. Initialize an empty tabu list.
4. Generate neighboring solutions by replacing one dealer's assigned delivery route.
5. Remove routes that violate the time constraints.
6. Evaluate each remaining solution using total enterprise profit.
7. Select a candidate route arrangement.
8. Store the selected solution or movement in the tabu list.
9. Update the tabu list.
10. Check the stopping rule.
11. Continue until termination.

Neighborhood Structure

A solution is represented as a set of dealer routes.

Each element identifies the route used to supply one dealer.

A neighboring solution is created by:

- selecting one dealer,
- replacing that dealer's route with another feasible route,
- and keeping the remaining dealer routes unchanged.

With 57 dealers, the neighborhood initially contains approximately one replacement candidate for each dealer before constraint filtering.

Tabu Step Length

The paper refers to tabu tenure as the **tabu step length**.

A short step length:

- requires less memory,
- leaves more moves available,
- and creates a broader immediate search range.

However, it may permit cycling or local-optimum entrapment.

A long step length:

- prevents more repeated movements,
- but may prohibit too many useful moves,
- and may reduce computational efficiency.

The paper limits the step size to an integer no larger than 10.

The selected value is:

Tabu step length = 6

Tabu Search Parameters

The reported parameter settings are:

Parameter	Value
-----------	-------

Maximum iterations 190

Tabu step length 6

Number of dealers 57

Neighborhood size 57

Initial solution Original transportation plan

The evaluation function is enterprise profit.

The aspiration or amnesty criterion is determined using:

- objective-function improvement,
- and repetition or cycling behavior.

Algorithm Comparison

The authors compare:

- Tabu Search,
- Simulated Annealing,
- and a Genetic Algorithm.

The algorithms are implemented in MATLAB 2016b.

The Simulated Annealing and Genetic Algorithm parameter settings are based on previously published studies.

The main comparison measures are:

- converged transportation cost,
- number of iterations,
- computational time,
- absolute error,

- relative error,
- accuracy,
- recall,
- and receiver operating characteristic curves.

Transportation-Cost Results

The reported converged transportation costs are:

Algorithm	Converged cost
Tabu Search	1.2 yuan per kilometer per vehicle set
Simulated Annealing	3.2 yuan per kilometer per vehicle set
Genetic Algorithm	3.8 yuan per kilometer per vehicle set

Tabu Search produces the lowest reported transportation cost.

The paper states that:

- Tabu Search converges at approximately iteration 18.
- The Genetic Algorithm converges after approximately 12 iterations.
- Simulated Annealing converges after approximately 35 iterations.

Although the Genetic Algorithm requires fewer iterations, it converges to a much higher transportation cost.

Runtime Results

In the primary algorithm-performance comparison:

- Tabu Search requires more than 100 seconds.

- Simulated Annealing requires less than 10 seconds.
- The Genetic Algorithm requires less than 10 seconds.

Thus, the stronger cost result of Tabu Search comes with a substantial runtime disadvantage in this experiment.

The paper later reports a separate route-planning comparison in which Tabu Search runs in between 1.26 and 3.56 seconds. This appears to be based on a different experiment or evaluation procedure and should not be treated as the same runtime measurement.

Error Results

The maximum absolute errors are reported as:

Algorithm	Maximum absolute error
Tabu Search	0.20 yuan per kilometer per vehicle set
Genetic Algorithm	0.63 yuan per kilometer per vehicle set
Simulated Annealing	0.64 yuan per kilometer per vehicle set

Tabu Search produces the lowest error across the examined routes.

The graphical results also show its relative error remaining below those of the other algorithms.

Accuracy Results

The reported starting and converged accuracy values include:

Algorithm	Initial accuracy	Accuracy after convergence
Tabu Search	77%	About 95%
Genetic Algorithm	68%	About 81%
Simulated Annealing	Not clearly stated numerically	About 80%–81% in the graph

Tabu Search reaches the highest reported accuracy.

Recall Results

After convergence, the reported recall values are approximately:

Algorithm	Recall
Tabu Search	92%
Genetic Algorithm	78%
Simulated Annealing	77%

Tabu Search again produces the highest reported value.

ROC-Curve Results

The algorithms are evaluated using receiver operating characteristic curves on:

- the NGSIM dataset,
- and a second dataset referred to as the high-D dataset.

The ROC curves plot:

- true-positive rate,
- against false-positive rate.

In both datasets, the Tabu Search curve has a visibly larger area under the curve than the curves for the Genetic Algorithm and Simulated Annealing.

The authors interpret this as evidence that Tabu Search performs more strongly and that its result is not limited to one dataset.

However, the paper does not clearly explain how classification-oriented metrics such as accuracy, recall, and ROC are mapped onto the route-optimization problem.

Company X Case Study

The complete route-optimization model is applied to a logistics company identified as **Company X**.

The company has:

- two manufacturers,
- 57 dealers,
- five existing distribution centers,
- and plans to establish four additional distribution centers.

Before optimization:

- road transport accounts for approximately 87% of the company's routes,
- only a small number of route sections use railway transport,
- and water transportation is not used.

The improved P-median model is solved using LINGO to determine the new distribution-center locations.

Tabu Search is then applied to optimize the company's multimodal transportation routes.

Route-Mode Results

The analysis focuses on 15 routes.

After optimization:

- Routes 1, 3, 4, and 8 continue to use only one transportation mode.
- Routes 2, 7, 9, 14, and 15 change from single-mode to multimodal transport.
- Most remaining routes use all three main modes: road, railway, and waterway.

Therefore:

- 4 routes remain unchanged,
- and 11 routes are optimized.

Profit Results

The profits of Routes 1, 3, 4, and 8 remain unchanged.

The profits of the other 11 routes increase.

Route 9 experiences the largest reported improvement:

Route 9 profit increase = 18%

The results suggest that the introduction of multimodal transportation increases profit on most routes.

Additional Optimization Comparison

The paper also presents a separate comparison involving:

- SSA,
- GA,
- an SSA–GA hybrid,
- local search,
- and Tabu Search.

The abbreviation SSA is used in the table, although the surrounding text describes it as a Simulated Annealing method.

The results are:

Algorithm	Runtime	Maximum deviation	Average deviation
SSA	5.64–7.24 seconds	94.5%	26.4%
GA	4.38–8.54 seconds	89.6%	25.1%
SSA–GA	2.25–6.38 seconds	83.7%	22.3%
Local search	4.97–9.84 seconds	92.4%	32.8%
Tabu Search	1.26–3.56 seconds	0%	0%

Tabu Search is treated as the reference or optimal result, which explains why its deviation is reported as zero.

This table indicates a much shorter Tabu Search runtime than the earlier comparison reporting more than 100 seconds. The paper does not clearly reconcile the two results.

Main Findings

The paper's major findings are:

- Multimodal transport can reduce logistics costs and increase enterprise profit.
- Distribution-center placement should be optimized before route selection.
- An improved P-median model can incorporate construction costs, production capacities, vehicle types, and dealer demand.
- Graph traversal can eliminate routes that violate transportation-time constraints.
- Tabu Search can optimize the remaining route and transportation-mode combinations.

- Tabu Search obtains the lowest reported transportation cost among the three main algorithms.
- It produces lower absolute and relative errors.
- It achieves higher reported accuracy and recall.
- Its ROC curve has a larger area on both evaluated datasets.
- Eleven of the 15 Company X routes are changed through optimization.
- All changed routes produce higher profits.
- Route 9 achieves the largest profit increase at 18%.
- The original road-heavy transportation system becomes more multimodal after optimization.

Important Limitations and Inconsistencies

Vehicle Loading Capacity

The authors acknowledge that the model does not comprehensively account for the loading capacity of the actual transportation vehicles.

They identify this as an area for future refinement.

Conflicting Runtime Results

The paper first states that Tabu Search requires more than 100 seconds, compared with less than 10 seconds for the Genetic Algorithm and Simulated Annealing.

A later table reports Tabu Search completing in only 1.26 to 3.56 seconds.

The paper does not clearly explain whether these results come from:

- different datasets,
- different experiments,
- different stopping conditions,
- or different implementations.

Unclear Use of Accuracy, Recall, and ROC

Accuracy, recall, true-positive rate, false-positive rate, and ROC curves are normally associated with classification tasks.

The paper does not provide a sufficiently clear explanation of:

- what is classified as positive or negative,

- what constitutes a true positive,
- or how these metrics represent route quality.

Unclear Baseline in Deviation Table

The later comparison gives Tabu Search zero maximum and average deviation.

This appears to result from defining the Tabu Search solution as the reference optimum rather than independently proving that it is globally optimal.

Limited Company Case

The business application uses one company with:

- two manufacturers,
- 57 dealers,
- and 15 highlighted routes.

The findings may not automatically generalize to other vehicle-logistics networks.

Algorithm-Comparison Fairness

The competing algorithms use parameters taken from previous references.

The paper does not clearly establish that all methods receive equivalent:

- computational budgets,
- tuning effort,
- hardware conditions,
- neighborhood evaluations,
- or stopping rules.

Conclusion

The paper concludes that a combination of:

- distribution-center optimization,
- constrained graph traversal,
- and Tabu Search

can improve multimodal vehicle-logistics planning.

The distribution-center model determines where facilities should be located. Graph traversal removes routes that violate time constraints. Tabu Search then selects profitable combinations of routes and transportation modes.

In the reported experiments, Tabu Search produces:

- the lowest transportation cost,
- the smallest error,
- the highest accuracy and recall,
- and the largest ROC-curve area.

In the Company X case study, 11 of 15 routes are optimized, and every changed route produces increased profit. The largest increase is 18% for Route 9.

The paper therefore presents Tabu Search as an effective method for vehicle-logistics route optimization, while acknowledging that the model requires further refinement to represent vehicle loading capacity more comprehensively.

Tabu Search and Particle Swarm Optimization Algorithms for Two Identical Parallel Machines Scheduling Problem with a Single Server

Ibrahim Alharkan, Mustafa Saleh, Mageed A. Ghaleb,
Husam Kaid, Abdulsalam Farhan, and A. Almarfadi

Summary

This paper develops and evaluates two metaheuristic algorithms for a manufacturing scheduling problem involving:

- two identical parallel machines,
- one shared setup server,
- and a collection of independent jobs.

The two proposed methods are:

- **Tabu Search**
- **Geometric Particle Swarm Optimization**

The server must prepare or load a job onto a machine before that machine can begin processing it. Because only one server is available, setup operations cannot occur simultaneously, even though the two machines may process jobs at the same time.

The main objective is to minimize the **makespan**, which is the completion time of the last finished job.

The proposed algorithms are compared with three previously published methods:

- Simulated Annealing
- Genetic Algorithm
- an Iterated Local Search method called I-L

The experiments use benchmark instances ranging from 8 to 1,000 jobs. The main findings indicate that Tabu Search and Geometric PSO perform strongly on medium and large instances. Tabu Search is particularly successful at reaching the known lower bound, while Geometric PSO generally provides close-to-lower-bound results and performs consistently on large instances.

Scheduling Problem

The study considers a set of n independent jobs that must be processed by two identical parallel machines.

Each job has:

- a setup or loading time,
- and a processing time.

The setup must be completed by the single shared server before processing can begin.

The main resources are:

- Machine 1
- Machine 2
- one shared setup server

Only one setup can be performed at a time because there is only one server.

However, after a setup is complete, the machine processes the job independently. The server may then move to the other machine and prepare another job.

Practical Meaning of the Server

The shared server may represent:

- a worker,
- a robot,
- a loading device,
- a maintenance technician,
- or a specialized tool.

The model can apply to areas such as:

- automated material-handling systems,
- semiconductor production,
- maintenance scheduling,
- textile manufacturing,
- and network computing.

The shared server creates additional idle time because a machine may be ready but unable to begin a job until the server becomes available.

Problem Assumptions

The paper makes the following assumptions:

- There are two identical parallel machines.
- Every job is processed by exactly one machine.
- Each job requires one setup before processing.
- Setup and processing times are known in advance.
- The setup server can prepare only one job at a time.
- Each machine can process only one job at a time.
- Setup and processing are non-preemptive.
- Once a setup or processing operation begins, it cannot be interrupted.
- All jobs, machines, and the server are available at time zero.
- A job begins processing immediately after its setup is completed.

Important Variables

The paper uses the following notation:

- n = number of jobs
- m = number of machines
- j = index of a job
- i = index of a machine
- p_j = processing time of job j
- s_j = setup time of job j
- $\text{setup_start_}j$ = starting time of setup for job j
- C_j = completion time of job j
- $C_{\max}$ = makespan

The completion time of a job can be written as:

$$C_j = \text{setup starting time} + \text{setup time} + \text{processing time}$$

The optimization objective is:

Minimize $C_{\max}$

Where:

$C_{\max}$ = maximum completion time among all jobs

Why the Problem Is Difficult

The scheduling problem is strongly NP-hard.

The difficulty does not come only from assigning jobs to two machines. The algorithm must also coordinate:

- the order of jobs,
- machine availability,
- server availability,
- setup operations,
- and processing operations.

A poor sequence may cause:

- the server to wait for a machine,
- a machine to wait for the server,
- or one machine to finish much later than the other.

The objective is therefore to create a sequence that reduces both machine idle time and server waiting time.

Schedule Construction

The optimization algorithms generate a permutation or sequence of jobs.

The jobs are then assigned greedily to the next available machine.

For each job in the sequence:

1. Determine which machine becomes available first.
2. Wait until both that machine and the setup server are available.
3. Use the server to prepare the job.
4. Begin processing immediately after setup.
5. Continue with the next job in the sequence.

The job sequence therefore determines the resulting machine assignments and makespan.

Eight-Job Example

The paper provides an example containing eight jobs.

The sequence is:

3, 1, 8, 5, 2, 4, 6, 7

The jobs are assigned as follows:

- Machine 1 processes jobs 3, 8, 2, and 6.
- Machine 2 processes jobs 1, 5, 4, and 7.

The resulting makespan is:

$C_{\max} = 35$

The Gantt chart distinguishes:

- setup periods,
- processing periods,
- server waiting,
- and machine waiting.

For example:

- The server sometimes waits because both machines are still processing.
- A machine may also wait because the server is setting up a job on the other machine.

The example demonstrates that the main scheduling challenge is coordinating all three resources.

Lower Bound

The paper uses a lower bound developed in earlier research to evaluate solution quality.

The lower bound is:

$LB = \text{maximum of } LB1, LB2, \text{ and } LB3$

The components are written in copy-safe form below.

$LB1 = 0.5 \times [\text{sum of all setup and processing times} + \text{minimum setup time}]$

$LB2 = \text{sum of all setup times} + \text{minimum processing time}$

$LB3 = \text{minimum value of (setup time} + \text{processing time) among all jobs}$

The lower bound represents the smallest makespan that could theoretically be expected under simplified resource constraints.

When an algorithm produces:

$$C_{\max} = LB$$

the solution is treated as optimal relative to the bound.

However, when the makespan is greater than the lower bound, the paper cannot always prove whether the lower bound itself is attainable.

Solution-Quality Measure

The paper evaluates a solution using the ratio:

$$\text{Error ratio} = C_{\max} / LB$$

Interpretation:

- $\text{Error ratio} = 1.00$ means that the solution reaches the lower bound.
- A value slightly above 1 means that the solution is slightly worse than the lower bound.
- Lower values are better.

The study reports two forms of this measure:

- R_{avg} = average error ratio across instances
- R_{max} = maximum error ratio among the instances

Tabu Search Method

Tabu Search uses one current job sequence and repeatedly generates neighboring sequences.

The method uses memory to avoid cycling back to recently visited moves.

Its main components are:

- initial solution,
- neighborhood operator,
- tabu list,
- aspiration criterion,
- and stopping condition.

Tabu Search Initial Solution

The initial sequence is created using the **Longest Processing Time rule**.

Jobs are arranged in non-increasing order of processing time.

In simple terms:

1. Place the job with the longest processing time first.
2. Continue in descending processing-time order.
3. Use the resulting sequence as the initial Tabu Search solution.

Tabu Search Neighborhood

The study uses a pairwise-interchange or swap operator.

To create one neighboring solution:

1. Select two jobs randomly.
2. Exchange their positions in the sequence.
3. Decode the new sequence into a two-machine schedule.
4. Calculate its makespan.

Each Tabu Search iteration generates:

70 candidate neighboring solutions

The best admissible candidate becomes the new current solution.

Tabu List

The tabu list stores information about recent moves.

Its purpose is to prevent the search from immediately reversing a swap or repeating a recent search pattern.

The selected tabu-list size is:

7 elements

The list uses a first-in, first-out procedure:

- the newest tabu item is added,
- and the oldest item is removed when the list is full.

Aspiration Criterion

A tabu move may still be accepted when it produces a solution better than the best solution found so far.

In plain terms:

Accept a tabu move when its makespan is lower than the best-known makespan.

This prevents the tabu list from blocking a major improvement.

Tabu Search Stopping Conditions

The algorithm stops when:

- the lower bound is reached,
- or the maximum computational time is reached.

The time limits are aligned with the benchmark studies.

For small instances, the time limit is:

$\text{Time limit} = (300 / 8) \times n \text{ seconds}$

For large instances, the time limit is:

3,600 seconds

Using the same time limit is intended to make the algorithm comparison fairer.

Basic Tabu Search Procedure

The procedure can be summarized as follows:

1. Generate the Longest Processing Time initial sequence.
2. Store it as the current and best-known solution.
3. Initialize the tabu list.

4. Generate 70 swap-based candidate solutions.
5. Remove tabu candidates unless the aspiration criterion applies.
6. Select the admissible candidate with the lowest makespan.
7. Move to that candidate.
8. Update the tabu list.
9. Update the best-known solution when an improvement is found.
10. Stop after reaching the lower bound or time limit.

Tabu Search Implementation Challenges

The paper identifies several practical challenges in Tabu Search:

- designing the tabu list,
- determining the appropriate tabu-list size,
- selecting a neighborhood structure,
- choosing which move attributes to store,
- managing prohibited moves,
- and defining an effective aspiration rule.

The method may be sensitive to tabu-list size.

A list that is too short may allow cycling, while an excessively long list may block useful moves.

Standard Particle Swarm Optimization

Standard PSO represents each solution using a numerical position and velocity.

Each particle is influenced by:

- its previous movement,
- its personal best position,
- and the best position found by the entire swarm.

This structure is naturally suited to continuous numerical optimization.

However, the scheduling problem uses job permutations rather than continuous vectors.

For example, a schedule may be represented as:

3, 1, 8, 5, 2, 4, 6, 7

Ordinary velocity addition cannot be applied directly to such a sequence.

Geometric Particle Swarm Optimization

The paper therefore uses **Geometric Particle Swarm Optimization**, or GPSO.

GPSO is a discrete form of PSO designed for combinatorial representations.

Instead of updating:

- velocity first,
- and position second,

GPSO updates the particle's permutation directly.

It uses a crossover operation inspired by evolutionary algorithms.

Three Parents in GPSO

A particle's new position is created from three parent sequences:

1. The particle's current or previous position
2. The particle's personal-best position
3. The swarm's global-best position

The update can be described as:

`New particle = 3PMX(previous particle, personal best, global best)`

Where 3PMX means **three-parent mask-passed crossover**.

GPSO Weights

Three weights determine how likely each position is to inherit information from each parent:

- `w1` = weight for the particle's previous position
- `w2` = weight for the personal-best position
- `w3` = weight for the global-best position

The selected values are:

- `w1 = 0.2`
- `w2 = 0.4`

- $w_3 = 0.4$

This means that the personal best and global best each have twice the influence of the previous position.

Three-Parent Mask-Passed Crossover

The crossover uses a mask containing values 1, 2, or 3.

Each mask value indicates which parent should supply the job at that sequence position:

- 1 means use the previous particle.
- 2 means use the personal-best particle.
- 3 means use the global-best particle.

Because each job must appear exactly once, the parent permutations are adjusted through swaps as the new child is created.

This ensures that the resulting sequence remains a valid permutation.

GPSO Example

The paper uses the following three parents:

Previous particle:

3, 1, 2, 4, 5

Personal best:

5, 1, 3, 2, 4

Global best:

1, 5, 4, 2, 3

The weights are:

- previous position: 0.2
- personal best: 0.4
- global best: 0.4

A random mask is generated:

3, 1, 2, 2, 3

The algorithm processes the mask position by position and rearranges the parent permutations to preserve valid job sequences.

The resulting new particle is:

1, 3, 5, 2, 4

GPSO Mutation

After the three-parent crossover, a mutation is applied.

The selected mutation is a swap:

1. Randomly select two positions.
2. Exchange their job values.

Mutation introduces additional variation and creates a nearby candidate that may not be generated through crossover alone.

GPSO Procedure

The GPSO process is:

1. Randomly initialize a swarm of job permutations.
2. Evaluate the makespan of every particle.
3. Record each particle's personal best.
4. Record the global-best particle.
5. Generate a new position using the three-parent crossover.
6. Apply swap mutation.
7. Evaluate the resulting schedule.
8. Update personal-best and global-best positions.
9. Repeat until the lower bound or time limit is reached.

GPSO Parameter Tuning

The paper uses a full-factorial experimental design to select GPSO parameters.

The tested swarm sizes are:

- 30
- 50
- 70

The tested mutation methods are:

- swap
- insertion

The tested weight values are:

- **w1**: 0.1, 0.2, and 0.3
- **w2**: 0.3, 0.4, and 0.5
- **w3**: 0.3, 0.4, and 0.5

The selected configuration is:

GPSO parameter	Selected value
Swarm size	70
Mutation method	Swap
Previous-position weight, w1	0.2
Personal-best weight, w2	0.4
Global-best weight, w3	0.4

Experimental Setup

Both proposed algorithms are implemented in MATLAB.

The experiments are conducted on a computer with:

- Intel Core i7-4702MQ processor at 2.2 GHz
- 16 GB RAM

The benchmark instances are taken from previous studies so that the proposed algorithms can be compared with existing results.

Problem Sizes

The tested job counts are:

- 8
- 20
- 50
- 100
- 200
- 250
- 300
- 400
- 500
- 600
- 700
- 800
- 900
- 1,000

This allows the authors to examine performance from very small to very large scheduling instances.

Server-Load Parameter

The instances also vary according to a server-load parameter called L .

The tested values are:

- 0.1
- 0.5
- 0.8
- 1.0

- 1.5
- 1.8
- 2.0

Processing times are generated uniformly from:

0 to 100

Setup times are generated uniformly from:

0 to $100 \times L$

A larger L generally means that setup operations consume more time relative to processing operations.

Number of Instances

For each combination of job count and server load:

- five instances are generated,
- except for 8-job and 20-job problems, where ten instances are generated.

The experiment therefore covers a large and varied collection of scheduling conditions.

Benchmark Algorithms

The proposed methods are compared with:

Simulated Annealing

A temperature-based local-search method that sometimes accepts worse job sequences.

Genetic Algorithm

A population-based evolutionary method using selection, crossover, and mutation.

I-L Algorithm

An Iterated Local Search method developed for very large instances of the same scheduling problem.

The results for these three algorithms come from the earlier benchmark study rather than from fresh implementations created by the current authors.

Difficulty by Server Load

The paper finds that instances with the following L values are particularly difficult:

- 0.8
- 1.0
- 1.5

For these values, the algorithms frequently fail to reach the lower bound.

Instances with the following values are generally easier:

- 0.1
- 0.5
- 1.8
- 2.0

The difficulty therefore does not increase monotonically with setup time.

Intermediate balances between setup and processing time appear to create the most difficult schedules.

Worst Average Error Values

The largest observed R_{avg} values for each algorithm are:

Algorithm	Largest average error ratio
Tabu Search	1.031613
GPSO	1.035704

Simulated Annealing 1.050076

I-L 1.103816

Genetic Algorithm 1.108872

Because values closer to 1 are better, Tabu Search produces the best worst-case average result.

GPSO ranks second.

Interpretation of the Worst Average Errors

The largest Tabu Search average error of 1.031613 means that, in its most difficult tested configuration, the average makespan was approximately:

3.16% above the lower bound

For GPSO, the largest average error of 1.035704 is approximately:

3.57% above the lower bound

For the Genetic Algorithm, the largest average error of 1.108872 is approximately:

10.89% above the lower bound

Small-Instance Results

For the smallest problems:

- Simulated Annealing and the Genetic Algorithm often perform very strongly.
- At 8 jobs, Simulated Annealing reaches the lower bound for every tested server-load value.
- Tabu Search and GPSO are not consistently superior on the smallest instances.
- The initialization and overhead of the proposed metaheuristics may provide less advantage when the search space is small.

Medium-Instance Results

For approximately 200 to 600 jobs:

- all algorithms produce relatively similar average error ratios,
- and their results are generally close to the lower bound.

The performance differences are less pronounced in this range.

Large-Instance Results

For 700 or more jobs:

- Tabu Search, GPSO, Simulated Annealing, and I-L perform more consistently than the Genetic Algorithm.
- The Genetic Algorithm's error increases substantially as the number of jobs approaches 1,000.
- GPSO performs particularly consistently on large instances.
- Tabu Search maintains strong results and frequently reaches the lower bound.

Performance at 1,000 Jobs

At 1,000 jobs and $L = 1.0$, the reported average error ratios are:

Algorithm	Average error ratio
I-L	1.010157
GPSO	1.022207
Tabu Search	1.031613
Simulated Annealing	1.050076

For this specific difficult condition, I-L produces the lowest average error.

However, the broader results still show Tabu Search reaching the lower bound more frequently across large-instance categories.

Reaching the Lower Bound

The study also calculates the percentage of instances for which each algorithm reaches the lower bound.

This is treated as an indication that the algorithm may have found an optimal solution.

The percentage generally decreases as the number of jobs increases.

Simulated Annealing Lower-Bound Results

At 8 jobs:

Simulated Annealing reaches the lower bound in 100% of the tested instances.

As problem size increases, its lower-bound success rate declines.

For more than 700 jobs, the reported lower-bound success rate of Simulated Annealing becomes zero.

Tabu Search Lower-Bound Results

Tabu Search achieves the strongest overall success rate for reaching the lower bound on large instances.

The paper states that Tabu Search outperforms all other methods in lower-bound attainment for instances greater than 200 jobs.

The exceptions noted in the conclusion are:

- 8-job instances
- 200-job instances

For most other sizes, Tabu Search reaches the lower bound on the largest proportion of instances.

GPSO Lower-Bound Results

GPSO generally ranks behind Tabu Search.

For problems containing more than 400 jobs, GPSO reaches the lower bound more often than:

- Simulated Annealing
- and the Genetic Algorithm

This indicates that the discrete swarm approach scales reasonably well.

Runtime Comparison

The authors do not present one algorithm as faster based on CPU time because all methods are assigned the same maximum runtime.

The comparison is therefore based primarily on:

- solution quality within the same time allowance,
- error relative to the lower bound,
- and frequency of reaching the lower bound.

This is an important distinction.

The study does not show that Tabu Search uses less time than the other methods. It shows that under the same time allowance, Tabu Search often obtains better large-instance solutions.

Main Findings

The paper's major findings are:

- Scheduling two machines with one shared setup server is strongly NP-hard.
- Server waiting and machine waiting both affect makespan.

- The job sequence strongly influences the final machine schedule.
- Tabu Search and Geometric PSO can both solve large instances effectively.
- Standard continuous PSO must be substantially modified for permutation scheduling.
- GPSO uses three-parent crossover instead of conventional velocity updates.
- Tabu Search uses swap neighborhoods and short-term memory.
- Tabu Search produces the lowest worst-case average error ratio among the compared algorithms.
- GPSO produces the second-lowest worst-case average error ratio.
- Simulated Annealing and the Genetic Algorithm perform strongly on some small instances.
- Intermediate server-load values are generally more difficult than very low or very high values.
- The algorithms perform similarly for many instances containing 200 to 600 jobs.
- The Genetic Algorithm deteriorates substantially on very large instances.
- Tabu Search reaches the lower bound most frequently for most problem sizes above 200 jobs.
- GPSO reaches the lower bound more often than SA and GA for instances above 400 jobs.
- Effective sequencing reduces both machine idle time and server waiting time.

Important Limitations

Benchmark Results Were Taken from Earlier Research

The proposed TS and GPSO methods were implemented by the current authors.

However, the SA, GA, and I-L results were taken from a previous study.

Although the same instances and time limits were used, differences may remain in:

- hardware,
- programming implementation,
- software environment,
- parameter tuning,
- and coding efficiency.

Lower Bound Is Not Always a Proven Optimum

Reaching the lower bound indicates an optimal solution.

However, failing to reach it does not prove that the algorithm failed to find the true optimum because the lower bound may not always be attainable.

Limited Machine Configuration

The study considers exactly:

- two identical machines,
- and one shared server.

The findings do not automatically apply to:

- more than two machines,
- non-identical machines,
- multiple servers,
- or unrelated parallel machines.

One Main Objective

The only optimization objective is makespan.

The study does not optimize:

- total tardiness,
- average completion time,
- number of late jobs,
- energy use,
- or setup cost.

Randomly Generated Instances

The test data are randomly generated benchmark instances.

The paper does not evaluate a real factory dataset.

Limited Tabu Search Neighborhood

Tabu Search uses only a pairwise swap operator.

Other operators, such as:

- insertion,
- block movement,
- reversal,
- or adaptive neighborhoods

could potentially change its performance.

GPSO Complexity

GPSO requires:

- a specialized permutation crossover,
- mutation,
- personal-best tracking,
- global-best tracking,
- and several parameter values.

Its implementation is more complicated than standard continuous PSO.

Parameter Dependence

The TS results depend on choices such as:

- tabu-list size,
- number of generated candidates,
- and initial sequence.

The GPSO results depend on:

- swarm size,
- three parent weights,
- mutation operator,
- and crossover design.

Future Work

The authors recommend extending the problem to:

- more than two machines,
- arbitrary numbers of machines,
- and one shared server.

They also suggest considering additional objective functions such as:

- total tardiness,
- and total completion time.

Other heuristics and metaheuristics could also be developed and compared.

Conclusion

The paper concludes that Tabu Search and Geometric Particle Swarm Optimization are effective methods for scheduling two identical parallel machines that share one setup server.

Both methods generate job permutations and then construct schedules by assigning jobs greedily according to machine and server availability.

Tabu Search achieves the strongest overall result in terms of:

- worst-case average error,
- and frequency of reaching the lower bound on large instances.

GPSO also performs well, particularly as problem size grows, and generally ranks second in lower-bound attainment.

The findings show that high-quality scheduling depends heavily on sequencing jobs so that:

- machines spend less time idle,
- the server spends less time waiting,
- and neither machine waits unnecessarily for setup.

The study presents both methods as practical approaches for medium- and large-scale scheduling problems where exact mathematical optimization becomes computationally expensive.

Multi-Objective Flexible Job-Shop Scheduling Problem Using Modified Discrete Particle Swarm Optimization

Song Huang, Na Tian, Yan Wang, and Zhicheng Ji

Summary

This paper proposes a modified **Multi-Objective Particle Swarm Optimization algorithm**, called **MOPSO**, for solving the **Flexible Job-Shop Scheduling Problem**.

The Flexible Job-Shop Scheduling Problem is more complex than ordinary job-shop scheduling because each operation may be processed by one of several eligible machines. The scheduling system must therefore decide both:

- which machine should process each operation,
- and the order in which operations should be processed.

The paper considers three objectives simultaneously:

1. Minimize makespan.
2. Minimize total machine workload.
3. Minimize the workload of the most heavily used machine.

To solve the problem, the authors modify standard continuous PSO so that it can operate on discrete schedules. The proposed method includes:

- a two-vector schedule representation,
- heuristic population initialization,
- discrete crossover-style position updates,
- a disturbance operator based on earliest machine completion,
- personal-best archives,
- a global-best archive,
- a non-dominated archive update strategy,
- and variable neighborhood search.

The method is tested on standard Kacem and Brandimarte benchmark instances. The results indicate that the proposed MOPSO generally produces a strong and diverse collection of non-dominated schedules and compares favorably with several previously published algorithms.

Flexible Job-Shop Scheduling Problem

The Flexible Job-Shop Scheduling Problem, or FJSP, contains:

- a set of jobs,
- a set of machines,
- and several ordered operations within each job.

Each operation must be assigned to one eligible machine.

Unlike the ordinary Job-Shop Scheduling Problem, an operation does not necessarily have only one possible machine. It may be processed by any machine from a specified candidate set.

The scheduling procedure must therefore determine:

- machine assignments,
- operation sequences,
- and operation starting times.

Partial and Total Flexibility

The paper distinguishes between two forms of flexibility.

Partial Flexibility

An operation can be processed only by a subset of the available machines.

Total Flexibility

An operation can be processed by every machine in the system.

The processing time may differ depending on which machine is selected.

Problem Assumptions

The model uses the following assumptions:

- Jobs and machines are independent.
- Machine setup times are ignored.
- Transportation times between operations are ignored.

- Each machine can process only one operation at a time.
- Only one operation of the same job can be processed at a time.
- Operations belonging to different jobs do not have precedence constraints between them.
- The operations within one job must follow their required sequence.
- Processing cannot be interrupted after it begins.

Optimization Objectives

The paper uses three scheduling objectives.

Makespan

Makespan is the completion time of the final operation.

In copy-safe form:

CM = maximum completion time among all machines

A lower makespan means the entire set of jobs is completed sooner.

Total Workload

Total workload is the sum of the processing times assigned to all machines.

In copy-safe form:

WT = sum of all operation processing times on their assigned machines

This objective encourages the algorithm to select machines that process operations efficiently.

Critical Machine Workload

Critical machine workload is the workload of the most heavily loaded machine.

In copy-safe form:

WM = maximum workload among all machines

This objective encourages a more balanced distribution of work.

Multi-Objective Nature of the Problem

The three objectives may conflict.

For example:

- A schedule with the lowest makespan may place excessive work on one machine.
- A schedule with balanced machine workloads may require a longer completion time.
- Selecting the fastest machine for every operation may overload that machine.

Because of these conflicts, the paper does not combine the objectives into one fixed final score.

Instead, it searches for a set of **non-dominated solutions**.

Non-Dominated Solutions

A schedule dominates another schedule when it is:

- no worse in every objective,
- and better in at least one objective.

A non-dominated schedule is one for which no other known schedule is better across all three objectives.

The collection of non-dominated solutions represents an approximate **Pareto front**.

This allows decision-makers to select among different trade-offs rather than being given only one schedule.

Schedule Representation

The paper represents every schedule using two vectors:

1. Operation sequence vector
2. Machine assignment vector

Operation Sequence Vector

The operation sequence vector determines the order in which operations are considered.

Each number represents a job.

The first occurrence of a job number represents that job's first operation, the second occurrence represents its second operation, and so on.

For example:

2, 1, 1, 3, 2, 1, 2, 3

This represents:

021, 011, 012, 031, 022, 013, 023, 032

Where:

- 021 means the first operation of Job 2.
- 012 means the second operation of Job 1.

This encoding automatically preserves the required operation order within each job.

Machine Assignment Vector

The second vector specifies which machine processes each operation.

For example:

1, 3, 2, 1, 3, 1, 3, 2

Each position corresponds to an operation arranged according to job and operation number.

Together, the two vectors define:

- operation order,
- machine assignment,
- and the resulting schedule.

Schedule Decoding

A raw encoded sequence may produce a **semi-active schedule**, meaning that some operations could be shifted earlier without violating constraints.

To improve the schedule, the authors use a left-shift procedure.

The procedure moves operations earlier whenever:

- their preceding job operation is finished,

- the assigned machine is available,
- and no other scheduling constraint is violated.

This converts the decoded schedule into an **active schedule**, where no operation can be shifted earlier without delaying another operation.

Active schedules generally have better makespan values than semi-active schedules.

Population Initialization

The initial population is created using a combination of heuristic and random rules.

For machine assignments:

- 20% of individuals use a Global Processing Time rule.
- 20% use a Local Processing Time rule.
- 60% use random assignment.

The operation-sequence vectors are generated using a random dispatching rule.

This mixture is intended to provide both:

- high-quality starting schedules,
- and sufficient population diversity.

Purely random initialization may provide diversity but weak initial solutions.

Purely heuristic initialization may provide strong solutions but insufficient diversity.

Standard Particle Swarm Optimization

In ordinary PSO, every particle has:

- a current position,
- a velocity,
- a personal-best position,
- and access to a global-best position.

The basic continuous update may be written as:

`New velocity = inertia contribution + cognitive contribution + social contribution`

`New position = current position + new velocity`

The cognitive component moves a particle toward its personal best.

The social component moves it toward the swarm's global best.

However, ordinary arithmetic cannot be applied directly to operation sequences and machine assignments.

The authors therefore replace standard velocity and position arithmetic with discrete operators.

Modified Discrete Position Update

The modified update uses three possible behaviors.

Preserve the Current Position

With a selected probability, the particle remains unchanged.

This represents the inertia component.

Learn from a Personal or Global Best

The current particle is combined with:

- one solution from its personal-best archive,
- or one solution from the global-best archive.

This represents cognitive and social learning.

Apply a Disturbance Operator

When the current position is not preserved, a disturbance operator may alter the schedule.

This increases exploration and helps prevent premature convergence.

The discrete update can be understood as:

`New particle = preserve current schedule, learn from an archived schedule, or apply disturbance`

The choice is controlled probabilistically.

Operator f1

Operator **f1** preserves the current particle.

With probability **omega**, the particle remains unchanged before the learning operator is applied.

The selected value is:

omega = 0.98

This creates strong retention of existing schedule information.

Operator f2

Operator **f2** allows a particle to learn from:

- its personal-best archive,
- or the global-best archive.

It operates separately on the two schedule vectors.

For the operation sequence, it uses:

- Improved Precedence Operation Crossover, or IPOX.

For the machine assignment vector, it uses:

- Multipoint Preservative Crossover, or MPX.

Improved Precedence Operation Crossover

IPOX combines the operation sequences of two parent schedules.

The procedure is:

1. Randomly divide the jobs into two groups.
2. Copy operations belonging to the first group from Parent 1 into the same positions in Child 1.
3. Copy operations belonging to the first group from Parent 2 into the same positions in Child 2.

4. Fill the remaining spaces in Child 1 using the order of operations from Parent 2.
5. Fill the remaining spaces in Child 2 using the order from Parent 1.

This preserves valid job-operation sequences while combining information from both parents.

Multipoint Preservative Crossover

MPX combines two machine-assignment vectors.

The procedure is:

1. Generate a binary decision vector containing zeros and ones.
2. At positions marked 1, copy machine assignments between the two parents according to the crossover rule.
3. At the remaining positions, retain the alternative parent assignments.

This creates new valid combinations of machine assignments.

Operator f3

Operator **f3** is a disturbance or perturbation operator intended to prevent premature convergence.

It uses an **Earliest Completion Machine rule**.

For a selected operation:

1. Estimate its completion time on every eligible machine.
2. Identify the machine that would complete it earliest.
3. Reassign the operation to that machine.

This can improve machine assignment quality.

However, calculating earliest completion times for every option is computationally expensive.

The operator is therefore used only with a small probability:

c3 = 0.02

Personal-Best Archives

Unlike ordinary PSO, each particle does not keep only one personal-best solution.

Each particle maintains a personal-best archive containing several non-dominated solutions discovered during its search history.

The maximum personal archive size is:

$$N_p = 5$$

This allows one particle to remember multiple useful trade-offs among:

- makespan,
- total workload,
- and critical-machine workload.

Global-Best Archive

The algorithm also maintains a global-best archive containing non-dominated solutions found by the entire population.

The maximum global archive size is:

$$N_a = 15$$

This archive serves as the swarm's social memory.

Instead of every particle following one global-best schedule, particles can learn from different non-dominated schedules.

This helps preserve diversity and reduces the risk that the whole swarm converges prematurely to one trade-off.

Archive Update Process

For a personal-best archive:

1. Combine the existing archive with the particle's new solution.
2. Remove dominated solutions.
3. Apply the archive-update strategy when too many solutions remain.
4. Retain up to five solutions.

For the global-best archive:

1. Combine the current archive with non-dominated solutions from the population.
2. Remove dominated solutions.
3. Reduce the archive to no more than 15 solutions.

Weighted Archive Update Strategy

When too many non-dominated solutions are available, the paper uses a weighted-sum strategy.

Three random weights are generated, one for each objective.

These weights are combined with fixed scale coefficients so that the objectives have more comparable numerical effects.

The archive members are then ranked using the weighted result.

Because the weights are randomly generated during updates, different objective preferences are emphasized at different times.

This helps preserve solutions from different areas of the Pareto front.

Selecting Personal and Global Guides

After the archives are updated:

- the personal-best guide is randomly selected from the particle's own archive,
- and the global-best guide is randomly selected from the global archive.

Random guide selection reduces the likelihood that every particle follows the same schedule.

Variable Neighborhood Search

The proposed MOPSO includes a variable neighborhood search procedure.

It is applied to schedules stored in the global-best archive.

The purpose is to improve local exploitation after PSO has identified promising regions.

The search uses information from the schedule's critical path.

Critical Path and Critical Blocks

The critical path is the longest path through the schedule.

It determines the makespan.

Operations on this path are called critical operations.

A **public critical block** is a consecutive sequence of critical operations:

- processed on the same machine,
- and shared among all critical paths.

The authors focus neighborhood modifications on these critical blocks because changing noncritical operations may not reduce makespan.

Neighborhood NH1

NH1 performs a machine reassignment and insertion.

The procedure is:

1. Identify machines associated with the maximum makespan.
2. Select one such machine.
3. Select a critical operation on that machine.
4. Choose another eligible machine.
5. Insert the operation into a valid position on the new machine.

The insertion must preserve the operation order of the job.

Neighborhood NH2

NH2 also changes a critical operation's machine.

The procedure is:

1. Select a critical operation with more than one eligible machine.
2. Rank its eligible machines by processing time.
3. Select a machine from the faster half of the candidate list.
4. Reassign the operation to that machine.

This directs the search toward promising machine assignments.

Neighborhood NH3

NH3 changes the operation sequence within a public critical block.

When the block contains three operations:

- the middle operation may be swapped with the first or last operation, subject to job constraints.

When the block contains more than three operations:

- the first or last operation may be inserted into another valid position inside the block.

NH3 changes sequencing while NH1 and NH2 primarily modify machine assignments.

Variable Neighborhood Procedure

For each solution in the global archive:

1. Apply the first neighborhood.
2. Conduct a local search containing up to 20 neighborhood attempts.
3. Replace the solution if a dominating schedule is found.
4. Restart with the first neighborhood after an improvement.
5. Move to the next neighborhood when no improvement occurs.
6. Stop after all three neighborhoods fail to improve the solution.

The local neighborhood search size is:

NS = 20

Complete MOPSO Process

The complete algorithm can be summarized as follows:

1. Generate the initial population using heuristic and random rules.
2. Decode all particles into active schedules.
3. Evaluate makespan, total workload, and critical-machine workload.
4. Initialize personal-best and global-best archives.
5. Generate new particles using the discrete position-update operators.
6. Occasionally apply the earliest-completion-machine disturbance.
7. Decode and evaluate the new particles.
8. Update personal-best archives.

9. Update the global-best archive.
10. Apply variable neighborhood search to the global archive.
11. Select new personal and global guides.
12. Repeat until the generation limit is reached.

Stopping Criterion

The algorithm stops after a predefined number of generations.

For the principal experiments:

Maximum generations = 300

Experimental Configuration

The algorithm is implemented using MATLAB 7.1.

The reported computer has:

- 4 GB RAM
- Intel Core i3-3240 processor at 3.4 GHz

Each benchmark instance is run ten times.

The selected parameters are:

Parameter	Value
Population size	100
Maximum generations	300
Inertia probability, omega	0.98

Personal-best learning probability, c1	0.6
Global-best learning probability, c2	0.4
Disturbance probability, c3	0.02
Personal-best archive size	5
Global-best archive size	15
Local neighborhood search size	20

Benchmark Instances

The experiments use two benchmark groups:

1. Four Kacem instances
2. Ten Brandimarte instances

Kacem Instances

The four Kacem instances range from:

- 4 jobs and 5 machines
- to 15 jobs and 10 machines

The proposed MOPSO is compared with:

- Hybrid Tabu Search Algorithm
- Artificial Immune Algorithm
- Xing's algorithm
- Multi-Objective Genetic Algorithm

- Pareto Discrete Artificial Bee Colony
- Simple Evolutionary Algorithm

Kacem Results

For the 4-job, 8-job, and 10-job cases, all compared algorithms obtain at least some non-dominated solutions.

For the 15-job instance, the schedules generated by:

- Hybrid Tabu Search,
- Xing's method,
- and MOPSO

include the same strong objective combinations.

Some solutions from the Artificial Immune Algorithm, Multi-Objective Genetic Algorithm, and Artificial Bee Colony approach are dominated by MOPSO schedules.

Number of Non-Dominated Kacem Solutions

The reported numbers of non-dominated solutions are:

Instance	MOPSO solutions
4 jobs × 5 machines	3
8 jobs × 8 machines	4
10 jobs × 10 machines	4

15 jobs × 10 2
machines

MOPSO produces more non-dominated solutions than several competing methods across most instances.

The Simple Evolutionary Algorithm obtains one more solution than MOPSO on the smallest instance. However, the paper states that sufficient details were not provided to evaluate all of those solutions fully.

Example Kacem Objective Values

Examples of schedules found by MOPSO include:

CM = 11, WT = 32, WM = 10

and:

CM = 12, WT = 32, WM = 8

These two schedules illustrate a trade-off:

- the first has lower makespan,
- while the second has lower critical-machine workload.

Neither necessarily dominates the other.

Gantt Charts

The paper presents Gantt charts for the four Kacem instances.

These charts show:

- which machine processes each operation,
- the start and completion times,
- idle periods,
- and the overall makespan.

They illustrate that MOPSO produces feasible schedules while distributing operations among eligible machines.

Brandimarte Instances

The second benchmark group contains ten Brandimarte instances, identified as:

- MK01 through MK10

Their sizes range from:

- 10 jobs and 6 machines
- to 20 jobs and 15 machines

The proposed method is compared with:

- Xing's algorithm
- Multi-Objective Genetic Algorithm
- Hybrid Tabu Search Algorithm
- Hybrid Shuffled Frog-Leaping Algorithm
- Artificial Immune Algorithm
- Simple Evolutionary Algorithm
- and a Pareto-based discrete Artificial Bee Colony method

Brandimarte Non-Dominated Results

MOPSO dominates several reported solutions from the competing algorithms.

Examples include:

- Xing's solutions on MK02, MK05, MK06, and MK07
- MOGA solutions on MK01 and MK03
- HTSA results on MK07
- HSFLA results on MK02, MK05, and MK07
- AIA results on most of the ten instances

The main exception is MK08, where the single MOPSO solution is dominated by a MOGA solution.

Overall, the paper concludes that MOPSO produces a larger number of high-quality solutions across the Brandimarte benchmarks.

Best Makespan Results

Compared with the Simple Evolutionary Algorithm, MOPSO improves the best makespan on:

- MK06
- MK09
- MK10

It is not worse on the remaining instances.

Compared with the Pareto-based Artificial Bee Colony method, MOPSO improves the best makespan on:

- MK04
- MK06
- MK07

MOPSO is worse on MK05.

Selected Numerical Results

Examples of MOPSO best makespans include:

Instance	Best MOPSO makespan
MK01	40
MK02	26
MK03	204
MK04	61
MK05	173
MK06	62

MK07 139

MK08 523

MK09 310

MK10 214

Largest Reported Improvements

The strongest reported makespan improvement relative to the Simple Evolutionary Algorithm occurs on MK10:

Improvement = 16.08%

For MK06:

Improvement = 4.62%

For MK09:

Improvement = 0.32%

Compared with the Pareto Artificial Bee Colony method, the largest improvement is:

4.62% on MK06

Stability Across Repeated Runs

The authors report:

- average best makespan,
- standard deviation,
- and average computation time.

For most Brandimarte instances, the standard deviation of makespan is below 1.

This indicates that the method usually produces similar makespan values across independent runs.

MK09 has a higher standard deviation of approximately:

1.53

The paper interprets the generally low variation as evidence of stable search behavior.

Runtime Results

MOPSO often requires more computation time than the Pareto Artificial Bee Colony comparison method.

Examples of average MOPSO times include:

Instance	Average MOPSO time
MK01	10.22 seconds
MK04	58.22 seconds
MK06	75.22 seconds
MK09	104.42 seconds
MK10	156.73 seconds

The authors caution that direct runtime comparison is limited because the algorithms were implemented using:

- different programming languages,
- different computers,

- and different software environments.

The comparison algorithm was implemented in C++ on older hardware, while MOPSO was implemented in MATLAB.

Why the Authors Believe MOPSO Works

The paper attributes the method's performance to its multiple memory structures.

Cognitive Memory

Each particle stores several personal non-dominated solutions.

This allows the particle to remember different successful trade-offs.

Social Memory

The global archive stores non-dominated solutions discovered by the whole population.

This gives the swarm access to multiple global guides.

Diversity Preservation

Random selection from the archives prevents every particle from following exactly the same solution.

Local Improvement

Variable neighborhood search improves promising schedules in the global archive.

Exploration

The disturbance operator introduces alternative machine assignments and helps prevent stagnation.

The authors argue that the combination balances:

- global exploration,
- local exploitation,
- and diversity.

Approximate Pareto Fronts

The paper plots approximate Pareto fronts for MK03 and MK04.

The plots show the trade-offs among:

- makespan,
- total workload,
- and critical-machine workload.

The solutions form a spread rather than one single optimum.

This demonstrates that improving one objective may require sacrificing another.

Parameter Sensitivity Analysis

The authors conduct a Taguchi experimental design using the MK02 instance.

Four parameters are tested:

- population size, **N**
- inertia probability, **omega**
- cognitive and social probabilities, (**c1**, **c2**)
- disturbance probability, **c3**

Each parameter has four tested levels.

Every designed experiment is run five times.

The generation limit for this analysis is:

100 generations

Parameter Ranges

The tested values are:

Population Size

- 50
- 100

- 150
- 200

Inertia Probability

- 0.90
- 0.94
- 0.96
- 0.98

Cognitive and Social Probabilities

- (0.2, 0.8)
- (0.4, 0.6)
- (0.6, 0.4)
- (0.8, 0.2)

Disturbance Probability

- 0.001
- 0.01
- 0.05
- 0.10

The paper's table appears to contain a formatting duplication for the lowest **c3** value, but the intended four levels are represented by the listed range.

Parameter Importance

The importance of each parameter is estimated using the difference between its highest and lowest average makespan across tested levels.

The reported differences are:

Parameter	Difference
-----------	------------

Cognitive/social values, c_1 and c_2 0.60

Disturbance probability, c_3 0.50

Population size, N 0.45

Inertia probability, ω 0.40

The most influential setting is therefore the balance between:

- personal-best learning,
- and global-best learning.

The disturbance probability is the second most influential.

Main Findings

The paper's principal findings are:

- Flexible job-shop scheduling requires simultaneous machine assignment and operation sequencing.
- The problem becomes more realistic but also more difficult when operations have multiple eligible machines.
- Multi-objective scheduling cannot be judged by makespan alone.
- A two-vector encoding can represent machine assignment and operation sequencing separately.
- Standard continuous PSO requires specialized discrete operators for scheduling.
- Personal-best and global-best archives preserve multiple non-dominated solutions.
- Cognitive and social memories help maintain diversity.
- Random archive-guide selection reduces premature convergence.
- Earliest-completion-machine disturbance strengthens exploration.
- Variable neighborhood search improves local exploitation.
- Critical-path-based neighborhoods reduce unnecessary search.
- MOPSO produces high-quality non-dominated solutions on Kacem benchmarks.
- It dominates many solutions reported by competing algorithms on Brandimarte instances.

- Its repeated-run makespan results are generally stable.
- It can require greater computation time than some comparison methods.
- The cognitive and social learning probabilities have the greatest measured effect on performance.

Important Limitations

Comparisons Use Previously Published Results

Most competitor results are taken from earlier papers rather than produced through new implementations on the same machine.

Differences may therefore exist in:

- programming language,
- processor,
- stopping conditions,
- parameter tuning,
- and implementation quality.

Limited Number of Runs

The main experiments use ten runs per instance.

This provides some evidence of stability but remains relatively limited for a stochastic metaheuristic.

Empirical Parameter Selection

Most algorithm parameters are selected experimentally.

The best settings may differ for other benchmark families or real manufacturing systems.

Complex Hybrid Design

The proposed method is not a simple PSO.

It includes:

- heuristic initialization,
- two discrete crossover operators,

- a disturbance mechanism,
- personal archives,
- a global archive,
- weighted archive selection,
- and three local neighborhoods.

Its performance cannot be attributed to PSO alone.

Static Scheduling Environment

The model assumes that:

- processing times are known,
- machines do not break down,
- no urgent jobs arrive,
- and the scheduling environment does not change during execution.

Real manufacturing systems may be dynamic.

Neglected Setup and Movement Times

Machine setup times and operation-transfer times are treated as negligible.

These assumptions may limit practical applicability in systems where setup or transportation is substantial.

Three Fixed Objectives

The paper considers only:

- makespan,
- total workload,
- and critical-machine workload.

It does not include objectives such as:

- tardiness,
- energy consumption,
- production cost,
- machine reliability,
- or workforce availability.

Future Work

The authors recommend further work in three areas:

1. Developing stronger theory for multi-objective flexible job-shop scheduling.
2. Extending the model to dynamic and changing manufacturing environments.
3. Developing more efficient optimization methods for difficult FJSP instances.

Conclusion

The paper concludes that the proposed modified MOPSO is an effective method for multi-objective flexible job-shop scheduling.

Its central features are:

- discrete schedule representation,
- heuristic and random initialization,
- personal and social archives,
- non-dominated solution preservation,
- specialized crossover operators,
- disturbance-based exploration,
- and variable neighborhood local search.

The algorithm produces a diverse set of trade-off schedules rather than one single answer.

Across the Kacem and Brandimarte benchmarks, it generally obtains:

- more non-dominated solutions,
- competitive or improved makespans,
- and stable results across repeated runs.

The method's main disadvantage is increased computational complexity and runtime caused by its archive management and local-search components.

Application of Tabu Search for Job Shop Scheduling Based on Manufacturing Order Swapping

**Jung-Sing Jwo, Cheng-Hsiung Lee, Jian-Tan Chen,
Ching-Sheng Lin, Chun-Yu Lin, Wen-Kai Cheng,
Chun-Hao Chang, and Jen-Kai King**

Summary

This paper applies **Tabu Search** to a real job-shop scheduling problem in the aircraft manufacturing industry.

The study focuses on a work center where workers use pliers to trim composite-material parts. The production environment is characterized by:

- small production volumes,
- many different product varieties,
- changing daily manufacturing orders,
- limited working capacity,
- existing production delays,
- and strict due dates.

The scheduling system must decide the order in which manufacturing orders should be processed each day.

The authors propose a Tabu Search method that improves an initial schedule by repeatedly swapping manufacturing orders. The method considers:

- due dates,
- processing times,
- daily working hours,
- remaining lead time after the current work center,
- and current delays.

The proposed schedules are evaluated using three main measures:

1. Total expected early completion days
2. Total expected delay days
3. Total number of delayed manufacturing orders

The results show that the method substantially improves delay-related performance compared with directly using First In, First Out or Earliest Due Date scheduling. When initialized using First In, First Out, the proposed method reaches the theoretical lower bound for the number of delayed manufacturing orders on all five tested days.

Research Context

The paper is motivated by the development of smart manufacturing and the Internet of Things.

Aircraft production often involves composite materials and highly specialized manufacturing processes. Unlike mass production, the aircraft industry commonly has:

- low production quantities,
- many product variations,
- complex process requirements,
- and frequent changes in manufacturing orders.

The authors describe this as a **small-volume, large-variety** production environment.

In such an environment, inefficient scheduling can cause:

- missed due dates,
- long delays,
- excessive early completion,
- unfinished orders being carried into later days,
- and poor use of work-center capacity.

The existing scheduling system discussed in the paper adapts slowly when requirements or production conditions change.

Job-Shop Scheduling

Job-shop scheduling assigns operations or manufacturing orders to limited production resources over time.

The problem is treated as a combinatorial optimization problem and is classified as NP-hard.

This means that the number of possible schedules grows rapidly as the number of manufacturing orders increases.

The authors mention several previous scheduling approaches, including:

- queuing theory,

- simulation,
- Genetic Algorithms,
- Tabu Search,
- reinforcement learning,
- actor–critic models,
- Deep Q-Networks,
- and dueling double Deep Q-Networks.

The paper chooses Tabu Search because it can explore alternative schedules while using memory to prevent cycling.

Tabu Search Concepts

The paper highlights two main features of Tabu Search.

Local Search

The algorithm moves from one schedule to neighboring schedules.

It may accept a move that does not immediately improve the objectives when no better move is available. This allows the algorithm to escape a local optimum.

Tabu List

The tabu list stores recently visited schedules.

A candidate schedule already contained in the tabu list is skipped.

This prevents the search from repeatedly moving among the same schedules and becoming trapped in a cycle.

Industrial Case

The empirical case is based on one aircraft-industry work center.

The work center uses pliers to trim parts.

Every day, the work center receives new manufacturing orders.

The schedule for a particular day includes:

- new orders received that day,
- and unfinished orders carried over from previous days.

The scheduling method determines the order in which these manufacturing orders should be processed.

Orders that cannot be completed during the available working hours are moved into the following day's scheduling process.

Initial Scheduling Methods

The paper evaluates two methods for creating the initial order sequence.

First In, First Out

First In, First Out, or FIFO, processes manufacturing orders according to their arrival order.

Older orders are generally processed before newer orders.

Earliest Due Date

Earliest Due Date, or EDD, arranges orders according to their due dates.

Orders with the nearest due dates are processed first.

Tabu Search then improves the initial FIFO or EDD sequence through order swapping.

Important Variables

The paper defines the following scheduling information.

- WO = working hours available per day at the current work center
- i = manufacturing-order index
- $PT(i)$ = processing time of manufacturing order i , measured in hours
- $LT(i)$ = average lead time required after the current work center, measured in days
- $SD(i)$ = scheduled processing day of manufacturing order i
- $DD(i)$ = due date of manufacturing order i
- $BPDD(i)$ = number of days before or after the due date when the order reaches the current work center
- $EDD_FCA(i)$ = expected delay under the finite-capacity assumption

Current Due-Date Position

The value $BPDD(i)$ is calculated as:

$$BPDD(i) = \text{Today} - DD(i)$$

Interpretation:

- A positive value means the due date has already passed.
- A negative value means the order is still ahead of its due date.
- Zero means the due date is today.

Expected Delay Under Finite Capacity

The expected delay for an order is calculated as:

$$EDD_FCA(i) = BPDD(i) + SD(i) + LT(i)$$

This value combines:

- the order's current position relative to its due date,
- the day on which it is scheduled,
- and the remaining processing lead time after the current work center.

Interpretation:

- $EDD_FCA(i) > 0$ means the order is expected to be delayed.
- $EDD_FCA(i) < 0$ means the order is expected to finish before its due date.
- $EDD_FCA(i) = 0$ means it is expected to finish exactly at the due date.

Scheduling Objectives

The study considers three main minimization objectives.

Total Expected Early Completion Days

This is abbreviated as TEECD.

It is the total of the negative expected-delay values for orders expected to finish early.

In copy-safe form:

$TEECD = \text{sum of } EDD_FCA(i) \text{ for orders where } EDD_FCA(i) < 0$

The values are negative because they represent days before the due date.

The paper treats excessive early completion as undesirable because composite-material manufacturing may have timing or storage requirements.

A value closer to the designated upper bound is preferred.

Total Expected Delay Days

This is abbreviated as TEDD.

It is calculated as:

$TEDD = \text{sum of } EDD_FCA(i) \text{ for orders where } EDD_FCA(i) \geq 0$

A lower TEDD means that the combined expected delay across late orders is smaller.

Total Number of Delayed Manufacturing Orders

This is abbreviated as TNDMO.

It is calculated as:

$TNDMO = \text{number of orders where } EDD_FCA(i) \geq 0$

A lower TNDMO means fewer manufacturing orders are expected to miss their due dates.

Total Number of Early Completion Manufacturing Orders

This is abbreviated as TNECMO.

It is calculated as:

$TNECMO = \text{number of orders where } EDD_FCA(i) < 0$

This value is used when constructing the neighborhood of possible swaps.

Daily Scheduling Procedure

For each day:

1. New manufacturing orders are received.
2. Unfinished orders from the previous day are added.
3. An initial schedule is generated using FIFO or EDD.
4. Tabu Search attempts to improve the order sequence.
5. Scheduled days are calculated using cumulative processing times and daily working hours.
6. TEECD, TEDD, and TNDMO are calculated.
7. Orders that cannot be completed are moved to the next day.

This process is repeated over five scheduling days in the experiment.

Manufacturing-Order Swapping

The proposed neighborhood operation swaps manufacturing orders.

The algorithm identifies orders currently expected to be delayed.

For every delayed order, it attempts to exchange its position with other orders in the schedule.

After each swap, the algorithm recalculates:

- the scheduled processing days,
- expected delays,
- total early completion,
- total delay,
- and number of delayed orders.

The number of possible swaps is written as:

$$\text{Number of swaps} = \text{TNDMO} \times (\text{TNDMO} + \text{TNECMO} - 1)$$

Because:

$$\text{TNDMO} + \text{TNECMO} = \text{total number of manufacturing orders}$$

this effectively allows each delayed order to be compared with nearly every other order.

Tabu Search Algorithm

The initial schedule is denoted as S_{init} .

The main variables are:

- `sBest` = best schedule found during the complete search
- `iterBest` = best schedule found during the current iteration
- `tabuList` = stored schedules that should not be revisited
- `sNeighborhood` = set of candidate schedules created through swapping

The procedure is:

1. Set the initial schedule as the best-known schedule.
2. Add the initial schedule to the tabu list.
3. Identify the manufacturing orders currently expected to be delayed.
4. Swap each delayed order with other orders.
5. Store the generated schedules in the neighborhood.
6. Ignore candidate schedules already contained in the tabu list.
7. Compare candidates using TEECD, TEDD, and TNDMO.
8. Select the best candidate of the iteration.
9. Compare it with the overall best schedule.
10. Add the selected schedule to the tabu list.
11. Repeat for the predefined number of iterations.

Schedule Comparison

The paper uses a function called `checkMinOptimization` to compare schedules.

A candidate is preferred when it provides lower values for the optimization measures.

The paper does not present one explicit weighted objective formula combining all three criteria.

Instead, the algorithm compares:

- TEECD,
- TEDD,
- and TNDMO

within the scheduling procedure.

The number of delayed manufacturing orders is described as the most critical practical factor.

Finite-Capacity Assumption

The actual Tabu Search schedule uses a finite-capacity assumption.

This means that the work center has only a limited number of working hours per day.

When the total required processing time exceeds daily capacity, some orders must be moved to later days.

This reflects the real production environment.

Infinite-Capacity Bounds

The authors also create theoretical extreme bounds using an infinite-capacity assumption.

Under this assumption:

- every new manufacturing order could be completed on the same day,
- and no order would need to be carried into the next day because of capacity limits.

These bounds are not realistic schedules. They represent ideal reference values.

The three bounds are:

- **UBTEECD** = upper bound for total expected early completion days
- **LBTEDD** = lower bound for total expected delay days
- **LBTNMO** = lower bound for the total number of delayed manufacturing orders

The proposed finite-capacity schedule is compared with these ideal values.

Experimental Measures

The daily data include:

- **NMO** = number of new manufacturing orders received that day
- **TMO** = total manufacturing orders available that day
- **FMO** = number of manufacturing orders completed that day
- **FDMO** = completed manufacturing orders that are already delayed
- **TNDMO** = total number of delayed manufacturing orders

The total manufacturing orders consist of:

TMO = new orders + unfinished orders from earlier days

Five-Day Workload

The number of new manufacturing orders for the five days is:

Day	New manufacturing orders
-----	--------------------------

Day 1	369
-------	-----

Day 2	66
-------	----

Day 3	94
-------	----

Day 4	123
-------	-----

Day 5	121
-------	-----

Because many orders are not completed immediately, the total number of available orders increases over time.

FIFO Initialization Results

With FIFO initialization, the total manufacturing orders are:

Day	Total manufacturing orders
-----	----------------------------

Day 1	369
-------	-----

Day 2	393
-------	-----

Day 3 402

Day 4 508

Day 5 594

The number of completed orders is:

Day	Completed orders
------------	-----------------------------

Day 1	42
-------	----

Day 2	85
-------	----

Day 3	17
-------	----

Day 4	35
-------	----

Day 5	56
-------	----

The number of delayed manufacturing orders produced by Tabu Search is:

Day	TNDMO
------------	--------------

Day 1	63
-------	----

Day 2 61

Day 3 21

Day 4 40

Day 5 50

The theoretical lower bounds are exactly the same:

Day	Lower-bound TNDMO
-----	-------------------

Day 1	63
-------	----

Day 2	61
-------	----

Day 3	21
-------	----

Day 4	40
-------	----

Day 5	50
-------	----

Therefore, the Tabu Search schedule initialized with FIFO reaches the lower bound for the number of delayed orders on all five days.

EDD Initialization Results

With EDD initialization, the total manufacturing orders are:

Day	Total manufacturing orders
------------	-----------------------------------

Day 1	369
-------	-----

Day 2	397
-------	-----

Day 3	451
-------	-----

Day 4	557
-------	-----

Day 5	655
-------	-----

The number of delayed manufacturing orders is:

Day	TNDMO
------------	--------------

Day 1	71
-------	----

Day 2	81
-------	----

Day 3	53
-------	----

Day 4	65
-------	----

Day 5 84

The corresponding lower bounds are:

Day	Lower-bound TNDMO
-----	-------------------

Day 1	63
-------	----

Day 2	68
-------	----

Day 3	45
-------	----

Day 4	60
-------	----

Day 5	84
-------	----

The EDD-initialized version reaches the lower bound only on Day 5.

FIFO is therefore a much stronger initial scheduling method for this particular case.

Direct FIFO Versus Tabu Search with FIFO

The paper compares ordinary FIFO scheduling with the Tabu Search schedule initialized by FIFO.

Day 1

Ordinary FIFO:

- TEECD = 4,524
- TEDD = 1,230

- TNDMO = 86

Tabu Search:

- TEECD = 4,312
- TEDD = 1,079
- TNDMO = 63

Bound:

- TEECD = 5,113
- TEDD = 1,043
- TNDMO = 63

The Tabu Search method reduces delayed orders from 86 to 63.

Day 2

Ordinary FIFO:

- TEDD = 1,029
- TNDMO = 67

Tabu Search:

- TEDD = 898
- TNDMO = 61

Bound:

- TEDD = 882
- TNDMO = 61

Day 3

Ordinary FIFO:

- TEDD = 572
- TNDMO = 30

Tabu Search:

- TEDD = 475
- TNDMO = 21

Bound:

- TEDD = 463
- TNDMO = 21

Day 4

Ordinary FIFO:

- TEDD = 883
- TNDMO = 43

Tabu Search:

- TEDD = 758
- TNDMO = 40

Bound:

- TEDD = 752
- TNDMO = 40

Day 5

Ordinary FIFO:

- TEDD = 876
- TNDMO = 54

Tabu Search:

- TEDD = 655
- TNDMO = 50

Bound:

- TEDD = 653
- TNDMO = 50

Across all five days, Tabu Search initialized with FIFO reaches the lower bound for TNDMO and produces TEDD values very close to the theoretical minimum.

Delay Percentage Relative to the Bound

The reported TEDD differences from the lower bound for the FIFO-initialized Tabu Search are:

Day	Difference from TEDD lower bound
-----	----------------------------------

Day 1	3%
-------	----

Day 2	2%
-------	----

Day 3	3%
-------	----

Day 4	1%
-------	----

Day 5	0%
-------	----

This means that the proposed method produces total expected delays within 0% to 3% of the theoretical minimum under the ideal infinite-capacity reference.

Direct EDD Versus Tabu Search with EDD

The proposed method also improves schedules initialized using EDD.

For example, on Day 1:

Ordinary EDD:

- TEDD = 1,154
- TNDMO = 95

Tabu Search with EDD initialization:

- TEDD = 1,131
- TNDMO = 71

Bound:

- TEDD = 1,043
- TNDMO = 63

The improvement is meaningful, but the final result remains worse than the FIFO-initialized Tabu Search.

Across the five days, the Tabu Search schedule with EDD initialization remains approximately:

- 5% to 19% above the lower bound in TNDMO on Days 1 to 4,
- and reaches the bound on Day 5.

Comparison of Initializations

The paper consistently finds that FIFO initialization produces better final results than EDD initialization.

FIFO initialization achieves:

- lower total expected delays,
- fewer delayed manufacturing orders,
- and closer results to the theoretical bounds.

The authors describe the EDD initialization as not promising for this case.

This does not mean that EDD is universally weak. It means that FIFO produces the better starting structure for the particular aircraft work-center data used in the study.

Early Completion Results

The proposed method performs well in reducing delay, but its TEECD results are weaker.

The percentage differences from the early-completion bound remain approximately:

- 14% to 20% below the bound,
- depending on the day and initialization.

The authors conclude that the method still has room for improvement in managing early completion.

This is important because the scheduling problem does not simply reward finishing every order as early as possible.

For the composite-material process, excessive early completion may be undesirable.

Main Findings

The paper's principal findings are:

- Job-shop scheduling in aircraft manufacturing is difficult because of high product variety and limited capacity.
- Manufacturing orders can be improved through pairwise position swapping.
- Tabu Search can prevent repeated schedules from being revisited.
- The method considers due dates, daily capacity, current delay, processing time, and remaining lead time.
- FIFO and EDD can both be used to create the initial schedule.
- FIFO provides the stronger initialization in the tested work center.
- Direct FIFO and EDD scheduling both produce weaker results than the Tabu Search method.
- Tabu Search initialized with FIFO reaches the lower bound for delayed manufacturing orders on all five days.
- Its total expected delay is within approximately 0% to 3% of the ideal lower bound.
- EDD initialization improves after Tabu Search but remains weaker than FIFO initialization.
- The proposed method performs particularly well for delay-related objectives.
- Its handling of early completion remains less effective.
- The results indicate practical feasibility in the aircraft-industry case.

Important Limitations

Single Work Center

The experiment uses only one work center involving plier-based trimming of aircraft parts.

The results may not automatically generalize to:

- machining centers,
- assembly lines,
- painting,
- inspection,
- or other production stages.

Five-Day Evaluation

The case study covers only five scheduling days.

A longer evaluation would be needed to assess performance under:

- seasonal workload variation,
- equipment breakdown,
- employee absence,
- changing production priorities,
- and larger order backlogs.

No Direct Comparison with Other Metaheuristics

The paper does not experimentally compare its method with:

- Simulated Annealing,
- Particle Swarm Optimization,
- Genetic Algorithms,
- or reinforcement-learning methods.

The literature review mentions alternative approaches, but only FIFO, EDD, and the theoretical bounds are directly evaluated.

Unclear Multi-Objective Decision Rule

The scheduling method evaluates TEECD, TEDD, and TNDMO.

However, the paper does not clearly specify:

- whether the objectives are ranked,
- whether one objective has priority,
- or how conflicts between objectives are resolved numerically.

TNDMO appears to be treated as the most important practical criterion.

Complete Schedule Storage

The tabu list appears to store complete schedules rather than move attributes.

For large schedules, this may require more memory than storing only recent swaps.

The paper does not report:

- tabu-list length,
- memory usage,
- or how old schedules are removed.

Neighborhood Growth

Every delayed order may be swapped with nearly every other order.

As the number of manufacturing orders increases, the number of candidate schedules can become large.

The paper does not provide detailed runtime or computational-complexity results.

Infinite-Capacity Bounds

The bounds assume all new orders can be completed immediately.

This is intentionally idealized and may not represent a feasible factory schedule.

Reaching a bound under this comparison does not necessarily prove global optimality under every possible scheduling formulation.

Static Processing Data

The method assumes known processing times and lead times.

It does not explicitly address:

- uncertain processing time,
- machine breakdown,
- rework,
- emergency orders,
- material shortages,
- or worker availability.

Future Work

The authors identify two main future directions.

Improve Early-Completion Performance

The current method performs strongly for:

- total delay,
- and number of delayed orders.

However, it remains weaker for total early completion.

Future work should improve the trade-off between avoiding lateness and avoiding excessive early completion.

Test Additional Work Centers

The method should be applied to other work centers to evaluate:

- effectiveness,
- robustness,
- and generalizability.

Conclusion

The paper concludes that Tabu Search based on manufacturing-order swapping is a practically feasible method for aircraft job-shop scheduling.

The method begins with FIFO or EDD scheduling and improves the sequence by swapping delayed orders with other manufacturing orders.

In the five-day empirical study, the FIFO-initialized Tabu Search method:

- reaches the lower bound for the number of delayed manufacturing orders every day,
- produces total expected delay values close to the theoretical minimum,
- and clearly outperforms direct FIFO and EDD scheduling.

The main unresolved weakness is excessive early completion.

The authors therefore present the method as a strong delay-reduction approach that should be extended and refined for additional production environments.

Feature Selection Using Enhanced Particle Swarm Optimisation for Classification Models

Hailun Xie, Li Zhang, Chee Peng Lim, Yonghong Yu, and Han Liu

Summary

This paper proposes two enhanced Particle Swarm Optimization methods for feature selection in classification problems.

The two proposed models are called:

- **PSOVA1**
- **PSOVA2**

They are designed to address two major weaknesses of standard Particle Swarm Optimization:

1. Premature convergence
2. Weak local exploitation near promising or near-optimal solutions

The authors integrate the two PSO variants with a K-Nearest Neighbor classifier and use them as wrapper-based feature-selection methods.

The models search for subsets that:

- maximize classification accuracy,
- reduce the number of selected features,
- preserve search diversity,
- and avoid becoming trapped in local optima.

The algorithms are evaluated on 13 datasets containing between 30 and 22,283 features. They are compared with:

- five classical optimization algorithms,
- and ten advanced PSO variants.

The results show that PSOVA1 and PSOVA2 obtain the highest classification accuracy and F-score values across all 13 datasets. PSOVA2 generally performs better than PSOVA1, particularly on high-dimensional and complex classification tasks.

Feature Selection Problem

Many real-world datasets contain large numbers of attributes.

High-dimensional data can cause several problems:

- increased risk of overfitting,
- reduced generalization,
- higher computational cost,
- redundant information,
- irrelevant features,
- contradictory features,
- and complicated interactions among variables.

Feature selection aims to identify a smaller subset of useful features while removing irrelevant or redundant ones.

An exhaustive search is often impractical because a dataset containing n features has:

2^n possible feature subsets

For example, a dataset with thousands of features has an impossibly large number of possible combinations.

The authors therefore use population-based optimization methods to search the space of possible feature subsets.

Filter and Wrapper Feature Selection

The paper distinguishes between two broad feature-selection approaches.

Filter Methods

Filter methods rank features using statistical criteria such as:

- chi-square scores,
- mutual information,
- correlation,
- or other feature-importance measures.

These methods are generally computationally efficient.

However, they evaluate features individually and may fail to account for interactions among variables.

A feature that appears weak on its own may become useful when combined with another feature.

Wrapper Methods

Wrapper methods evaluate a feature subset by training or testing a classification algorithm using that subset.

The classifier is placed inside the feature-selection process.

Wrapper methods can therefore account for feature interactions and feature combinations.

However, they are more computationally expensive because many subsets must be evaluated.

The proposed PSOVA1 and PSOVA2 methods use a wrapper approach with K-Nearest Neighbor classification.

Standard Particle Swarm Optimization

PSO represents each candidate solution as a particle.

Each particle contains:

- a current position,
- a velocity,
- a personal-best position,
- and access to the global-best position found by the swarm.

The standard velocity update can be written in copy-safe form as:

`new velocity = inertia component + personal-best component +
global-best component`

More specifically:

`$v(t+1) = w \times v(t) + c1 \times r1 \times [pbest - x(t)] + c2 \times r2 \times [gbest - x(t)]$`

The position update is:

`$x(t+1) = x(t) + v(t+1)$`

Where:

- $x(t)$ is the current position,
- $v(t)$ is the current velocity,
- $pbest$ is the particle's historical best position,
- $gbest$ is the swarm's global best,
- w is the inertia weight,
- $c1$ and $c2$ are learning coefficients,
- and $r1$ and $r2$ are random values between 0 and 1.

Weaknesses of Standard PSO

The paper identifies several problems with the original PSO model.

Single Global Leader

All particles are strongly influenced by the same global-best solution.

When that leader represents a local rather than global optimum, the entire swarm may converge toward the wrong region.

Weak Particle Cooperation

Particles learn mainly from:

- their own personal best,
- and the single global best.

They do not exchange enough information with other promising particles.

Repeated Search

Particles may repeatedly revisit previously explored regions because they continue following the same historical best signals.

Loss of Diversity

Particle positions become increasingly similar over time.

Once diversity is lost, the swarm has difficulty exploring alternatives.

Weak Fine-Tuning

Standard PSO may converge quickly toward a promising region but lacks strong local-search mechanisms for precisely refining near-optimal solutions.

These weaknesses become more serious in feature-selection landscapes because interactions between features create many local optima.

Proposed PSOVA1 Model

PSOVA1 introduces four main improvements:

1. Enhanced global and personal best signals
2. Spiral-based local exploitation
3. Gaussian-based swarm-leader enhancement
4. Mutation-based improvement of weak particles

The authors intend these mechanisms to improve both exploration and exploitation.

PSOVA1 Initialization

The initial swarm is generated using a Logistic chaotic map rather than ordinary uniform random initialization.

Chaotic initialization is intended to distribute particles more broadly across the search space.

Each particle initially represents one possible feature subset.

Swarm-Leader Enhancement in PSOVA1

The global-best particle is locally modified using three Gaussian distributions:

- positively skewed,
- negatively skewed,
- and non-skewed.

The authors associate these different search effects with feature selection.

Positively Skewed Gaussian Search

This tends to remove features from the current leader.

It may help eliminate:

- irrelevant variables,
- noisy variables,
- or redundant variables.

Negatively Skewed Gaussian Search

This tends to include more features.

It may recover useful attributes omitted from the current global-best subset.

Standard Gaussian Search

This performs balanced local perturbation around the leader without a strong preference for adding or removing features.

The simplified leader update is:

`new gbest = current gbest + step size × Gaussian value × search range`

The step size is set to:

`0.1`

The new leader replaces the original global best only when it produces better fitness.

Worst-Solution Enhancement in PSOVA1

PSOVA1 identifies the three weakest particles in the swarm.

Different mechanisms are applied to improve them.

Mirroring Mutation

The worst particle is regenerated using a modified version of the global-best solution.

For each dimension, the value may either:

- retain the global-best value,

- or reverse its sign.

In copy-safe form:

`new value = -gbest value when mutation occurs`

`new value = gbest value otherwise`

The mutation probability threshold is:

`rmu = 0.9`

This produces an offspring that preserves much of the best solution while changing selected dimensions.

Differential Evolution Mutation

The second- and third-worst particles are modified using three randomly selected personal-best solutions.

The donor solution can be written as:

`donor = pbest1 + F × (pbest2 - pbest3)`

A crossover operation then chooses values from either:

- the donor solution,
- or the particle's personal-best solution.

The Differential Evolution weight `F` is generated using a sinusoidal chaotic map.

The crossover factor is generated using the Logistic chaotic map.

Acceptance of Worse Mutated Solutions

A better mutated solution is accepted directly.

A worse solution may still be accepted using an annealing-style probability.

In copy-safe form:

`acceptance probability = exp(-fitness difference / temperature)`

The temperature is reduced using:

$\text{new temperature} = 0.9 \times \text{current temperature}$

This mechanism allows weak particles to explore alternatives instead of accepting only immediate improvements.

Enhanced Personal and Global Signals

PSOVA1 improves the conventional **pbest** and **gbest** signals.

Enhanced Global Signal

The algorithm identifies a strong neighboring personal-best solution that is highly different from the current global best.

The enhanced global signal is:

$\text{enhanced gbest} = (\text{gbest} + \text{most dissimilar strong pbest}) / 2$

This gives the particle guidance from:

- the strongest known global solution,
- and a different but still high-performing personal solution.

Enhanced Personal Signal

A strong neighboring personal-best solution is selected randomly.

The enhanced personal signal is:

$\text{enhanced pbest} = (\text{current particle pbest} + \text{selected neighboring pbest}) / 2$

The normal PSO position update is then conducted using the enhanced personal and global signals.

This is intended to increase cooperation between particles and reduce dependence on one leader.

Spiral Local Search

PSOVA1 also uses a logarithmic spiral search around the global best.

The simplified position update is:

```
new position = distance from gbest × exponential spiral movement + gbest
```

The search follows a spiral-shaped path around the leader.

This creates a local region around the global best in which particles can perform detailed fine-tuning.

The spiral search is used more frequently during the later stages of optimization.

Dynamic Switching in PSOVA1

PSOVA1 switches between:

- enhanced PSO exploration,
- and spiral local exploitation.

The switching probability is:

```
pswitch = 1 - (current iteration / maximum iterations)^2
```

At the beginning:

- `pswitch` is high,
- so enhanced PSO exploration is used more often.

Near the end:

- `pswitch` becomes lower,
- so spiral exploitation is used more frequently.

This gradually changes the search from broad exploration to detailed refinement.

Proposed PSOVA2 Model

PSOVA2 further modifies PSOVA1.

It introduces four main strategies:

1. Adaptive exemplar breeding
2. Nonlinear search-coefficient generation

3. Hybrid redispaching of weak particles
4. Exponential search around the swarm leader

PSOVA2 gives particles more freedom in choosing:

- which target solution to follow,
- and how to move toward or away from that target.

Adaptive Exemplar Breeding

Rather than following only one global best, PSOVA2 creates an **exemplar** by combining:

- several personal-best solutions,
- and the global-best solution.

The number of personal-best solutions used decreases over time.

Early Search

During approximately the first 25 iterations:

3 personal-best solutions are combined

This creates broad perturbation and diversity.

Middle Search

During the next stage:

2 personal-best solutions are combined

Later:

1 personal-best solution is used

Final Search

Near the end:

0 personal-best solutions are used

The exemplar then becomes dominated by the global-best solution.

This creates the following transition:

- early iterations emphasize diversity,
- later iterations emphasize convergence.

Adaptive Exemplar Weights

Two dynamic weights control the contributions of:

- the global-best solution,
- and the combined personal-best solution.

The global-best weight increases from approximately:

0.4 toward 1.0

The personal-best contribution decreases from approximately:

0.4 toward 0

The exemplar can be represented as:

$$\text{exemplar} = \text{global-best weight} \times \text{gbest} + \text{personal-best weight} \times \text{combined pbest}$$

As the search progresses, the exemplar becomes increasingly similar to the global best.

Nonlinear Search Coefficients

PSOVA2 generates movement coefficients using four nonlinear functions based on:

- hyperbolic tangent,
- sine,
- cosine,
- and inverse cosine.

Each function produces a different type of movement.

Function 1

Produces gradual values ranging approximately from:

-0.3 to 0.5

This allows particles to:

- slowly move toward the target,
- or mildly move away from it.

Function 2

Produces larger and more irregular values ranging approximately from:

-0.9 to 0.9

This creates large jumps and may help particles escape local optima.

Function 3

Produces high positive values approximately between:

0.5 and 1.0

This pushes particles strongly toward promising solutions and accelerates convergence.

Function 4

Produces gradual positive values approximately between:

0 and 0.7

This supports detailed exploitation near strong solutions.

Each particle randomly selects one of the four coefficient-generation functions.

The particle also chooses between:

- the adaptive exemplar,
- and the global-best solution

as its target in each dimension.

PSOVA2 Position Update

The simplified update is:

```
new position = current position + search coefficient × (target -  
current position) + Gaussian disturbance
```

The target may be:

- the bred exemplar,
- or the global best.

The movement coefficient may cause the particle to:

- move rapidly toward the target,
- move slowly toward it,
- overshoot it,
- or temporarily move away from it.

This produces varied search trajectories.

Hybrid Redispatching of Weak Particles

PSOVA2 replaces the three weakest particles using a hybrid donor-generation mechanism.

Two possible donor sources are created.

Random Donor

A value is generated randomly within the allowed search range.

In copy-safe form:

```
random donor = lower bound + random value × (upper bound - lower  
bound)
```

Personal-Best Donor

Each dimension is inherited from a randomly selected personal-best solution.

Hybrid Offspring

For each dimension:

- there is a 50% chance of using the random donor,
- and a 50% chance of using the personal-best donor.

This combines:

- exploration from random reinitialization,
- and useful information from previously successful particles.

The procedure is repeated to replace the three weakest swarm members.

PSOVA2 Leader Enhancement

PSOVA2 locally improves the global best using a small exponential search step.

The generated step is limited to approximately:

`-0.5 to 0.5`

The simplified update is:

`new gbest = current gbest + exponential step × search range`

The offspring replaces the leader only when it produces better fitness.

This allows local exploration around the current strongest solution.

Feature-Subset Representation

Particles contain continuous numerical values during optimization.

Before evaluation, every dimension is converted into a binary feature-selection decision.

The paper uses the following interpretation:

- positive value = feature selected
- non-positive value = feature not selected

Therefore:

`1 = selected feature`

`0 = unselected feature`

The number of particle dimensions equals the number of features in the dataset.

K-Nearest Neighbor Classifier

The selected subsets are evaluated using a K-Nearest Neighbor classifier.

The number of neighbors is:

$K = 5$

During training, the classifier uses:

`10-fold cross-validation`

The classifier is trained using only the features selected by the particle.

The final global-best feature subset is then evaluated on an unseen test set.

Fitness Function

The fitness function combines:

- classification accuracy,
- and the inverse of the number of selected features.

In copy-safe form:

`fitness = 0.9 × classification accuracy + 0.1 × (1 / number of selected features)`

This gives greater importance to accuracy.

The weights are:

- classification accuracy weight = 0.9
- feature-count weight = 0.1

A strong particle therefore has:

- high predictive accuracy,
- and relatively few selected features.

Experimental Dataset Collection

The paper uses 13 datasets.

They cover:

- disease diagnosis,
- gene expression,
- speech and voice,
- human activity,
- facial expressions,
- seizure detection,
- and general classification.

The datasets contain between:

- 30 and 22,283 attributes,
- 2 and 10 classes,
- and 106 to 4,600 instances.

Datasets Used

Dataset	Feature s	Classe s	Instance s
Crohn	22,283	2	127
Myeloma	12,625	2	173
Arcene	10,000	2	200
MicroMass	1,300	10	360

Parkinson's	753	2	756
Activity	561	6	1,000
Voice	310	2	126
Facial Expression	301	2	1,062
Seizure	178	2	4,600
Acute Lymphoblastic Leukaemia	80	2	180
Heart	72	4	124
Ionosphere	33	2	253
Wisconsin Breast Cancer	30	2	569

The datasets include extremely high-dimensional cases with relatively few observations, especially:

- Crohn,
- Myeloma,
- and Arcene.

Comparison Algorithms

The proposed models are compared with five classical optimization methods:

1. Standard PSO

2. Differential Evolution
3. Sine Cosine Algorithm
4. Dragonfly Algorithm
5. Grey Wolf Optimization

They are also compared with ten PSO variants:

1. Competitive Swarm Optimizer
2. HPSO-SSM
3. Binary PSO
4. Modified Binary PSO
5. Catfish Binary PSO
6. Genetic PSO
7. MPSO-ELM
8. Multi-Subpopulation PSO
9. Bare-Bones PSO Variant
10. Asynchronous Learning PSO

Experimental Fairness

All algorithms use the same maximum number of fitness evaluations.

The selected settings are:

- population size = 30
- maximum iterations = 100
- total evaluations = 3,000
- independent runs = 30

This controls the principal computational budget across the methods.

Parameters for the comparison algorithms are taken from their original studies.

Performance Measures

The study evaluates:

- classification accuracy,
- F-score,
- number of selected features,
- convergence behavior,
- computational cost,

- and statistical significance.

Results are averaged over 30 independent runs.

Classification Accuracy Results

PSOVA1 and PSOVA2 obtain the highest classification accuracy on all 13 datasets.

PSOVA1 obtains the highest accuracy on:

- Parkinson's
- Facial Expression

PSOVA2 obtains the highest accuracy on the remaining 11 datasets.

Examples of PSOVA2 mean accuracy include:

Dataset	PSOVA2 accuracy
Crohn	0.8333
Myeloma	0.7545
Arcene	0.7694
MicroMass	0.6612
Activity	0.9117
Voice	0.8632

Seizure 0.8860

ALL 0.9241

Heart 0.7241

Ionosphere 0.8434

Wdbc 0.9585

Notable Accuracy Improvements

On the Heart dataset, PSOVA2 outperforms three strong classical methods by approximately:

- 6.21%
- 7.97%
- 8.06%

On MicroMass, it outperforms three leading alternatives by approximately:

- 4.88%
- 4.94%
- 5.51%

The advantages are especially visible on:

- high-dimensional datasets,
- datasets with small class separation,
- and datasets with complicated feature interactions.

F-Score Results

PSOVA1 and PSOVA2 also obtain the highest F-score results across all 13 datasets.

PSOVA2 produces the highest F-score on most datasets.

Examples include:

Dataset	PSOVA2 F-score
Crohn	0.8585
Myeloma	0.8551
Arcene	0.7130
MicroMass	0.6967
Activity	0.9131
Voice	0.7852
Seizure	0.8759
ALL	0.9408
Heart	0.7271
Ionosphere	0.8625
Wdbc	0.9432

The standard deviations are generally comparable to or lower than those of the baseline methods.

The authors interpret this as evidence that the models produce reliable results across repeated runs.

Clinical Feature Findings

The authors examine some features frequently selected by the models.

Acute Lymphoblastic Leukaemia

Frequently selected characteristics include:

- cytoplasm area,
- nucleus area,
- nucleus-to-cytoplasm ratio,
- form factor,
- compactness,
- perimeter,
- and eccentricity.

The nucleus-to-cytoplasm ratio is highlighted as particularly important.

The authors argue that baseline methods sometimes select only one of the interacting nucleus or cytoplasm measures, while the proposed methods more consistently preserve their combined diagnostic information.

Heart Disease

Frequently selected features include:

- chest-pain type,
- serum cholesterol,
- maximum heart rate,
- and ST depression.

These are recognized indicators for coronary heart disease classification.

Statistical Testing

The paper uses the Wilcoxon rank-sum test to compare classification accuracy distributions.

A result below:

$$p = 0.05$$

is treated as statistically significant.

PSOVA1 significantly outperforms all 15 baseline methods on five datasets:

- Crohn
- Myeloma
- MicroMass
- Parkinson's
- Activity

PSOVA2 significantly outperforms all baseline methods on eight datasets:

- Crohn
- Myeloma
- Arcene
- MicroMass
- Parkinson's
- Activity
- Seizure
- Heart

PSOVA2 therefore shows stronger statistical superiority than PSOVA1.

Number of Selected Features

The proposed models do not always produce the smallest feature subsets.

Some competing methods, particularly Competitive Swarm Optimization and Grey Wolf Optimization, select fewer features on several datasets.

However, these extremely small subsets frequently produce lower classification accuracy.

The authors argue that selecting the fewest features is not automatically desirable when important information is removed.

PSOVA1 and PSOVA2 instead seek a balance between:

- dimensionality reduction,
- and predictive performance.

Feature-Elimination Rates

PSOVA2 eliminates approximately:

- 65.46% of Crohn features
- 66.22% of Myeloma features
- 65.88% of Arcene features
- 63.32% of MicroMass features
- 63.51% of Parkinson's features
- 66.93% of Activity features
- 64.84% of Voice features
- 67.54% of Facial Expression features

These reductions are achieved while obtaining the strongest classification performance.

Example Selected Feature Counts

Average PSOVA2 feature counts include:

Dataset	Original features	PSOVA2 selected
Crohn	22,283	7,697.6
Myeloma	12,625	4,264.5
Arcene	10,000	3,412.4
MicroMass	1,300	476.8
Activity	561	185.5

Seizure	178	12.2
ALL	80	15.8
Ionosphere	33	6.9
Wdbc	30	7.9

Convergence Results

The authors analyze convergence on the high-dimensional:

- Myeloma dataset,
- and Crohn dataset.

PSOVA1 and PSOVA2 converge faster than most baseline methods while continuing to improve throughout the search.

PSOVA2 performs particularly well in later iterations.

The authors attribute this to:

- adaptive exemplar breeding,
- multiple target signals,
- and nonlinear movement coefficients.

Competitive Swarm Optimization converges rapidly, but the authors argue that it stagnates early after removing too many features.

Its rapid convergence therefore does not produce the strongest final classification results.

Computational Cost

All methods use the same number of function evaluations.

Because KNN-based fitness evaluation is the most expensive part of the process, overall computational cost is mainly determined by:

`population size × number of iterations × classifier evaluation cost`

Internal algorithm operations create smaller differences.

For the analyzed datasets, PSOVA1 and PSOVA2 generally have comparable or lower training time than most baseline methods.

Examples of average training times include:

Crohn

- PSOVA1 = approximately 0.347 seconds
- PSOVA2 = approximately 0.317 seconds

Myeloma

- PSOVA1 = approximately 0.290 seconds
- PSOVA2 = approximately 0.266 seconds

Seizure

- PSOVA1 = approximately 12.6 seconds
- PSOVA2 = approximately 11.6 seconds

PSOVA2 is the fastest method reported for the Seizure dataset.

Component Evaluation

The paper performs an ablation-style evaluation to examine the contribution of each proposed mechanism.

For PSOVA1, the authors progressively add:

1. Leader enhancement
2. Worst-particle enhancement
3. Enhanced personal and global signals
4. Spiral search

For PSOVA2, they progressively add:

1. Leader enhancement
2. Worst-particle enhancement
3. Exemplar breeding
4. Nonlinear coefficient generation

Each added component improves classification accuracy on the Seizure and Voice datasets.

PSOVA1 Component Results

On the Seizure dataset:

Model stage	Accuracy
Standard PSO	0.8459
Leader enhancement	0.8475
Leader and weak-particle enhancement	0.8510
Enhanced signals added	0.8672
Spiral search added	0.8698

On the Voice dataset:

Model stage	Accuracy
Standard PSO	0.8237

Leader enhancement	0.8281
--------------------	--------

Leader and weak-particle enhancement	0.8316
--------------------------------------	--------

Enhanced signals added	0.8491
------------------------	--------

Spiral search added	0.8526
---------------------	--------

PSOVA2 Component Results

On the Seizure dataset:

Model stage	Accuracy
Standard PSO	0.8459
Leader enhancement	0.8463
Leader and weak-particle enhancement	0.8495
Exemplar breeding added	0.8733
Nonlinear coefficients added	0.8860

On the Voice dataset:

Model stage	Accuracy
Standard PSO	0.8237
Leader enhancement	0.8254
Leader and weak-particle enhancement	0.8298
Exemplar breeding added	0.8535
Nonlinear coefficients added	0.8632

The largest improvements come from:

- enhanced signals and spiral search in PSOVA1,
- exemplar breeding and nonlinear search in PSOVA2.

Benchmark-Function Evaluation

The authors also test the proposed mechanisms on 11 numerical benchmark functions with:

30 dimensions

These include:

- Ackley
- Dixon
- Griewank
- Rastrigin
- Rotated Hyper-Ellipsoid
- Rosenbrock
- Sphere
- Sum Powers

- Zakharov
- Sum Squares
- Powell

The purpose is to determine whether each added mechanism improves optimization independently of the feature-selection application.

The results generally show progressive improvement as each component is added.

The complete PSOVA1 and PSOVA2 models achieve substantially lower objective values than standard PSO.

PSOVA2 generally produces the strongest final values on many benchmark functions.

Main Findings

The paper's principal findings are:

- Standard PSO is vulnerable to premature convergence in feature-selection problems.
- Following one global leader reduces swarm diversity.
- Weak local exploitation limits fine-tuning near optimal subsets.
- PSOVA1 improves PSO through enhanced leader signals, mutation, Gaussian search, and spiral exploitation.
- PSOVA2 further improves search using adaptive exemplars and nonlinear movement functions.
- Hybrid leader information improves cooperation between particles.
- Weak-particle redispaching restores diversity.
- Leader enhancement strengthens exploitation around promising solutions.
- The fitness function balances classification performance and subset size.
- Both proposed methods achieve the highest classification accuracy on all 13 datasets.
- They also obtain the highest F-scores across all datasets.
- PSOVA2 performs best on 11 of the 13 accuracy comparisons.
- PSOVA2 shows statistically significant superiority over all baselines on eight datasets.
- Both models work particularly well on high-dimensional tasks.
- They reduce dataset dimensionality substantially without eliminating too much useful information.
- Extremely small feature subsets can reduce predictive performance.
- The proposed models converge quickly while continuing to improve in later iterations.
- Computational costs remain comparable with the baseline algorithms.
- Component tests show that every proposed mechanism contributes to performance.
- PSOVA2 generally provides a better exploration–exploitation balance than PSOVA1.

Important Limitations

Complex Algorithm Design

The proposed methods contain many interacting components.

PSOVA1 includes:

- chaotic initialization,
- Gaussian leader improvement,
- mirroring mutation,
- Differential Evolution mutation,
- annealing-based acceptance,
- enhanced personal and global signals,
- and spiral search.

PSOVA2 includes:

- adaptive exemplar breeding,
- four nonlinear movement functions,
- hybrid redispaching,
- exponential leader search,
- and chaotic initialization.

It is therefore difficult to attribute improvements to PSO alone.

Many Empirically Selected Parameters

Several settings are selected using:

- recommendations from previous papers,
- or trial-and-error experiments.

Examples include:

- mutation probability,
- step size,
- switching thresholds,
- exemplar probability,
- and fitness weights.

Different datasets may require different values.

KNN-Specific Evaluation

The wrapper uses only a five-neighbor KNN classifier.

The selected subsets may behave differently with:

- support-vector machines,
- decision trees,
- neural networks,
- logistic regression,
- or ensemble classifiers.

Accuracy-Heavy Fitness Function

The fitness function assigns:

- 90% importance to accuracy,
- and 10% to subset size.

This favors predictive performance over aggressive dimensionality reduction.

A different weighting could produce different subsets and algorithm rankings.

Limited Sample Sizes in Some Datasets

Some high-dimensional datasets contain relatively few instances.

For example:

- Crohn has 22,283 features but only 127 samples.
- Myeloma has 12,625 features but only 173 samples.
- Arcene has 10,000 features but only 200 samples.

These conditions create a high risk of overfitting.

No External Validation Dataset

The study uses training, cross-validation, and unseen test partitions, but it does not validate the selected features using an entirely separate external dataset.

No Direct Interpretability Analysis for Every Dataset

The paper discusses clinically meaningful features for selected medical datasets.

However, it does not deeply interpret the selected features for all 13 tasks.

Computational Scale

Although the reported times are modest, wrapper feature selection requires repeated classifier training.

The method may become costly with:

- much larger sample sizes,
- slower classifiers,
- deeper models,
- or many more iterations.

Future Work

The authors propose exploring:

- additional hybrid leader-breeding mechanisms,
- more advanced search-signal generation,
- computer-vision applications,
- deep neural architecture generation,
- object detection,
- image classification,
- and image-description generation.

Conclusion

The paper concludes that PSOVA1 and PSOVA2 are effective wrapper-based feature-selection methods.

Both variants improve standard PSO by introducing:

- multiple search signals,
- leader enhancement,
- weak-particle improvement,
- and diverse local and global search operations.

PSOVA1 improves cooperation and local refinement through:

- modified personal and global best signals,
- Gaussian leader search,

- mutation,
- and spiral exploitation.

PSOVA2 provides stronger performance through:

- adaptive exemplar breeding,
- nonlinear search coefficients,
- hybrid redispaching,
- and exponential leader enhancement.

Across 13 datasets, both methods produce:

- high classification accuracy,
- strong F-scores,
- substantial feature reduction,
- competitive computation times,
- and statistically significant improvements over many comparison methods.

PSOVA2 is the stronger of the two proposed models because it provides particles with more diverse target solutions and more varied ways of moving through the search space.

A PSO-Based Multi-Objective Multi-Label Feature Selection Method in Classification

Yong Zhang, Dun-wei Gong, Xiao-yan Sun, and Yi-nan Guo

Summary

This paper proposes a Particle Swarm Optimization-based method for feature selection in **multi-label classification**.

The proposed algorithm is designed to identify multiple useful feature subsets representing different trade-offs between two conflicting objectives:

1. Minimize multi-label classification error.
2. Minimize the number of selected features.

Rather than combining these objectives into one weighted score, the method treats feature selection as a **multi-objective optimization problem**. It searches for a Pareto set of non-dominated feature subsets so that a decision-maker can choose between:

- a smaller subset with slightly higher error,
- or a larger subset with better classification performance.

The proposed algorithm contains four important mechanisms:

- probability-based particle encoding,
- an external archive of non-dominated solutions,
- adaptive uniform mutation,
- and a local learning strategy.

The method is tested on six multi-label datasets and compared with:

- ReliefF combined with Binary Relevance,
- a mutual-information feature-selection method,
- and NSGA-II.

The results show that the proposed PSO method generally finds feature subsets with lower Hamming loss, stronger Pareto-front coverage, and better multi-objective performance than the comparison methods.

Multi-Label Classification

In traditional single-label classification, every example belongs to one class.

In multi-label classification, one example may belong to several labels at the same time.

For example:

- an image may contain a beach, sunset, and people,
- a song may be described as energetic, happy, and fast,
- and a document may belong to several topics.

The labels are not mutually exclusive.

This makes feature selection more difficult because a feature may be useful for one label but less useful for another.

Multi-Label Feature Selection

Multi-label feature selection aims to remove:

- irrelevant features,
- redundant features,
- noisy features,
- and features that do not contribute meaningfully to label prediction.

A smaller feature subset may provide:

- lower classifier complexity,
- faster model training,
- reduced storage requirements,
- simpler models,
- and potentially improved classification performance.

However, removing too many features can eliminate information required to predict some labels.

The method must therefore balance feature reduction against predictive error.

Difficulty of the Search Problem

For a dataset with D available features, the number of possible subsets is:

2^D

This search space grows exponentially.

Even moderately sized feature sets can therefore contain too many combinations for exhaustive evaluation.

The paper uses Particle Swarm Optimization because PSO offers:

- simple implementation,
- population-based global exploration,
- fast convergence,
- and the ability to search many candidate feature subsets simultaneously.

Limitations of Earlier Approaches

Earlier multi-label feature-selection methods often transform a multi-label task into one or more single-label problems.

However, these transformations may:

- create a large number of artificial classes,
- lose relationships among labels,
- increase computational cost,
- and reduce learning performance.

Previous evolutionary methods include Genetic Algorithms and Differential Evolution.

The authors note that Genetic Algorithm-based approaches can require substantial computation and may converge prematurely.

The proposed method directly treats multi-label feature selection as a multi-objective search problem.

Multi-Objective Formulation

The proposed algorithm optimizes two objectives:

Objective 1: Classification Error

Classification error is measured using **Hamming loss**.

A lower Hamming loss means that fewer labels are incorrectly predicted.

Objective 2: Feature-Subset Size

The second objective is the number of selected features.

A smaller value means the classifier uses fewer variables.

The multi-objective problem can be written in copy-safe form as:

`Minimize [Hamming loss, number of selected features]`

The algorithm does not force these objectives into one score.

Instead, it identifies feature subsets that are non-dominated.

Pareto Dominance

A feature subset dominates another subset when it is:

- no worse in both objectives,
- and better in at least one objective.

For example, Feature Subset A dominates Feature Subset B when:

- A has equal or lower Hamming loss,
- A uses equal or fewer features,
- and at least one of those comparisons is strictly better.

A non-dominated subset cannot be improved in one objective without worsening the other.

The complete collection of non-dominated subsets forms an approximate **Pareto front**.

Standard Particle Swarm Optimization

In standard PSO, every particle has:

- a current position,
- a velocity,
- a personal-best position,
- and a global-best guide.

The velocity update can be represented in copy-safe form as:

```
new velocity = inertia × current velocity  
  
+ cognitive coefficient × random value × (personal best - current  
position)  
  
+ social coefficient × random value × (global best - current position)
```

The new position is:

```
new position = current position + new velocity
```

The main parameters are:

- w = inertia weight
- $c1$ = personal learning coefficient
- $c2$ = social learning coefficient
- $r1$ and $r2$ = random values between 0 and 1

Probability-Based Encoding

Feature selection is inherently binary:

- 1 means the feature is selected.
- 0 means the feature is not selected.

However, ordinary PSO operates in a continuous numerical space.

The paper uses probability-based real encoding to connect continuous PSO with discrete feature selection.

Each particle contains one real value for every feature.

Each value represents the probability or tendency that the corresponding feature is selected.

The decoding rule is:

```
Select feature d when particle value d > 0.5
```

```
Do not select feature d when particle value d <= 0.5
```

For example, the particle:

0.82, 0.34, 0.71, 0.19

represents the feature subset:

1, 0, 1, 0

Therefore, Features 1 and 3 are selected.

This representation allows standard continuous PSO movement equations to be used while still producing discrete feature subsets.

Hamming Loss

Hamming loss measures the fraction of labels that are predicted incorrectly.

It considers both:

- labels that should have been predicted but were missed,
- and labels that were predicted incorrectly.

A copy-safe representation is:

Hamming loss = total number of incorrect label decisions / total number of possible label decisions

For a dataset with:

- S test examples,
- and C possible labels,

the denominator is:

$S \times C$

A lower Hamming loss is better.

A Hamming loss of zero would mean every label for every example was predicted correctly.

Particle Fitness

Each particle is evaluated using two values:

Fitness of particle = [Hamming loss, number of selected features]

The classifier used to calculate Hamming loss is **ML-KNN**, a multi-label adaptation of K-Nearest Neighbor.

A particle is therefore not assigned one scalar fitness value. It is evaluated according to its position relative to other particles in the two-objective space.

External Archive

The algorithm maintains an external archive containing non-dominated solutions discovered during the search.

The archive allows the swarm to remember several strong feature subsets rather than only one global-best subset.

The archive performs several roles:

- preserves elite solutions,
- represents the current Pareto front,
- provides global guides to particles,
- and offers candidate solutions for local learning.

When new particles are evaluated:

1. Non-dominated particles are added to the archive.
2. Solutions dominated by new entries are removed.
3. If the archive exceeds its maximum size, crowded solutions are deleted.

Crowding Distance

Crowding distance estimates how isolated a solution is from neighboring archive solutions.

A large crowding distance means the solution lies in a relatively sparse part of the objective space.

A small crowding distance means many similar solutions exist nearby.

When the archive is full, solutions in crowded regions are more likely to be removed.

This preserves a diverse spread of trade-offs across the Pareto front.

Crowding distance is also used when selecting global-best guides.

Solutions in sparse regions have a greater probability of being selected.

Updating Personal-Best Solutions

Each particle maintains a personal-best solution.

The update is based on Pareto dominance.

For a particle:

- if its new position dominates its previous personal best, the new position replaces it;
- otherwise, the previous personal best remains stored.

The paper does not state that an incomparable new solution automatically replaces the old one.

Selecting Global-Best Guides

Unlike standard PSO, there is no single global-best solution because several non-dominated feature subsets may be equally valuable.

For each particle:

1. Calculate crowding distances for archive members.
2. Conduct a binary tournament between archive solutions.
3. Prefer the solution with the larger crowding distance.
4. Use the selected archive member as the particle's global guide.

This encourages particles to explore sparse regions of the current Pareto front.

Adaptive Uniform Mutation

Fast convergence can cause PSO to approach a false or incomplete Pareto front.

The proposed adaptive mutation is intended to improve exploration.

It controls both:

- the probability that a particle is mutated,
- and the number of dimensions changed.

The mutation parameter decreases exponentially as the search progresses.

In simplified form:

$\text{mutation parameter} = 0.5 \times \text{exponential decay} + 0.01$

The exact decay depends on:

- the current iteration,
- and the maximum number of iterations.

Mutation Range

The number of particle dimensions mutated is:

$K = \text{maximum of } 1 \text{ and ceiling}(D \times \text{mutation parameter})$

Where:

- D is the number of available features,
- and K is the number of particle values to be reinitialized.

The mutation process is:

1. Generate the current mutation parameter.
2. Compare it with a random number.
3. Mutate the particle when the mutation parameter is larger.
4. Select K dimensions randomly.
5. Reinitialize those dimensions within the search space.

Early and Late Mutation Behavior

At the beginning of the search:

- mutation probability is relatively high,
- mutation affects many dimensions,
- and the swarm performs broad exploration.

Later in the search:

- mutation probability decreases,
- fewer dimensions are modified,
- and the algorithm performs finer local refinement.

Thus, the same operator changes gradually from exploration to exploitation.

Local Learning Strategy

The proposed method also includes a local learning strategy based on differential movement.

Its purpose is to explore sparse areas near high-quality archive solutions.

The procedure is:

1. Select an archive solution with a large crowding distance as the base solution.
2. Select two additional archive solutions randomly.
3. Calculate the difference between the two random solutions.
4. Add a scaled version of that difference to the base solution.
5. Generate several new candidate solutions.
6. Add suitable non-dominated candidates to the archive.

The update can be written as:

```
new solution = sparse elite solution + F × (archive solution 1 -  
archive solution 2)
```

Where:

F is randomly selected between 0.1 and 0.9

Purpose of Local Learning

The base solution has a large crowding distance, meaning it lies in a sparse part of the Pareto front.

The differential movement creates new solutions around that area.

This can:

- fill gaps in the Pareto front,
- improve solution diversity,
- exploit promising but underexplored regions,
- and generate additional trade-off solutions.

Complete Algorithm Procedure

The proposed method operates as follows:

1. Set the swarm size, archive capacity, and maximum iterations.
2. Randomly initialize particle positions.
3. Decode each particle into a feature subset.
4. Evaluate Hamming loss and number of selected features.
5. Save non-dominated solutions in the external archive.
6. Update each particle's personal-best solution.
7. Select a global guide from the archive using crowding distance.
8. Update particle velocities and positions.
9. Apply adaptive uniform mutation.
10. Evaluate the new particles.
11. Update the archive using Pareto dominance.
12. Remove crowded archive members when necessary.
13. Apply the local learning strategy.
14. Repeat until the maximum iteration count is reached.
15. Output the final archive as the Pareto set.

The flowchart on page 4 presents this as a repeating cycle from personal-best and global-best updating through particle movement, mutation, fitness evaluation, archive updating, and local learning.

Time-Varying Learning Coefficients

The algorithm uses iteration-dependent acceleration coefficients rather than keeping c_1 and c_2 fixed.

These coefficients change linearly throughout the search.

The purpose is to modify the relative influence of:

- individual particle experience,
- and social guidance from the archive.

The exact parameter equations are referenced from prior multi-objective PSO research.

Computational Complexity

The authors estimate the theoretical worst-case complexity as:

$$O(M \times N \times \log N)$$

Where:

- M is the number of objectives,
- N is the combined number of swarm and archive solutions.

The most computationally important operations include:

- Pareto-dominance comparisons,
- crowding-distance calculations,
- archive management,
- and local learning.

However, the authors emphasize that practical runtime is dominated by classifier-based fitness evaluation.

The actual runtime depends on:

- the number of features,
- the number of examples,
- the classifier,
- and the selected feature-subset sizes.

Experimental Datasets

The method is evaluated on six publicly available multi-label datasets.

Dataset	Training samples	Testing samples	Labels	Features
Flags	129	65	7	19
CAL500	250	252	174	68
Emotions	391	202	6	72
Yeast	1,500	917	14	103

Birds	322	323	19	260
Scene	1,211	1,196	6	294

The datasets cover areas including:

- image classification,
- bioinformatics,
- music emotion,
- and other multi-label applications.

Classification Method

The study uses:

ML-KNN

ML-KNN is a lazy-learning method designed for multi-label classification.

It predicts multiple labels based on the labels associated with nearby training examples.

The same classifier is used when evaluating feature subsets across the compared evolutionary algorithms.

Comparison Methods

The proposed method is compared with three alternatives.

RF-BR

This combines:

- ReliefF feature ranking,
- and Binary Relevance classification.

Binary Relevance transforms a multi-label problem into several independent binary classification tasks.

MI-PPT

This is a mutual-information-based feature-selection method using a problem-transformation approach.

NSGA-II

NSGA-II is a widely used multi-objective Genetic Algorithm.

Its major components include:

- non-dominated sorting,
- crossover,
- mutation,
- elitism,
- and crowding-distance-based diversity preservation.

Experimental Settings

For the proposed algorithm and NSGA-II:

- population or swarm size = 20
- total fitness evaluations = 2,000

For NSGA-II:

- mutation rate = $1 / \text{number of features}$
- crossover probability = 0.9

Using the same number of fitness evaluations gives the two evolutionary algorithms a similar evaluation budget.

Best Hamming-Loss Results

The authors first compare the individual feature subset with the smallest Hamming loss produced by each method.

Results for Emotions, Yeast, and Scene are:

Dataset	Method	Hamming loss	Selected features
Emotions	Proposed PSO	0.178	27
Emotions	MI-PPT	0.229	43
Emotions	RF-BR	0.220	17
Emotions	NSGA-II	0.183	47
Yeast	Proposed PSO	0.193	45
Yeast	MI-PPT	0.194	91
Yeast	RF-BR	0.240	41
Yeast	NSGA-II	0.196	56
Scene	Proposed PSO	0.088	164
Scene	MI-PPT	0.092	278
Scene	RF-BR	0.120	56

Scene	NSGA-II	0.092	123
-------	---------	-------	-----

The proposed algorithm obtains the lowest Hamming loss on all three datasets.

Comparison with RF-BR

RF-BR sometimes selects fewer features.

However, its Hamming loss is consistently higher.

For Scene:

- RF-BR selects 56 features,
- while the proposed algorithm selects 164.

However:

- RF-BR Hamming loss = 0.120
- proposed PSO Hamming loss = 0.088

The proposed algorithm therefore sacrifices greater dimensionality reduction to obtain substantially better label prediction.

Comparison with MI-PPT

For Yeast:

- proposed PSO selects 45 features,
- MI-PPT selects 91 features.

The proposed method also produces slightly lower Hamming loss:

- proposed PSO = 0.193
- MI-PPT = 0.194

For Scene:

- proposed PSO uses 164 features,
- MI-PPT uses 278,
- and the proposed method reduces Hamming loss from 0.092 to 0.088.

Comparison with NSGA-II

The proposed method achieves lower Hamming loss than NSGA-II on all three highlighted datasets.

For Emotions:

- proposed PSO = 0.178 using 27 features
- NSGA-II = 0.183 using 47 features

For Yeast:

- proposed PSO = 0.193 using 45 features
- NSGA-II = 0.196 using 56 features

For Scene:

- proposed PSO = 0.088 using 164 features
- NSGA-II = 0.092 using 123 features

For Scene, NSGA-II uses fewer features, but the proposed method produces better classification error.

Parallel Pareto Search

The paper emphasizes that population-based multi-objective PSO can generate several trade-off solutions in one run.

MI-PPT may need multiple executions to create a comparable set of alternatives.

The proposed method simultaneously explores solutions with:

- very small feature sets,
- medium-sized subsets,
- and larger, more accurate subsets.

Emotions Pareto Results

For Emotions, the proposed method performs worse than MI-PPT when only approximately six or seven features are selected.

However, its Hamming loss improves rapidly as additional features are included.

The proposed method's Hamming loss decreases from approximately:

23.6% to 18%

as the number of features increases from approximately:

8 to 19

The MI-PPT results fluctuate around approximately 24.5% and do not improve consistently.

The proposed algorithm's best plotted solution is approximately:

18.23% Hamming loss using 19 features

Yeast Pareto Results

For Yeast, the proposed method reduces Hamming loss from approximately:

20.6% to 19.3%

as the selected subset increases from roughly:

17 to 45 features

MI-PPT displays a less stable curve.

Its best Hamming loss is approximately:

19.7% using 35 features

The proposed method reaches approximately:

19.67%

and its best reported value is around:

19.3% using 45 features

Scene Pareto Results

For Scene, the methods perform similarly when fewer than approximately 80 features are selected.

Once more than 80 features are included, the proposed method produces clearly lower Hamming loss.

Its best result is:

8.8% Hamming loss using 164 features

This is approximately 0.8 percentage points lower than the best MI-PPT result.

Set-Coverage Metric

The paper uses set coverage to compare two Pareto fronts.

For two solution sets, $Z1$ and $Z2$:

$SC(Z1, Z2)$ = proportion of solutions in $Z2$ dominated by or equal to at least one solution in $Z1$

Interpretation:

- $SC(Z1, Z2) = 1$ means every solution in $Z2$ is dominated by or equal to a solution in $Z1$.
- $SC(Z1, Z2) = 0$ means no solution in $Z2$ is dominated by a solution in $Z1$.

Set-Coverage Results

Dataset	Proposed dominates MI-PPT	MI-PPT dominates proposed
Emotions	0.8571	0.1429
Yeast	1.0000	0
Scene	0.6429	0.4286

For Emotions:

- approximately 85.71% of MI-PPT solutions are dominated by the proposed method,
- while approximately 14.29% of proposed solutions are dominated by MI-PPT.

For Yeast:

- every MI-PPT solution is dominated by the proposed method,
- and no proposed solution is dominated by MI-PPT.

The Scene values still favor the proposed algorithm overall, although the difference is smaller.

Hypervolume Metric

Hypervolume measures both:

- convergence toward the ideal objective region,
- and diversity across the Pareto front.

A higher hypervolume value generally indicates a stronger multi-objective solution set.

The proposed algorithm and NSGA-II are each run:

`30 times`

A paired t-test is conducted at:

`significance level = 0.05`

Hypervolume Results

Dataset	Proposed algorithm	NSGA-II
Flags	0.738	0.592
CAL500	0.798	0.595

Emotions	0.766	0.553
Yeast	0.709	0.508
Birds	0.859	0.580
Scene	0.756	0.560

The value after the slash in the paper's table represents standard deviation.

The proposed method achieves higher average hypervolume on all six datasets.

The paired t-tests classify the proposed method as significantly better than NSGA-II for every dataset.

Pareto-Front Comparison with NSGA-II

The plots on page 7 compare the two algorithms across all six datasets.

The proposed algorithm's Pareto fronts are generally:

- closer to the low-error region,
- spread across more feature-subset sizes,
- and positioned below the NSGA-II fronts.

The authors report that every NSGA-II solution is dominated by at least one solution generated by the proposed PSO algorithm.

This indicates stronger convergence and overall Pareto-front quality.

Analysis of the Local Learning Strategy

The complete proposed algorithm is called:

MPSOFS

A version without the local learning strategy is called:

MPSOFS/LLS

The hypervolume curves on page 8 show that the complete method performs better throughout most of the search.

For Emotions at iteration 100:

- MPSOFS hypervolume = approximately 76.4%
- MPSOFS/LLS hypervolume = approximately 70.7%

The difference is almost six percentage points.

The Yeast and Scene curves show the same general pattern.

The authors conclude that local learning improves:

- convergence,
- sparse-region exploration,
- and final Pareto-front quality.

Analysis of Adaptive Mutation

A version without mutation is called:

MPSOFS/M

The complete mutation-based method achieves higher hypervolume curves on:

- Emotions,
- Yeast,
- and Scene.

For Yeast at iteration 100:

- MPSOFS hypervolume = approximately 70.9%
- MPSOFS/M hypervolume = approximately 67.1%

This supports the claim that mutation helps prevent premature convergence and improves exploration.

Comparison of Mutation Operators

The adaptive uniform mutation is compared with:

- non-uniform mutation,
- and Pareto-rank-based mutation.

The algorithms are:

- **MPSOFS** = proposed adaptive mutation
- **MPSOFS-NM** = non-uniform mutation
- **MPSOFS-PRM** = Pareto-rank mutation

Average hypervolume values are:

Dataset	MPSOFS	MPSOFS-NM	MPSOFS-PRM
Emotions	0.766	0.762	0.764
Yeast	0.709	0.686	0.710
Scene	0.756	0.738	0.754

The proposed mutation performs better than non-uniform mutation on all three datasets.

Its performance is very close to Pareto-rank mutation.

Pareto-rank mutation performs slightly better on Yeast, while the proposed mutation performs slightly better on Emotions and Scene.

Runtime of Mutation Methods

Pareto-rank mutation requires the construction of a Pareto-ranking relationship among particles whenever mutation is applied.

This increases runtime.

For Emotions, the approximate runtime ratio is:

MPSOFS : MPSOFS-PRM = 1 : 1.07

The proposed adaptive mutation therefore provides similar or slightly stronger performance with lower computational overhead.

Main Findings

The paper's main findings are:

- Multi-label feature selection naturally involves multiple conflicting objectives.
- Hamming loss and feature count should be optimized simultaneously.
- Probability-based encoding allows continuous PSO to represent discrete feature subsets.
- An external archive can preserve multiple non-dominated solutions.
- Crowding distance maintains a diverse Pareto front.
- Adaptive mutation provides broad exploration early and finer refinement later.
- Local differential learning improves sparse areas of the Pareto front.
- The proposed method finds multiple useful feature subsets in one execution.
- It obtains the lowest Hamming loss on the highlighted Emotions, Yeast, and Scene datasets.
- It generally dominates the MI-PPT Pareto solutions.
- It produces significantly higher hypervolume values than NSGA-II on all six datasets.
- Its Pareto fronts show stronger convergence than NSGA-II.
- Removing local learning reduces performance.
- Removing mutation also reduces performance.
- The adaptive mutation is more effective than non-uniform mutation.
- It performs similarly to Pareto-rank mutation with less computational overhead.

Important Limitations

Limited Dataset Scale

The largest dataset contains:

- 294 features,
- 1,196 test examples,
- and 19 labels at most, except CAL500 with 174 labels.

The study does not test modern datasets containing:

- tens of thousands of features,

- millions of examples,
- or extremely large label spaces.

One Classification Algorithm

All evolutionary methods use ML-KNN.

The selected subsets may perform differently with:

- neural networks,
- support-vector machines,
- decision trees,
- classifier chains,
- label-powerset methods,
- or deep multi-label models.

Hamming Loss as the Main Error Objective

Hamming loss gives equal importance to every label decision.

It may not fully represent:

- ranking quality,
- rare-label detection,
- example-level accuracy,
- label imbalance,
- or application-specific costs.

No Explicit Feature Costs

All features are treated as equally expensive.

The study does not consider:

- acquisition cost,
- sensor cost,
- medical test cost,
- computation time per feature,
- or missing-feature availability.

Fixed Threshold

A feature is selected when its particle value exceeds:

0.5

The paper does not compare this with:

- adaptive thresholds,
- learned thresholds,
- sigmoid sampling,
- or direct binary PSO representations.

Parameter Dependence

Performance depends on settings such as:

- swarm size,
- archive capacity,
- maximum iterations,
- acceleration coefficients,
- mutation decay,
- local-learning sample count,
- and differential scale factor.

The paper does not conduct a broad sensitivity analysis for all parameters.

Comparison Scope

The method is directly compared with only:

- two conventional feature selectors,
- and one multi-objective evolutionary algorithm.

It is not compared with a wide range of later or more advanced multi-objective PSO methods.

Runtime Reporting

The paper discusses theoretical complexity and relative mutation overhead but does not provide a comprehensive runtime table across all datasets and algorithms.

Statistical Testing

The paired t-test is used for hypervolume results.

The paper does not discuss whether distributional assumptions required by the test were checked.

Future Work

The authors propose extending metaheuristic feature selection to additional problems, particularly:

- cost-sensitive feature selection.

This would allow features to have different acquisition or usage costs rather than treating subset size as the only measure of simplicity.

Conclusion

The paper concludes that the proposed multi-objective PSO algorithm is an effective method for multi-label feature selection.

It searches simultaneously for feature subsets that:

- reduce Hamming loss,
- and reduce the number of selected features.

Its main mechanisms are:

- real-valued probability encoding,
- Pareto-based personal and archive updating,
- crowding-distance diversity preservation,
- adaptive uniform mutation,
- and differential local learning.

The experimental results show that the method:

- produces strong low-error feature subsets,
- generates several Pareto alternatives in one run,
- generally dominates the mutual-information method,
- and significantly outperforms NSGA-II in hypervolume across all six datasets.

The adaptive mutation and local learning components both make measurable contributions to search performance.

The Application of Simulated Annealing Method for Optimal Route Detection Between Objects

Peter Grabusts, Jurijs Musatovs, and Vladimir Golenkov

Summary

This paper demonstrates how **Simulated Annealing** can be applied to a practical route-optimization problem modeled as a Traveling Salesman Problem.

The study focuses on finding the shortest route connecting eight dairy and meat-processing enterprises in Belarus. Their geographic locations are represented using GPS coordinates, and the algorithm searches for a closed route that:

- visits every enterprise once,
- returns to the starting location,
- and minimizes the total travel distance.

The paper is primarily a practical demonstration rather than an extensive comparative or large-scale experimental study. It explains the basic concepts of Simulated Annealing, introduces the Traveling Salesman Problem, describes the neighborhood and cooling mechanisms used, and presents a route of approximately 648 kilometers after 20 search steps.

Simulated Annealing

Simulated Annealing, or SA, is a stochastic optimization method inspired by the physical annealing of materials.

In physical annealing, a material is heated and then cooled gradually. When cooling is sufficiently slow, the atoms have time to settle into a low-energy and highly ordered structure.

When cooling occurs too quickly, the material may freeze into a defective or nonoptimal arrangement.

The optimization equivalent is:

- the physical configuration represents a candidate solution,
- energy represents the objective or cost value,
- temperature controls the amount of randomness,

- and cooling gradually makes the search more selective.

The paper explains that SA can search objective functions containing several local minima without necessarily becoming trapped in the first one encountered.

Difference from Ordinary Local Search

A conventional local-search method generally accepts a new candidate only when it improves the current solution.

This can cause the search to stop at a local minimum.

Simulated Annealing may accept a worse solution temporarily.

This allows it to:

- leave a local minimum,
- cross an unfavorable region,
- and potentially reach a better part of the search space.

The acceptance of worse solutions becomes less likely as the temperature decreases.

Metropolis Acceptance Rule

The paper bases the SA method on the Metropolis procedure.

For every proposed movement, the change in objective value is calculated.

In copy-safe form:

$\text{Change in cost} = \text{candidate cost} - \text{current cost}$

The decision rule is:

- If the candidate has a lower cost, accept it.
- If the candidate has a higher cost, accept it with a probability determined by temperature.

The probability of accepting a worse solution can be written as:

$\text{Acceptance probability} = \exp(-\text{change in cost} / \text{temperature})$

The original physical expression also includes Boltzmann's constant:

Acceptance probability = $\exp(-\text{change in energy} / (\text{Boltzmann constant} \times \text{temperature}))$

For an optimization implementation, the constants may be absorbed into the artificial temperature scale.

Interpretation of Temperature

At high temperature:

- worsening moves are accepted relatively often,
- the algorithm explores broadly,
- and the route may change significantly.

At low temperature:

- worsening moves are accepted rarely,
- the algorithm becomes more conservative,
- and it focuses on improving the current route.

When the route no longer changes meaningfully, the paper describes the process as reaching **thermal equilibrium**.

Required Elements of a Simulated Annealing Model

The paper identifies several components that must be specified before SA can be applied.

Objective Function

The objective function measures the cost to be minimized.

In this study:

Objective value = total length of the complete route

Set of Candidate Solutions

The possible solutions are all valid tours through the enterprises.

Every valid tour must visit each object once and return to the initial object.

Variation Generator

The algorithm requires a method for producing a neighboring route from the current one.

The paper uses a route-reversal operation related to the 2-opt neighborhood.

Temperature

Temperature controls the probability of accepting worse solutions.

Cooling Schedule

The cooling schedule specifies how the temperature decreases throughout the search.

Apple-Basket Analogy

The paper uses an apple-basket analogy to explain the difference between deterministic and stochastic search.

In a complete deterministic search:

- an apple is removed,
- weighed,
- and not returned to the basket,
- while the best apple is retained.

In the SA analogy:

- an apple is drawn,
- weighed,
- returned to the basket,
- and may be selected again later.

This illustrates that SA can revisit candidate regions and does not permanently eliminate a state after one evaluation.

Basic Simulated Annealing Procedure

The algorithm can be represented in copy-safe form as follows:

1. Set the initial temperature.
2. Generate an initial route.
3. Repeat until thermal equilibrium is reached:
 - Generate a neighboring route.
 - Calculate the difference in route length.
 - Accept the neighbor when it is shorter.
 - When it is longer, generate a random number between 0 and 1.
 - Accept the longer route when the annealing probability is greater than the random number.
4. Reduce the temperature.
5. Continue until the stopping condition is met.
6. Return the shortest route found.

Traveling Salesman Problem

The Traveling Salesman Problem, or TSP, asks for the shortest closed route among several locations.

A valid route must:

- start from one location,
- visit each remaining location exactly once,
- and return to the starting location.

For n locations, the number of possible tours grows rapidly as n increases.

The paper treats each route as a cyclic permutation of the locations.

The route objective can be written as:

Route length = sum of the distances between consecutive locations
+ distance from the final location back to the starting location

The objective is:

Minimize total route length

Symmetric Distance Matrix

The paper considers a symmetric TSP.

This means:

Distance from location i to location j = distance from location j to location i

The distances are calculated from the GPS positions of the enterprises.

The paper describes these as points in a two-dimensional Euclidean space.

Neighboring Routes

A neighboring route is one that can be reached from the current route through one permitted change.

The study uses a **2-opt transition**.

In a 2-opt move:

1. Select two positions or cities in the route.
2. Reverse the order of the cities between those positions.
3. Reconnect the resulting route.

For example:

Current route:

A, B, C, D, E, F

After reversing the section from C through E:

A, B, E, D, C, F

The paper's Figure 1 illustrates the current route and the resulting route after reversing a selected section.

This operation changes the route without producing duplicates or omitting any location.

Cooling Coefficient

The temperature change coefficient used in the described approach is:

Cooling coefficient = 0.925

A simple cooling rule can therefore be represented as:

New temperature = 0.925 × current temperature

This gradually lowers the probability of accepting worse routes.

Practical Case Study

The application concerns the State Association Holding managing company known as **Concern Brestmiasomolprom**.

The organization operates in the Belarusian meat and dairy industry.

The paper states that the corporation has:

- approximately 13,000 employees,
- annual milk-processing capacity of approximately 2 million tons,
- and livestock-processing capacity of up to approximately 250,000 tons.

The route study includes eight dairy enterprises belonging to the group.

Enterprises Included

The eight locations are:

1. Liakhovich Milk Factory
2. Belovezhskaya Cheese
3. Berioza Cheese-Producing Plant
4. Baranovitchi Dairy Integrated Plant
5. Savushkin Product
6. Pruzhany Milk Mill
7. Luninets Dairy Factory
8. Kobrin Butter and Cheese Making Factory

GPS Coordinates

The paper provides the following geographic coordinates:

Enterprise	Latitude	Longitude
Liakhovich Milk Factory	53.031017	26.257463
Belovezhskaya Cheese	52.372727	23.369195
Berioza Cheese-Producing Plant	52.542828	24.988239
Baranovitchi Dairy Integrated Plant	53.115339	25.949680
Savushkin Product	52.112929	23.765061
Pruzhany Milk Mill	52.550889	24.458614
Luninets Dairy Factory	52.245241	26.846835
Kobrin Butter and Cheese Making Factory	52.211610	24.327513

Commas used as decimal separators in the original table have been converted here into decimal points for easier reuse.

Experimental Procedure

The eight locations are modeled as a TSP with eight objects.

The authors apply the SA algorithm for:

20 steps

During these steps, the candidate route length fluctuates.

This fluctuation occurs because SA may temporarily accept a longer route rather than improving monotonically at every step.

The objective value eventually stabilizes.

The authors interpret this stabilization as the attainment of thermal equilibrium.

Thermal-Equilibrium Results

The convergence chart on page 6 shows the route length moving through several increases and decreases.

The route begins at approximately:

745 kilometers

It then fluctuates considerably, including values around:

- 755 kilometers,
- 730 kilometers,
- 685 kilometers,
- 725 kilometers,
- 648 kilometers,
- 700 kilometers,
- and 645 to 650 kilometers.

The curve stabilizes during approximately Steps 16 through 20.

The chart labels the final stable result as:

Thermal equilibrium reached at route length 648 kilometers

This non-monotonic pattern is consistent with SA's willingness to accept worsening solutions during the search.

Shortest Route Result

The paper reports the shortest route as approximately:

648 kilometers

The route map on page 6 gives a more precise value of:

648.69 kilometers

The mapped route connects the eight enterprises in a complete cycle.

The plotted sequence appears to pass through the general order:

- Luninets
- Liakhovichi
- Baranovitchi
- Berioza
- Pruzhany
- Belovezhskaya
- Savushkin
- Kobrin
- and back toward Luninets

Because a TSP tour is cyclic, the same route may be written starting from any enterprise or in reverse order without changing its total length.

Geographic Route Map

The map on page 6 displays:

- latitude on the vertical axis,
- longitude on the horizontal axis,
- the eight enterprise locations,
- and straight route segments connecting them.

The route covers western and central areas of the Brest region.

The points farthest apart geographically appear to include:

- Luninets in the northeast,
- Savushkin and Belovezhskaya in the southwest,
- and Baranovitchi and Liakhovichi in the east.

The optimized tour attempts to connect nearby locations sequentially while avoiding unnecessary repeated crossings.

Route Recalculation

The paper notes that individual enterprises can be removed from the dataset and the route recalculated.

This makes the software potentially useful when:

- one location temporarily does not require service,
- a facility is excluded from a delivery cycle,
- a network changes,
- or logistics planners need to compare alternative combinations of destinations.

The model can then generate a revised route among the remaining objects.

Software Output

The authors state that they developed software that:

- stores object locations,
- models them on a map,
- applies Simulated Annealing,
- and calculates a short route.

The paper does not provide extensive software-architecture details or source code.

Its main purpose is to demonstrate that mathematical optimization can be integrated with GPS-based object mapping.

Main Findings

The paper's main findings are:

- Simulated Annealing can be applied to a practical GPS-based route problem.
- The Traveling Salesman Problem provides an appropriate model for visiting several enterprises.
- A 2-opt route-reversal operation can generate neighboring tours.
- SA may accept worse routes during the search.
- This allows the algorithm to move away from local minima.
- Temperature controls the probability of accepting longer routes.
- A cooling coefficient of 0.925 is used in the described implementation.
- The search reaches a stable route after approximately 20 steps.

- The final reported route is approximately 648 kilometers.
- The mapped result gives a more precise distance of approximately 648.69 kilometers.
- Locations can be removed and the route recalculated.
- The authors regard the approach as useful for practical transport-logistics planning.

Important Limitations

Very Small Problem Instance

The experiment contains only eight locations.

A TSP with eight objects is small enough that exact enumeration or a standard exact solver could feasibly determine the true optimum.

The paper does not compare SA with an exact solution.

Therefore, it does not prove that the 648-kilometer route is the global optimum.

No Comparison Algorithms

The study does not compare SA with methods such as:

- nearest-neighbor search,
- exhaustive enumeration,
- Tabu Search,
- Genetic Algorithms,
- Ant Colony Optimization,
- Particle Swarm Optimization,
- or exact TSP solvers.

Its results therefore demonstrate feasibility but do not establish superiority.

Limited Number of Steps

The algorithm is reported as running for only 20 steps.

The paper does not clearly specify:

- the number of candidate evaluations within each step,
- whether each step represents one temperature level,
- or how many neighbors are examined before equilibrium is declared.

Incomplete Parameter Reporting

The cooling coefficient is provided, but several other details are unclear, including:

- initial temperature,
- final temperature,
- number of repetitions,
- stopping threshold,
- random seed,
- number of transitions per temperature,
- and neighborhood-sampling procedure.

This makes exact replication difficult.

Straight-Line Geographic Distances

The route map is based on GPS coordinates and appears to connect points using geometric distance.

It does not clearly account for:

- actual road networks,
- road curvature,
- one-way roads,
- speed limits,
- bridge locations,
- traffic,
- road quality,
- tolls,
- or vehicle restrictions.

The calculated route may therefore differ from the shortest drivable route.

No Capacity or Delivery Constraints

The model does not consider:

- vehicle capacity,
- demand at each enterprise,
- delivery quantities,
- service times,
- time windows,
- fuel usage,
- driver hours,

- multiple vehicles,
- or depot constraints.

It is a basic TSP rather than a complete Vehicle Routing Problem.

Global-Optimality Claim

The paper states that SA can guarantee a global optimum.

In theory, convergence guarantees require carefully controlled cooling and potentially extremely long execution.

The short 20-step practical experiment does not independently establish that the final route is globally optimal.

No Repeated Trials

Because SA is stochastic, repeated runs may produce different routes.

The paper reports one principal result and does not provide:

- average route length,
- standard deviation,
- best and worst runs,
- or success frequency.

Limited Statistical Analysis

The study does not perform statistical tests or sensitivity analysis.

It does not examine how performance changes with:

- initial route,
- starting temperature,
- cooling coefficient,
- neighborhood choice,
- or number of iterations.

Conclusion

The paper concludes that Simulated Annealing can be used successfully for practical combinatorial optimization and route planning.

The authors apply SA to a Traveling Salesman Problem containing eight Belarusian dairy-processing enterprises represented through GPS coordinates.

The algorithm uses:

- probabilistic acceptance of worse routes,
- a gradually decreasing temperature,
- and a 2-opt route-reversal neighborhood.

After 20 reported steps, the route stabilizes at approximately:

648 kilometers

The mapped result gives a total of approximately:

648.69 kilometers

The developed software can also remove selected objects and recalculate the route.

Overall, the study presents a straightforward example of using Simulated Annealing and geographic coordinates to support logistics-route planning, although it does not provide enough comparative or statistical evidence to prove that the final route is globally optimal.

A Novel, Evolutionary, Simulated Annealing Inspired Algorithm for the Multi-Objective Optimization of Combinatorial Problems

Elias D. Niño, Carlos J. Ardila, and Anangélica Chinchilla

Summary

This paper proposes a hybrid multi-objective optimization algorithm called **EMSA**, or **Evolutionary Metaheuristic of Simulated Annealing**.

EMSA combines ideas from:

- Simulated Annealing,
- Genetic Algorithms,
- deterministic swapping,
- and multi-objective optimization.

The algorithm is presented as an improvement over an earlier method called **MODS**, or **Metaheuristic of Deterministic Swapping**.

The main difference is that EMSA creates a temporary **search direction** by assigning random weights to the objective functions. It then uses mutation, crossover, and probabilistic acceptance of dominated solutions to search for a diverse set of non-dominated solutions.

The method is evaluated using ten 100-city instances of the **Bi-Objective Traveling Salesman Problem** from TSPLIB. EMSA is compared only with MODS.

According to the reported metrics, EMSA produces substantially larger and more accurate non-dominated solution sets than MODS. In the paper's evaluation, EMSA achieves a reported error rate of 0% on every tested instance, while MODS records error rates between 99% and 100%.

Combinatorial Optimization

The paper defines combinatorial optimization as the search for the best solution within a discrete set of feasible alternatives.

Examples of candidate solutions may include:

- permutations,
- schedules,
- routes,
- assignments,
- and graph structures.

The Traveling Salesman Problem is presented as one of the most familiar combinatorial optimization problems.

Traveling Salesman Problem

The Traveling Salesman Problem, or TSP, asks for a minimum-cost tour through a collection of cities.

A valid solution must:

- leave each city exactly once,
- enter each city exactly once,
- visit every city,
- avoid disconnected subtours,
- and form one complete closed route.

The objective may be written in copy-safe form as:

Minimize total tour cost = sum of $C_{ij} \times X_{ij}$ for all city pairs i and j

Where:

- C_{ij} is the cost of traveling from city i to city j .
- $X_{ij} = 1$ when the route travels directly from city i to city j .
- $X_{ij} = 0$ otherwise.

The constraints require:

Each city has exactly one outgoing route.

Each city has exactly one incoming route.

Disconnected cycles containing only a subset of cities are prohibited.

The paper notes that TSP-related structures also appear in areas such as:

- heterogeneous machine scheduling,
- hard scheduling optimization,
- operations research,
- and theoretical computer science.

Multi-Objective Optimization

Unlike single-objective optimization, multi-objective optimization attempts to optimize two or more objectives simultaneously.

The general formulation is:

Optimize $F(X) = [f_1(X), f_2(X), \dots, f_n(X)]$

Subject to:

Equality constraints = 0

Inequality constraints ≤ 0

Lower bounds $\leq X \leq$ upper bounds

Because objectives may conflict, there is usually no single solution that is best in every objective.

The desired output is therefore a set of **non-dominated solutions**.

Dominated and Non-Dominated Solutions

A solution is dominated when another solution is:

- no worse in every objective,
- and strictly better in at least one objective.

A solution is non-dominated when no known solution dominates it.

The collection of non-dominated solutions forms a **Pareto front**.

For a bi-objective route problem, one route may be better for Objective 1 while another route is better for Objective 2. Neither route necessarily dominates the other.

Genetic Algorithms

The paper briefly explains Genetic Algorithms as methods inspired by natural selection.

Their three central stages are:

1. Selection
2. Crossover
3. Mutation

Selection

Two candidate solutions are selected from the population.

Crossover

Parts of the selected solutions are combined to produce a new candidate.

The purpose is to create solutions that inherit information from both parents.

Mutation

The new solution is perturbed.

Mutation introduces additional variation and may help the search escape local optima.

The paper mentions NSGA-II and SPEA2 as well-known multi-objective evolutionary algorithms.

Simulated Annealing

Simulated Annealing is presented as a probabilistic metaheuristic inspired by physical annealing.

At high temperature, the method may accept:

- good solutions,
- and worse solutions.

The probability of accepting a worse solution depends on:

- the amount by which it is worse,

- and the current temperature.

In copy-safe form:

```
Acceptance probability = exp(-change in weighted objective /
temperature)
```

At high temperature, dominated or worse solutions are more likely to be accepted.

As temperature decreases, the algorithm becomes more selective.

The paper states that at low temperature, Simulated Annealing begins to behave more like a restrictive local-search method.

Metaheuristic of Deterministic Swapping

MODS is the predecessor of EMSA.

It uses a deterministic finite-automaton representation called a **Multi-Objective Deterministic Finite Automaton**, or MDFA.

The automaton contains:

- states representing feasible solutions,
- an alphabet representing possible transformations,
- a transition function,
- an initial set of solutions,
- and the objective functions.

The notation can be summarized as:

```
MDFA = [states, input operations, transition rule, initial states,
objectives]
```

MODS Procedure

MODS uses two important sets.

S

This stores currently non-dominated solutions.

S*

This stores elite solutions that have dominated at least one other solution.

The simplified procedure is:

1. Create an initial solution set.
2. Store the initial solutions in the working set.
3. Select a solution randomly from the working or elite set.
4. Perturb the selected solution to generate neighbors.
5. Add non-dominated neighbors to the working set.
6. Add solutions that dominate existing members to the elite set.
7. Repeat until the stopping condition is met.

MODS relies mainly on deterministic neighborhood exploration.

Proposed EMSA Algorithm

EMSA combines:

- MODS-style deterministic swapping,
- Simulated Annealing acceptance,
- random objective weighting,
- mutation,
- and crossover.

Its purpose is to improve exploration and avoid becoming trapped around dominated or local-optimal solutions.

The algorithm maintains:

- an initial solution set,
- a feasible-solution space,
- a working non-dominated set,
- an elite set,
- objective functions,
- a permutation function,
- a crossover function,
- and a weight-generation function.

Random Objective Weights

A major contribution of EMSA is the use of randomly generated weights.

For a problem with n objectives, the algorithm generates:

$$W = [w_1, w_2, \dots, w_n]$$

Subject to:

$$w_1 + w_2 + \dots + w_n = 1$$

and:

$$0 \leq w_i \leq 1$$

Each weight determines the temporary importance of one objective.

The weighted objective is:

$$\text{Weighted objective} = w_1 \times f_1(X) + w_2 \times f_2(X) + \dots + w_n \times f_n(X)$$

Changing the weights changes the direction in which the search evaluates improvement.

The authors describe this as changing the **optimization angle** or search direction.

Search Direction

Suppose a bi-objective problem uses:

$$w_1 = 0.8$$

$$w_2 = 0.2$$

The current search direction emphasizes Objective 1.

Another iteration might use:

$$w_1 = 0.3$$

$$w_2 = 0.7$$

The search would then emphasize Objective 2.

By repeatedly changing these values, EMSA attempts to explore different regions of the Pareto front.

Initial Settings

The EMSA procedure begins by defining:

- the initial solution set,
- the working solution set,
- the elite set,
- the initial temperature,
- the number of objectives,
- and the cooling factor.

The experimental settings use:

`Initial temperature = 1,000`

`Cooling factor = 0.95`

The cooling operation is:

`New temperature = 0.95 × current temperature`

Mutation

The algorithm selects a solution from either:

- the working set,
- or the elite set.

It then applies a permutation or mutation function.

The mutated solution is placed into the relevant solution sets according to dominance relationships.

The algorithm continues mutating while the generated solution remains dominated under the current search condition.

Acceptance of Dominated Solutions

If a generated solution is dominated, EMSA may still accept it probabilistically.

The difference between the current and candidate solutions is calculated using the current random objective weights.

In copy-safe form:

$\text{Gamma} = \text{weighted value of current solution} - \text{weighted value of candidate solution}$

The probability value is:

$z = \exp(-\text{Gamma} / \text{current temperature})$

A random number between 0 and 1 is generated.

The dominated candidate is accepted when:

$\text{random number} < z$

This is the main Simulated Annealing-inspired mechanism.

At high temperature, a dominated solution has a greater probability of being accepted.

At low temperature, the probability decreases.

Crossover

EMSA also performs crossover between solutions.

The crossover combines:

- the current solution,
- and a newly generated or previously considered solution.

The crossover point is selected within the length of the solution representation.

The resulting candidate is sent back through the mutation and dominance-processing stages.

The paper states that crossover can involve:

- non-dominated solutions,
- and dominated solutions.

This allows information from otherwise rejected solutions to contribute to future candidates.

Removal of Dominated Solutions

At the end of a temperature stage, dominated solutions are removed from:

- the working set,
- and the elite set.

The remaining working set represents the current non-dominated population.

When the temperature reaches zero or the stopping condition is met, the final working set is returned as the estimated Pareto front.

Complete EMSA Procedure

The process can be summarized as follows:

1. Create an initial set of solutions.
2. Initialize the working and elite sets.
3. Set the starting temperature.
4. Set the cooling factor.
5. Reduce the temperature.
6. Select a solution randomly.
7. Generate a random set of objective weights.
8. Mutate the selected solution.
9. Evaluate dominance relationships.
10. Store useful solutions in the working or elite sets.
11. Consider accepting a dominated solution using the annealing probability.
12. Apply crossover when required.
13. Remove dominated solutions from the maintained sets.
14. Repeat with a new temperature and new search direction.
15. Return the final non-dominated set.

Avoiding Local Optima

The authors identify two main ways EMSA attempts to escape local optima.

Acceptance of Worse or Dominated Solutions

The Simulated Annealing mechanism allows the algorithm to move through unfavorable solutions, particularly while temperature is high.

Crossover

Crossover creates new candidates from multiple solutions, including solutions that may currently be dominated.

These mechanisms are intended to give EMSA broader exploration than MODS.

Computational Complexity

The paper estimates the upper-bound time complexity of EMSA as:

$$O(n^2)$$

The lower bound is expressed as:

$$\Omega(c \times n)$$

Where:

- n represents the dominant problem or operation size,
- and c is treated as a constant.

The derivation counts operations associated with:

- external iterations,
- creating search directions,
- crossover,
- dominance checking,
- and deleting dominated solutions.

This is a broad theoretical estimate rather than a detailed practical runtime model.

Evaluation Metrics

The paper uses several multi-objective performance measures.

The notation distinguishes between:

- PF_{true} = the reference or presumed true Pareto front
- PF_{known} = the Pareto front found by an algorithm

Generation of Non-Dominated Vectors

GNDV measures the total number of non-dominated solutions generated by an algorithm.

In copy-safe form:

$$\text{GNDV} = \text{number of solutions in PFknown}$$

A larger value indicates that the algorithm found more non-dominated solutions.

However, a larger set is not automatically better unless the solutions are accurate and well distributed.

Rate of Generation of Non-Dominated Vectors

RGNDV measures the size of the generated front relative to the reference front.

In simplified form:

$$\text{RGNDV} = \text{GNDV} / \text{number of solutions in PFtrue}$$

The table reports this as either a ratio or normalized measure.

A result of 1 indicates that the generated front matches the reference-front size under the paper's construction.

Real Generation of Non-Dominated Vectors

ReGNDV measures how many generated solutions also belong to the reference Pareto front.

In copy-safe form:

$$\text{ReGNDV} = \text{number of solutions shared by PFknown and PFtrue}$$

A larger value indicates that more reported solutions are treated as true Pareto solutions.

Real-Solution Percentage

The paper also reports:

$$\text{Real-solution percentage} = \text{ReGNDV} / \text{GNDV} \times 100\%$$

This estimates the percentage of an algorithm's generated non-dominated solutions that belong to the reference front.

Generational Distance

Generational Distance, or GD, measures how far the generated solutions are from the reference Pareto front.

A simplified copy-safe interpretation is:

GD = average distance from each generated solution to the nearest reference-front solution

Lower values are better.

A value near zero indicates that the generated front is close to the reference front.

Inverse Generational Distance

Inverse Generational Distance, or IGD, measures the reverse relationship.

A simplified interpretation is:

IGD = average distance from each reference-front solution to the nearest generated solution

Lower values are better.

GD emphasizes the accuracy of generated solutions.

IGD also reflects how completely the generated set covers the reference Pareto front.

Spacing

Spacing measures how evenly the solutions are distributed along the generated Pareto front.

A simplified interpretation is:

Spacing = variation in distances between neighboring generated solutions

Lower spacing values indicate a more uniformly distributed front.

A value near zero means neighboring points are spaced relatively evenly.

Error Rate

The paper defines error rate as:

Error rate = number of reference Pareto solutions not found / number of reference Pareto solutions $\times 100\%$

The equation is presented using the ratio between the reference-front size and the number of real non-dominated solutions recovered.

Lower values are better.

The best possible reported value is:

0% error

Experimental Problem

EMSA is tested on the **Bi-Objective Traveling Salesman Problem**, or BTSP.

Unlike the ordinary TSP, each route is evaluated using two cost objectives.

For example, the two matrices may represent two different route-cost structures.

The algorithm must find trade-off tours rather than only one shortest tour.

Benchmark Instances

Ten TSPLIB-based 100-city instances are used:

1. KROAB100
2. KROAC100
3. KROAD100
4. KROAE100

5. KROBC100
6. KROBD100
7. KROBE100
8. KROCD100
9. KROCE100
10. KRODE100

Each instance combines two known 100-city cost structures into one bi-objective problem.

Hardware and Runs

The experiments are performed using:

- a dual-core computer,
- and 2 GB of RAM.

Each algorithm is run:

10 times per instance

The best non-dominated solutions from those runs are retained for comparison.

Algorithm Parameters

The settings are:

Algorithm	Iterations	Perturbations	Initial temperature	Cooling factor
MODS	100	80	Not applicable	Not applicable
EMSA	100	Not specified	1,000	0.95

Both algorithms therefore use 100 main iterations, although their internal operations differ.

Construction of the Reference Pareto Front

An important methodological detail is that the paper does not use an independently proven exact Pareto front.

Instead, the reference optimal solutions are constructed from:

- the best non-dominated solutions found by MODS,
- and the best non-dominated solutions found by EMSA.

In simplified form:

Reference front = best combined solutions produced by MODS and EMSA

This strongly affects the interpretation of the reported metrics.

Because EMSA contributes most or all of the solutions used to construct the reference front, its reported perfect recovery does not independently prove that it found the true mathematical Pareto front.

Pareto-Front Graphs

The figure on page 6 compares the Pareto fronts generated by MODS and EMSA for nine of the BTSP instances.

The two axes represent:

- Objective 1,
- and Objective 2.

The EMSA points form long, dense trade-off curves.

The MODS results contain comparatively few points and are often sparse relative to the EMSA front.

Visually, EMSA provides:

- much greater front coverage,
- many more candidate trade-offs,
- and a smoother approximation of the apparent Pareto boundary.

KROAB100 Results

For KROAB100:

Metric	MODS	EMSA
GNDV	289	8,479
RGNDV	0.034	1
ReGNDV	0	8,479
Real-solution percentage	0%	100%
Spacing	0.019	0.001
GD	14.945	0.019
IGD	2,200.00 6	3.175
Error rate	100%	0%

EMSA generates approximately 29 times as many non-dominated vectors as MODS.

It also produces much lower spacing, GD, and IGD values.

KROAC100 Results

For KROAC100:

Metric	MODS	EMSA
GNDV	217	7,023
ReGNDV	0	7,023
Real-solution percentage	0%	100%
Spacing	0.034	0.001
GD	19.120	0.028
IGD	2,451.13 2	5.484
Error rate	100%	0%

The same general performance pattern is reported.

KROAD100 Results

For KROAD100:

Metric	MODS	EMSA
--------	------	------

GNDV	281	6,289
------	-----	-------

ReGNDV	0	6,289
--------	---	-------

Spacing	0.014	0.002
---------	-------	-------

GD	11.997	0.032
----	--------	-------

IGD	1,807.158	6.426
-----	-----------	-------

Error rate	100%	0%
------------	------	----

KROAE100 Results

For KROAE100:

Metric	MODS	EMSA
--------	------	------

GNDV	283	6,440
------	-----	-------

ReGNDV	0	6,440
--------	---	-------

Spacing	0.053	0.001
---------	-------	-------

GD	13.251	0.028
----	--------	-------

IGD	2,183.78 4	5.019
-----	---------------	-------

Error rate	100%	0%
------------	------	----

KROBC100 Results

For KROBC100:

Metric	MODS	EMSA
GNDV	298	6,919
ReGNDV	0	6,919
Spacing	0.016	0.001
GD	13.777	0.034
IGD	2,436.03 3	7.992

Error rate	100%	0%
------------	------	----

KROBD100 Results

For KROBD100:

Metric	MODS	EMSA
GNDV	241	5,934
ReGNDV	0	5,934
Spacing	0.024	0.001
GD	14.607	0.037
IGD	2,088.49 0	8.160
Error rate	100%	0%

KROBE100 Results

For KROBE100:

Metric	MODS	EMSA
GNDV	280	5,802
ReGNDV	0	5,802
Spacing	0.031	0.002

GD	13.395	0.037
----	--------	-------

IGD	2,424.67 2	7.848
-----	---------------	-------

Error rate	100%	0%
------------	------	----

KROCD100 Results

For KROCD100:

Metric	MODS	EMSA
--------	------	------

GNDV	286	6,301
------	-----	-------

ReGNDV	0	6,301
--------	---	-------

Spacing	0.018	0.001
---------	-------	-------

GD	11.887	0.029
----	--------	-------

IGD	1,834.38 8	5.229
-----	---------------	-------

Error rate	100%	0%
------------	------	----

KROCE100 Results

For KROCE100:

Metric	MODS	EMSA
GNDV	224	4,613
ReGNDV	1	4,613
Real-solution percentage	0.4%	100%
Spacing	0.028	0.003
GD	12.639	0.002
IGD	1,737.24 7	0.014
Error rate	99%	0%

This is the only instance where MODS recovers one solution that belongs to the constructed reference front.

KRODE100 Results

For KRODE100:

Metric	MODS	EMSA
GNDV	228	7,745
ReGNDV	0	7,745
Spacing	0.048	0.001
GD	16.010	0.026
IGD	1,720.44 9	5.227
Error rate	100%	0%

Overall Experimental Pattern

Across all ten instances, EMSA reports:

- thousands of non-dominated solutions,
- 100% of generated solutions counted as reference solutions,
- error rate of 0%,
- spacing near zero,
- very low GD,
- and much lower IGD than MODS.

MODS reports:

- only a few hundred non-dominated vectors,
- almost no recovered reference solutions,
- and error rates of 99% or 100%.

The largest reported EMSA front contains:

8,479 solutions

The smallest contains:

4,613 solutions

By comparison, MODS generates between:

217 and 298 solutions

Claimed Equal Computational Cost

The authors state that EMSA performs better than MODS with the same computational cost.

Both algorithms use 100 main iterations.

However, the paper does not provide:

- actual runtime measurements,
- counts of objective-function evaluations,
- crossover counts,
- mutation counts,
- or detailed operation totals from the experiments.

The equality of computational cost is therefore asserted primarily through the common iteration setting rather than demonstrated through measured runtime.

Main Findings

The paper's principal findings are:

- EMSA combines Simulated Annealing, Genetic Algorithms, and deterministic swapping.
- It extends MODS by adding random objective weighting.
- Random weights create different search directions across the multi-objective space.
- The annealing rule allows dominated solutions to be explored.
- Crossover creates new candidates from both dominated and non-dominated solutions.
- EMSA returns a set of Pareto trade-offs rather than one route.
- The method is tested on ten bi-objective, 100-city TSP instances.
- EMSA generates far more non-dominated solutions than MODS.
- Its Pareto fronts are visibly denser and broader.
- It reports much lower spacing, GD, and IGD values.
- EMSA obtains a reported 0% error rate on every instance.

- MODS obtains between 99% and 100% error under the paper's reference-front construction.
- The authors conclude that EMSA is consistently superior to MODS.

Important Limitations

Reference Front Is Not Independently Known

The most important limitation is that the reference Pareto front is constructed from the best solutions found by the same two algorithms being compared.

Because EMSA produces far more non-dominated solutions, the reference front largely reflects EMSA's own output.

Therefore, the reported results:

`EMSA error = 0%`

and:

`EMSA real-solution recovery = 100%`

do not independently prove that EMSA found the true Pareto front.

They show that EMSA recovered the combined reference set that it substantially helped create.

Comparison with Only One Algorithm

EMSA is compared only with MODS, its predecessor.

It is not compared with established multi-objective methods such as:

- NSGA-II,
- SPEA2,
- multi-objective PSO,
- Pareto Simulated Annealing,
- multi-objective Ant Colony Optimization,
- or exact multi-objective TSP methods.

Unequal Internal Work Is Possible

Both methods use 100 iterations, but EMSA additionally performs:

- weight generation,
- probabilistic acceptance,
- mutation,
- crossover,
- and dominance-set maintenance.

MODS uses 80 perturbations per iteration, while the number of EMSA internal perturbations is not clearly reported.

Iteration count alone does not establish equal computational effort.

No Runtime Table

The paper does not report:

- CPU time,
- wall-clock time,
- objective evaluations,
- memory consumption,
- or scalability trends.

Best-of-Ten Aggregation

Each algorithm is run ten times, but the comparison uses the best non-dominated solutions accumulated from those runs.

The paper does not report:

- average performance,
- variance,
- worst runs,
- median performance,
- or statistical significance.

The reported results therefore reflect best-case accumulated performance rather than typical single-run behavior.

Incomplete Algorithm Details

Several implementation details are insufficiently explained, including:

- how initial tours are constructed,
- the exact mutation or permutation operator,
- the crossover mechanism,

- the crossover position rule for TSP,
- archive size limits,
- duplicate-solution handling,
- and the precise stopping condition when temperature approaches zero.

Objective Meaning Is Not Explained

The benchmark combines two TSP cost structures, but the paper does not deeply explain the practical meaning of the two objectives.

The instances appear to be mathematical benchmark combinations rather than real logistics problems with clearly defined competing criteria.

Extremely Large Non-Dominated Sets

EMSA returns between approximately 4,600 and 8,500 solutions for 100-city instances.

The paper does not discuss:

- whether duplicate objective vectors are removed,
- whether all tours are structurally unique,
- how practical decision-makers would inspect thousands of alternatives,
- or whether archive size affects runtime.

Complexity Derivation Is Broad

The reported complexity of $O(n^2)$ is derived using simplified operation counts.

It does not fully account for:

- TSP route evaluation,
- number of maintained solutions,
- repeated dominance comparisons,
- archive growth,
- or the cost of crossover and mutation across large permutations.

Wording of Simulated Annealing Behavior

The paper states that when temperature falls, Simulated Annealing behaves similarly to Tabu Search.

This is an informal comparison.

Standard SA and TS remain fundamentally different because:

- SA uses temperature-dependent probabilistic acceptance,
- while TS uses adaptive memory and tabu restrictions.

Conclusion

The paper concludes that EMSA improves the MODS metaheuristic for multi-objective combinatorial optimization.

Its central innovation is the repeated creation of random weighted search directions:

$\text{Weighted objective} = \sum w_i \times f_i(X)$

These changing directions help the algorithm explore different portions of the Pareto front.

The method also uses two mechanisms to avoid local optima:

- probabilistic exploration of dominated solutions,
- and crossover between candidate solutions.

On the ten tested bi-objective TSP instances, EMSA produces substantially larger and more closely distributed Pareto fronts than MODS according to the paper's chosen metrics.

The results strongly favor EMSA within the study's evaluation procedure. However, the reported perfect recovery should be interpreted cautiously because the reference Pareto fronts were constructed using the outputs of EMSA and MODS rather than an independently established exact solution set.

Feature Selection for Learning-to-Rank Using Simulated Annealing

Mustafa Wasif Allvi, Mahamudul Hasan, Lazim Rayan, Mohammad Shahabuddin, Md. Mosaddek Khan, and Muhammad Ibrahim

Summary

This paper investigates the use of **Simulated Annealing** for selecting useful features in **learning-to-rank systems**.

Learning-to-rank combines machine learning with information retrieval. Given a user query, the system assigns relevance scores to documents and returns them in ranked order, ideally placing the most relevant documents near the top.

Such systems may use many features, including scores produced by information-retrieval functions such as:

- TF-IDF,
- BM25,
- query-document matching measures,
- document statistics,
- and link or content characteristics.

The authors argue that using every available feature is not always beneficial. Some features may be:

- irrelevant,
- redundant,
- misleading,
- or computationally expensive.

The proposed method uses Simulated Annealing to search for feature subsets that maintain or improve ranking performance while using fewer features.

The framework is tested on six established learning-to-rank datasets using the LambdaMART ranking algorithm. Ranking quality is measured using Normalized Discounted Cumulative Gain, or NDCG.

The results show that carefully selected subsets often perform as well as or better than the complete feature set. Simulated Annealing also generally outperforms random feature selection.

Information Retrieval

Information retrieval concerns finding useful information from a large collection of documents.

A typical retrieval process is:

1. A user submits a query.
2. The system retrieves documents associated with the query.
3. The system estimates the relevance of each document.
4. The documents are ranked.
5. The most relevant documents are placed near the top.

Traditional information-retrieval systems often use manually designed scoring functions.

Learning-to-rank instead learns a ranking function from labeled training data.

Learning-to-Rank

In a learning-to-rank problem, the training examples are usually query-document pairs.

Each pair contains:

- a query,
- a candidate document,
- a set of numerical features,
- and a human-assigned relevance label.

The model learns a function that predicts a relevance score for unseen query-document pairs.

A simplified representation is:

`Predicted relevance score = ranking model(query-document features)`

Documents are then sorted according to the predicted score.

Learning-to-Rank Workflow

The diagram on page 2 shows a typical learning-to-rank information-retrieval system.

The training process includes:

1. A query collection

2. A document collection
3. Top-k retrieval
4. Document labeling
5. Feature extraction
6. Creation of a training set
7. Learning the ranking function

During use:

1. A user provides a new query.
2. The system retrieves candidate documents.
3. Features are extracted.
4. The learned ranking function is applied.
5. A ranked document list is returned.

Feature quality therefore directly affects the ranking function.

Importance of Feature Selection

The paper explains that modern datasets may contain increasingly large numbers of variables.

Using more features may provide additional information, but it can also create problems.

Possible disadvantages include:

- slower model training,
- increased computational cost,
- reduced model interpretability,
- irrelevant information,
- misleading signals,
- increased noise,
- and lower predictive performance.

Feature selection attempts to retain only the useful variables.

Its goals include:

- improving predictive accuracy,
- reducing dimensionality,
- reducing training time,
- simplifying the learned model,
- improving data understanding,
- and eliminating unnecessary information.

The paper describes feature selection as an NP-hard problem because the number of possible subsets grows exponentially.

For n features:

Number of possible subsets = 2^n

Traditional Feature-Selection Methods

The literature review discusses several conventional approaches.

Filter Methods

Filter methods evaluate features independently of the final learning algorithm.

They may use:

- statistical scores,
- correlations,
- entropy,
- mutual information,
- or ranking measures.

They are usually computationally efficient but may overlook interactions among features.

Wrapper Methods

Wrapper methods evaluate a feature subset by training a model using that subset.

The subset is judged according to the resulting predictive performance.

Wrappers can account for interactions among features, but they are computationally expensive.

Embedded Methods

Embedded approaches perform feature selection while the model is being trained.

Forward and Backward Selection

Forward selection begins with a small or empty feature set and gradually adds features.

Backward elimination begins with all features and gradually removes them.

The paper notes that no one traditional wrapper method works best for every dataset and classifier.

Metaheuristic Feature Selection

Metaheuristic algorithms search large solution spaces using controlled randomization.

Unlike purely greedy algorithms, they may temporarily accept an apparently inferior choice to escape a local optimum.

The paper reviews feature-selection studies using:

- Grey Wolf Optimization,
- Chaotic Crow Search,
- Grasshopper Optimization,
- Genetic Algorithms,
- Whale Optimization,
- and Simulated Annealing.

Previous research had already applied Simulated Annealing to conventional supervised feature selection.

However, the authors state that its use specifically within a learning-to-rank framework had not been sufficiently investigated.

Simulated Annealing

The paper describes Simulated Annealing as a combination of:

- hill climbing,
- and random walk.

Hill Climbing

Hill climbing repeatedly chooses a better neighboring solution.

Its basic behavior is:

1. Start from one solution.
2. Evaluate neighboring solutions.

3. Move to a better neighbor.
4. Repeat until no better neighbor exists.

The problem is that it may stop at a local optimum.

Random Walk

A random walk moves from one state to a randomly chosen neighbor.

This encourages exploration but may wander without making consistent progress.

Simulated Annealing Combination

Simulated Annealing combines directed improvement with controlled randomness.

It normally accepts a better neighboring state.

A worse neighbor may also be accepted, particularly early in the search.

The probability decreases when:

- the neighbor is much worse,
- or the algorithm is at a later stage.

Acceptance Probability

The paper defines the change in state quality as:

$\Delta E = \text{neighbor quality} - \text{current quality}$

The probability of accepting a worse neighbor is represented as:

$\text{Acceptance probability} = \exp(\Delta E / T)$

Where:

- T is the temperature,
- and temperature decreases over time.

Because the paper treats larger quality values as better, a worse neighbor produces a negative ΔE . This makes the probability fall between zero and one.

A more negative quality difference produces a lower probability.

Temperature Behavior

At high temperature:

- worse solutions are more likely to be accepted,
- exploration is broader,
- and the search can leave local optima.

At low temperature:

- worse solutions are rarely accepted,
- the method behaves more like hill climbing,
- and the search concentrates on refinement.

The paper notes that this wider exploration can require substantial computation time.

Proposed Feature-Selection Framework

The proposed method defines three important components.

State

A state represents a subset of features.

For example, when the dataset contains 46 features and the current subset size is 10, one state contains exactly 10 selected feature indices.

Neighborhood

A neighboring state is created by randomly altering some feature indices in the current subset.

The paper does not provide a highly detailed formal neighborhood operator. It states that selected feature indices are changed randomly.

State Quality

The quality of a feature subset is measured using NDCG after training and testing a learning-to-rank model using only that subset.

A higher NDCG value means the subset produces better ranking performance.

Fixed Subset Size

The Simulated Annealing procedure receives a subset size called k .

For a selected value of k :

1. Randomly choose k features.
2. Treat them as the initial state.
3. Train the ranking model using only those features.
4. Evaluate the model using NDCG.
5. Generate a neighboring subset containing the same number of features.
6. Train and evaluate the neighboring subset.
7. Decide whether to accept it using Simulated Annealing.
8. Repeat for 100 iterations.
9. Retain the best subset found for that value of k .

The complete process is then repeated for every possible subset size.

Search Across All Subset Sizes

The authors begin with:

$k = 1$

They increase k one feature at a time until reaching the total number of available features.

For every k , they compare:

- a random subset of k features,
- and the subset found using Simulated Annealing.

The final recommended subset is the one that achieves the highest NDCG value across all tested subset sizes and SA iterations.

Important Characteristic of the Objective

The algorithm does not directly minimize feature count inside one combined objective function.

Instead:

- subset size is fixed during each individual run,

- ranking performance is optimized for that size,
- and the results across different values of k are compared afterward.

Therefore, the framework searches for the best-performing subset at each possible dimensionality.

Ranking Algorithm

The experiments use **LambdaMART** through the RankLib implementation.

LambdaMART combines:

- LambdaRank-style ranking gradients,
- and gradient-boosted decision trees.

The authors select it because prior studies found that tree-ensemble learning-to-rank methods often perform strongly.

Evaluation Metric

The paper uses **Normalized Discounted Cumulative Gain**, or NDCG.

NDCG measures ranking quality by assigning greater importance to relevant documents placed near the top of the ranked list.

A high NDCG score means that:

- highly relevant documents are ranked early,
- and less relevant documents appear lower.

The maximum possible NDCG value is:

1

The paper uses NDCG as both:

- the state-quality function inside Simulated Annealing,
- and the final evaluation metric.

Datasets

The experiments use six publicly available learning-to-rank datasets:

1. MQ2007
2. MQ2008
3. Ohsumed
4. NP2004
5. HP2004
6. TD2004

The datasets contain predefined:

- training sets,
- validation sets,
- and testing sets.

The authors preserve these official divisions.

Dataset Feature Counts

Dataset	Available features
MQ2007	46
MQ2008	46
Ohsumed	45
NP2004	64
HP2004	64

Experimental Procedure

For every dataset and every possible k value:

1. Select k features randomly.
2. Train LambdaMART using those random features.
3. Evaluate the model on the test data using NDCG.
4. Run Simulated Annealing for 100 iterations to identify a better subset containing k features.
5. Train LambdaMART using the SA-selected features.
6. Evaluate it using NDCG.
7. Plot both results.

The horizontal axis in the graphs represents:

Number of selected features

The vertical axis represents:

NDCG

The graphs compare:

- random feature selection,
- and Simulated Annealing feature selection.

MQ2007 Results

MQ2007 contains 46 features.

The graph on page 5 shows considerable variation when only a small number of features are used.

The difference between random selection and Simulated Annealing becomes less unstable as more features are included.

The authors report that the best ranking accuracy is obtained using approximately:

26 features

This is fewer than the complete set of 46.

The graph suggests that using all available features is unnecessary for achieving the best observed result.

MQ2008 Results

MQ2008 also contains 46 features.

The Simulated Annealing curve generally performs strongly from the beginning.

The authors report that using approximately:

20 features

can achieve accuracy similar to the model using all 46 features.

This corresponds to using less than half of the original variables.

The SA-selected subsets generally perform more consistently than random subsets.

Ohsumed Results

Ohsumed contains 45 features.

Both curves fluctuate across much of the feature range.

The difference between random selection and SA is especially visible for smaller subsets.

After approximately:

18 features

the gap between the methods becomes smaller.

The authors conclude that approximately 18 features may provide a useful efficiency-performance trade-off instead of using all 45.

The result emphasizes reduced training cost rather than a dramatic improvement in maximum NDCG.

NP2004 Results

NP2004 contains 64 features.

From approximately:

$k = 8$

the SA-selected feature subsets almost always outperform the randomly selected subsets.

The paper reports that a subset containing approximately:

39 features

outperforms the use of all 64 features.

The graph on page 5 shows substantial fluctuation, but the SA curve is generally above the random-selection curve.

HP2004 Results

HP2004 contains 64 features.

The graph on page 6 shows a pattern similar to NP2004.

From approximately:

$k = 8$

the Simulated Annealing subsets generally outperform random selection.

The authors state that approximately:

22 selected features

can outperform a configuration using 53 features.

The result indicates that much of the full feature set may be unnecessary or unhelpful.

TD2004 Results

TD2004 also contains 64 features.

The SA approach generally begins outperforming random selection after approximately:

8 selected features

The paper reports that approximately:

23 features

can provide ranking accuracy equivalent to using the complete 64-feature set.

The graph on page 6 shows that the SA curve frequently remains above the random-selection curve, particularly at larger subset sizes.

Results Summary

The approximate useful subset sizes identified by the authors are:

Dataset	Original features	Reported useful SA subset
MQ2007	46	About 26
MQ2008	46	About 20
Ohsumed	45	About 18
NP2004	64	About 39
HP2004	64	About 22
TD2004	64	About 23

These values are based on the interpretation of the plotted NDCG curves rather than a detailed numerical result table.

Main Experimental Findings

The authors draw several conclusions from the graphs.

Smaller Subsets Can Perform Well

For all six datasets, subsets containing fewer features can perform comparably to or better than models using all available features.

This suggests that simply including every available feature does not necessarily improve learning-to-rank accuracy.

Simulated Annealing Generally Beats Random Selection

Across the six datasets, the SA-selected subsets generally produce higher NDCG values than randomly selected subsets of the same size.

The advantage is particularly visible for:

- MQ2008,
- NP2004,
- HP2004,
- and TD2004.

Greater Benefit at Moderate Subset Sizes

When only one or a few features are used, results fluctuate heavily because each individual choice has a major effect.

As more features are selected, the curves generally become more stable.

Reduced Training Time

Using fewer features reduces the amount of data processed during model training.

The paper argues that SA-based feature selection may therefore improve both:

- ranking performance,

- and training efficiency.

Graphical Results

The plots on pages 5 and 6 show noticeable fluctuation in both the random and SA curves.

The SA curves are usually above the random-selection curves but do not improve monotonically as the number of features increases.

This means that:

- adding a feature may improve ranking,
- have almost no effect,
- or reduce ranking performance.

The graphs support the paper's argument that the identity of the selected features matters more than simply the number of features.

Randomness and Result Fluctuation

The authors acknowledge that Simulated Annealing is stochastic.

Different runs may therefore produce different subsets and NDCG scores.

The paper suggests that averaging the results of several runs would reduce the visible fluctuations.

However, the reported experiment appears to rely largely on one SA outcome for each subset size rather than presenting averages across repeated independent runs.

Main Findings

The paper's principal findings are:

- Learning-to-rank performance depends strongly on feature quality.
- Using all available features is not always optimal.
- Feature selection can improve model accuracy and reduce training time.
- Simulated Annealing can represent each feature subset as a search state.
- Random changes to feature indices can generate neighboring states.
- NDCG provides a task-specific state-quality measure.
- Simulated Annealing generally selects stronger subsets than random selection.

- All six datasets can use fewer features without major performance loss.
- Several datasets achieve better NDCG with reduced subsets than with the full feature set.
- The strongest reductions are reported for MQ2008, Ohsumed, HP2004, and TD2004.
- The ranking curves fluctuate because of the stochastic nature of the method.
- The authors believe more iterations and repeated runs could produce more reliable results.

Important Limitations

Only Random Selection Is Used as the Experimental Baseline

The proposed method is directly compared only with random feature selection.

It is not experimentally compared with:

- filter methods,
- forward selection,
- backward elimination,
- Genetic Algorithms,
- Particle Swarm Optimization,
- Grey Wolf Optimization,
- Whale Optimization,
- or other Simulated Annealing variants.

Therefore, the results show that SA is generally better than random selection, but not that it is superior to established feature-selection methods.

No Repeated-Run Statistics

Simulated Annealing is stochastic, but the paper does not report:

- average NDCG over multiple runs,
- standard deviation,
- confidence intervals,
- statistical tests,
- best and worst runs,
- or success rates.

The authors themselves state that averaging several runs would reduce fluctuation.

No Detailed Parameter Settings

The paper does not clearly report several critical SA settings, including:

- starting temperature,
- cooling schedule,
- final temperature,
- cooling coefficient,
- exact acceptance implementation,
- number of features changed per neighbor,
- and random seed.

This makes exact replication difficult.

Limited Neighborhood Definition

The neighborhood is described only as randomly altering feature indices.

The paper does not specify:

- whether one feature is replaced per move,
- whether multiple features are changed,
- whether swaps are used,
- or how duplicate indices are prevented.

Fixed Number of Features per Run

Each run optimizes ranking performance for a fixed subset size k .

Feature count is not included directly in the objective.

This requires separate runs for every possible subset size.

For a dataset with D features and 100 iterations per size, the total search process involves approximately:

$100 \times D$ SA iterations

This excludes the initial and random-baseline model evaluations.

Computational Cost

Every state evaluation requires training and testing LambdaMART.

The authors claim that the final subset reduces training time, but they do not provide a measured runtime table showing:

- feature-selection time,
- training time before selection,
- training time after selection,
- or total end-to-end computational cost.

The feature-selection process itself may be expensive because many ranking models are trained.

Test Set Used During Feature Search

The methodology states that each candidate model is evaluated on test data and that NDCG is used as the state quality.

This means the test set appears to influence feature selection.

Using the test set repeatedly during optimization can cause test-set overfitting and may make the reported performance optimistic.

A stronger experimental design would normally use:

- training data for fitting,
- validation data for feature selection,
- and test data only for final evaluation.

Validation Set Is Not Clearly Used

Although the datasets provide predefined validation sets, the paper does not clearly explain how those validation sets are incorporated into the SA procedure.

One Learning-to-Rank Model

Only LambdaMART is evaluated.

The selected feature subsets may behave differently with:

- RankNet,
- LambdaRank,
- RankBoost,
- coordinate ascent,
- random-forest ranking,
- linear ranking models,

- or support-vector ranking.

Small Feature Pools

The datasets contain only 45, 46, or 64 features.

The method is not tested on learning-to-rank datasets containing:

- hundreds,
- thousands,
- or millions of features.

No Feature Interpretation

The authors do not analyze:

- which features are repeatedly selected,
- whether selected features are redundant,
- why particular features matter,
- or whether the same features are chosen across runs.

Results Mainly Interpreted Visually

The paper does not provide a consolidated table containing:

- exact best NDCG values,
- exact selected features,
- full-feature NDCG,
- NDCG improvements,
- or measured training-time reductions.

Many conclusions are drawn from visual inspection of the curves.

Possible Acceptance-Formula Ambiguity

The paper writes the acceptance probability as:

$\exp(\Delta E / T)$

with:

$\Delta E = \text{neighbor quality} - \text{current quality}$

This works when lower-quality neighbors produce negative values.

However, the explanation could be clearer because many SA formulations define cost differences using the opposite sign.

Future Work

The authors propose three main extensions.

Additional Learning-to-Rank Algorithms

The framework should be tested with other ranking algorithms rather than only LambdaMART.

Larger Datasets

The method should be evaluated on larger learning-to-rank datasets.

Other Metaheuristics

Future studies should compare classical Simulated Annealing with:

- newer SA variants,
- and other contemporary metaheuristic optimization algorithms.

The paper also mentions the need for:

- more iterations,
- repeated-run averaging,
- and examination of the actual features selected.

Conclusion

The paper concludes that Simulated Annealing can be used to identify effective feature subsets for learning-to-rank systems.

The proposed framework treats:

- a feature subset as a state,
- random feature replacement as a neighborhood move,
- and NDCG as the state-quality function.

The method evaluates every possible subset size and recommends the subset producing the strongest ranking performance.

Across six benchmark datasets, the authors find that:

- fewer features often provide performance comparable to or better than the complete feature set,
- SA generally produces stronger subsets than random selection,
- and feature reduction may decrease ranking-model training time.

The paper presents the approach as an initial demonstration that Simulated Annealing is applicable to learning-to-rank feature selection, while acknowledging the need for larger datasets, additional ranking algorithms, stronger baselines, more iterations, and repeated experimental runs.

Particle Swarm Optimization

James Kennedy and Russell Eberhart

Summary

This paper introduces the original concept of **Particle Swarm Optimization**, or **PSO**, as a method for optimizing continuous nonlinear functions.

The method emerged from simulations of social behavior, particularly:

- bird flocking,
- fish schooling,
- collective movement,
- and the sharing of useful information within groups.

Each potential solution is represented by a **particle** moving through a multidimensional search space. A particle adjusts its movement using two main sources of information:

1. The best position it has personally discovered
2. The best position discovered by the entire swarm

The authors refer to these as:

- **pbest** = personal best
- **gbest** = global best

The paper explains how the algorithm developed from a bird-flocking simulation into a simplified optimization method. It also reports early experiments involving:

- nonlinear benchmark functions,
- neural-network training,
- the XOR problem,
- the Fisher Iris dataset,
- and electroencephalogram waveform classification.

The authors conclude that PSO is:

- simple,
- computationally inexpensive,
- easy to implement,
- memory-efficient,
- and effective on several difficult nonlinear optimization tasks.

Main Objective of the Paper

The main purpose of the paper is to introduce a new population-based optimization method for continuous nonlinear functions.

The authors aim to explain:

- where the method came from,
- how it developed,
- how particles move,
- why personal and social information are important,
- how the method differs from related evolutionary algorithms,
- and how well it performs in early experiments.

The paper is primarily foundational. It does not present the mature form of PSO commonly used today with inertia weights and constriction factors.

Instead, it presents an early PSO formulation based on:

- current velocity,
- attraction toward personal best,
- attraction toward global best,
- and random multipliers.

Origins in Artificial Life

The authors describe PSO as having roots in two main research areas.

Artificial Life and Swarm Behavior

The first influence comes from simulations of:

- birds,
- fish,
- herds,
- schools,
- and other socially interacting organisms.

These simulations attempted to explain how complex group behavior could emerge from simple rules followed by individual agents.

Evolutionary Computation

The second influence comes from:

- Genetic Algorithms,
- evolutionary programming,
- fitness-based search,
- and stochastic optimization.

PSO shares the evolutionary idea of maintaining several candidate solutions and evaluating them according to fitness.

However, it does not use conventional genetic operators such as:

- chromosome selection,
- crossover,
- and mutation.

Instead, particles move through the search space according to velocity adjustments.

Social Information-Sharing Principle

A central idea in the paper is that members of a social group can benefit from discoveries made by other members.

For example, in a flock searching for food:

- one bird may discover a promising location,
- information about that location influences the behavior of the rest of the flock,
- and the group can move toward that promising area.

The authors extend this idea to optimization.

Every particle has its own experience, but it also has access to information discovered by the swarm.

The algorithm therefore combines:

- individual learning,
- and social learning.

Human Social Behavior

The authors do not limit their analogy to physical animal movement.

They also compare swarm behavior with human social behavior.

Humans adjust:

- beliefs,
- attitudes,
- decisions,
- and behavior

according to both personal experience and information obtained from other people.

The authors describe the search space as an abstract multidimensional space rather than only a physical space.

A particle's movement may therefore represent a change in:

- neural-network weights,
- parameter values,
- decisions,
- or other numerical variables.

Early Bird-Flocking Simulation

The original simulation represented agents as birds moving across a two-dimensional grid.

Each agent had:

- an X position,
- a Y position,
- an X velocity,
- and a Y velocity.

The simulation initially used two mechanisms:

1. Nearest-neighbor velocity matching
2. A random mechanism called "craziness"

Nearest-Neighbor Velocity Matching

At every iteration, each agent identified its nearest neighbor.

The agent then adjusted its movement to match that neighbor's velocity.

This produced synchronized group movement.

However, the flock quickly settled into one unchanging direction.

Craziness Variable

To prevent the flock from becoming completely uniform, the authors introduced a random variable called **craziness**.

This variable occasionally altered an agent's velocity.

It made the movement appear:

- less predictable,
- more varied,
- and more lifelike.

However, later experiments showed that craziness was not necessary for optimization.

It was eventually removed from the algorithm.

Cornfield Experiment

The transformation from a flocking simulation to an optimizer occurred through an experiment involving a simulated cornfield.

The goal was placed at the coordinate:

`Target position = (100, 100)`

Every agent evaluated its current position according to its distance from this target.

The evaluation function can be written in copy-safe form as:

`Evaluation = square root of [(current X - 100)^2 + (current Y - 100)^2]`

The best possible value was:

`Evaluation = 0`

This occurred when an agent reached:

(100, 100)

Personal Best

Every agent remembered the best position it had personally discovered.

This was called:

`pbest`

For a two-dimensional problem, the stored coordinates were:

- personal-best X position,
- personal-best Y position.

Whenever an agent found a position with better fitness, its personal-best record was updated.

Conceptually, the authors compare personal best with:

- autobiographical memory,
- individual experience,
- and nostalgia.

The particle tends to return toward a position that previously produced a strong result.

Global Best

Every particle also had access to the best position discovered by any member of the swarm.

This was called:

`gbest`

The global-best position was shared across the population.

Conceptually, the authors compare it with:

- public knowledge,
- a social standard,
- a group norm,
- or a successful discovery communicated to everyone.

Early Velocity Adjustment

In the early cornfield model, a particle's velocity was adjusted according to whether it was:

- above or below the personal-best position,
- to the left or right of it,
- above or below the global-best position,
- and to the left or right of it.

A random amount was then added or subtracted from the corresponding velocity.

The personal and social adjustment strengths were originally controlled by:

- `p-increment`
- `g-increment`

Effects of Personal and Social Influence

The experiments showed that the relative strength of the two influences mattered.

Excessive Personal Influence

When personal influence was much stronger than social influence:

- particles wandered more independently,
- individuals remained isolated,
- and swarm convergence became weaker.

Excessive Social Influence

When social influence was much stronger:

- particles rushed toward the current global best,
- convergence occurred rapidly,
- and the swarm risked moving prematurely toward a local optimum.

Approximately Equal Influence

The authors observed that approximately equal personal and social adjustment values generally produced the most effective search behavior.

Elimination of Unnecessary Variables

The authors gradually simplified the original flocking model.

They found that optimization still worked after removing:

- craziness,
- nearest-neighbor velocity matching,
- collision avoidance,
- and several other simulation-specific features.

In fact, removing nearest-neighbor velocity matching slightly improved optimization speed.

The resulting population no longer behaved exactly like a bird flock.

The authors therefore began describing it as a **swarm**.

Multidimensional Extension

The original experiment used only two dimensions.

The authors extended the method to a search space containing any number of dimensions.

Instead of storing only X and Y positions, each particle stores:

- one position value for every dimension,
- and one velocity value for every dimension.

If:

- D = number of dimensions
- N = number of particles

then particle positions and velocities can be represented as:

$D \times N$ matrices

This allows PSO to optimize problems containing many numerical variables.

Acceleration According to Distance

The early version adjusted velocity only according to whether the particle was on one side or the other of a target position.

The authors later changed the rule so that acceleration depended on the actual distance between:

- the current position,
- and the best position.

The personal adjustment for one dimension can be written as:

`personal adjustment = random value × personal coefficient × (pbest - current position)`

The social adjustment can be written as:

`social adjustment = random value × social coefficient × (gbest - current position)`

This was more effective and easier to interpret than the earlier directional test.

Simplified PSO Velocity Formula

The final simplified version presented in the paper removes separate personal and global increment parameters.

The velocity update for one particle and one dimension can be written as:

`new velocity = current velocity`
`+ 2 × random value 1 × (personal best - current position)`
`+ 2 × random value 2 × (global best - current position)`

The position is then updated using:

`new position = current position + new velocity`

The two random values are generated independently.

Meaning of the Formula

The formula contains three main components.

Momentum

current velocity

This preserves part of the particle's previous movement.

The particle does not restart its direction from zero at every iteration.

Personal Attraction

$2 \times \text{random value} \times 1 \times (\text{personal best} - \text{current position})$

This pulls the particle toward the best position it has personally discovered.

Social Attraction

$2 \times \text{random value} \times 2 \times (\text{global best} - \text{current position})$

This pulls the particle toward the best position found by the swarm.

Why the Random Multiplier Is Scaled by 2

A random value between 0 and 1 has an average of approximately:

0.5

Multiplying it by 2 produces an average influence of approximately:

1

The authors state that this allows particles to overshoot the target around half the time.

Overshooting contributes to exploration because a particle does not simply stop when it reaches the current personal-best or global-best region.

Importance of Independent Random Terms

The authors tested a version that replaced the two attraction terms with movement toward the midpoint between personal best and global best.

That version tended to converge to the midpoint whether or not the midpoint was optimal.

The paper therefore argues that the two separate stochastic influences are necessary.

They allow the particle to explore the region between and beyond:

- its own best position,
- and the swarm's best position.

Importance of Momentum

The authors also tested a version that removed the current-velocity term.

The resulting update used only personal and social attraction.

That version was ineffective at finding global optima.

The paper concludes that momentum is an important part of PSO.

Momentum allows particles to:

- continue through promising regions,
- overshoot current targets,
- and explore previously unvisited areas.

Explorer and Settler Experiment

The authors tested a population containing two kinds of agents.

Explorers

Explorers used the older inequality-based velocity rule.

They tended to move far beyond targets and explore outside known regions.

Settlers

Settlers used distance-based movement.

They focused more closely on refining promising regions.

The intention was to combine:

- broad exploration,
- and detailed exploitation.

However, the mixed approach did not outperform the simpler PSO formulation.

Swarm Intelligence Principles

The paper relates PSO to five principles of swarm intelligence.

Proximity

The population must be able to perform basic calculations involving space and time.

PSO particles evaluate and modify positions across iterations.

Quality

The population must respond to indicators of environmental quality.

PSO responds to:

- personal-best fitness,
- and global-best fitness.

Diverse Response

The population should not follow one excessively narrow behavior.

Random personal and social influences create varied particle movement.

Stability

The swarm should not completely change its behavior after every minor event.

The population's shared direction changes mainly when the global best changes.

Adaptability

The population must be capable of changing behavior when new information justifies the computational cost.

PSO adapts when particles discover improved personal or global positions.

Why the Term “Particle” Is Used

The authors considered several possible terms for population members.

They selected **particle** because the candidates possess:

- positions,
- velocities,
- and accelerations.

Although the particles are abstract and have no meaningful physical size, the word captures their movement through the search space.

The complete population is therefore called a **particle swarm**.

Neural-Network Training

One of the first multidimensional applications involved training feedforward neural-network weights.

A particle represented a complete set of candidate network weights.

The search space contained one dimension for every adjustable weight or bias.

The particle’s fitness was based on the neural network’s prediction error.

XOR Experiment

The authors tested PSO on the exclusive-or, or XOR, problem.

The network architecture contained:

- 2 input processing elements,
- 3 hidden processing elements,
- 1 output processing element.

Including the network biases, the optimization problem contained:

13 parameters

The authors used:

20 particles

The stopping criterion was:

Average sum-squared error per processing element < 0.05

The network reached this criterion in an average of:

30.7 iterations

The authors report that PSO performed very well on this problem.

Fisher Iris Classification

PSO was also used to train a neural network for the Fisher Iris dataset.

Across ten training sessions, the optimizer required an average of:

284 epochs

The paper states that the resulting neural-network training was as effective as ordinary error backpropagation.

Electroencephalogram Classification

The authors describe an experiment involving electroencephalogram spike waveforms and false positives.

A neural network trained using backpropagation achieved:

89% test accuracy

A network trained using PSO achieved:

92% test accuracy

The authors treat this as informal evidence that PSO-trained weights may sometimes generalize better than weights obtained through gradient descent.

They do not present a larger statistical analysis confirming this advantage.

Schaffer f6 Benchmark

PSO is tested on the highly nonlinear Schaffer f6 function.

This benchmark has:

- many local optima,
- a difficult nonlinear surface,
- and strong potential for premature trapping.

The paper reports that the PSO method found the global optimum on every run.

Its number of function evaluations appeared comparable with the performance reported for elementary Genetic Algorithms.

Relationship to Genetic Algorithms

PSO and Genetic Algorithms share several characteristics.

Both:

- maintain a population of candidate solutions,
- evaluate candidates using fitness,
- use stochastic behavior,
- and search without relying on gradient information.

However, they manipulate candidate solutions differently.

Genetic Algorithms

Genetic Algorithms use mechanisms such as:

- selection,
- crossover,
- mutation,
- and generational replacement.

Particle Swarm Optimization

PSO moves existing candidates through the search space.

Particles retain:

- positions,
- velocities,
- personal memories,
- and social information.

The attraction toward personal and global best is described as conceptually similar to recombining information through crossover.

Relationship to Evolutionary Programming

The authors state that PSO lies conceptually between:

- Genetic Algorithms,
- and evolutionary programming.

Like evolutionary programming, PSO depends heavily on stochastic processes.

Like Genetic Algorithms, it uses:

- fitness,
- a population,
- and a form of information combination.

However, its defining characteristic is movement through the search space rather than reproduction.

Exploration and Exploitation

The paper describes the balance between:

- conservative testing of known promising regions,
- and risky exploration of unknown regions.

PSO achieves this balance through:

- attraction toward personal and global best,
- random multipliers,

- retained velocity,
- and overshooting.

Exploitation

Attraction toward strong known positions causes the swarm to refine promising areas.

Exploration

Randomness and momentum cause particles to move:

- between promising regions,
- past known targets,
- and into unexplored areas.

Computational Simplicity

The authors emphasize that PSO can be written using only a small amount of code.

It requires:

- a representation of the problem,
- a fitness function,
- particle positions,
- particle velocities,
- personal-best records,
- a global-best record,
- and a few numerical parameters.

The method uses only basic arithmetic operations.

The authors describe it as inexpensive in terms of:

- memory,
- processing speed,
- and implementation effort.

Main Findings

The paper's principal findings are:

- PSO emerged from simulations of social and flocking behavior.
- Individual and social information can guide numerical optimization.
- Each particle remembers its own best position.
- All particles learn from the best position found by the swarm.
- Personal and social influences should be reasonably balanced.
- Excessive personal influence causes wandering.
- Excessive social influence can cause premature convergence.
- Nearest-neighbor matching and artificial craziness are unnecessary for optimization.
- Movement proportional to distance from strong positions performs better than a simple directional rule.
- Separate personal and global random terms are more effective than movement toward their midpoint.
- Momentum is necessary for effective global search.
- Overshooting contributes to exploration.
- PSO can be extended naturally from two dimensions to many dimensions.
- The method performs well on early neural-network and benchmark applications.
- It is simple, compact, and computationally inexpensive.

Important Limitations

Early Version of PSO

This is the original 1995 formulation.

It does not contain several mechanisms commonly found in later PSO variants, such as:

- explicit inertia weight,
- velocity clamping rules,
- constriction factors,
- local-best topologies,
- adaptive coefficients,
- or diversity-restoration mechanisms.

The current-velocity term functions as momentum, but it is not multiplied by a separately adjustable inertia parameter.

Limited Benchmarking

The experimental evidence is based on a relatively small group of tasks:

- XOR neural-network training,
- Fisher Iris classification,

- electroencephalogram classification,
- and the Schaffer f6 function.

The paper does not use a broad benchmark suite.

Limited Statistical Analysis

The authors provide some average results, but they do not report:

- standard deviations for most experiments,
- confidence intervals,
- formal significance tests,
- detailed distributions,
- or extensive repeated-run comparisons.

Informal Comparison with Genetic Algorithms

The comparison with Genetic Algorithms relies mainly on previously reported results.

The paper does not present a fully controlled experiment using:

- identical hardware,
- identical evaluation budgets,
- and equally tuned implementations.

Informal Generalization Claim

The electroencephalogram example suggests that PSO-trained networks may generalize better than gradient-descent networks.

However, the evidence consists of one reported comparison:

- 92% for PSO,
- 89% for backpropagation.

This is insufficient to establish a general advantage.

Continuous Optimization Focus

The method is presented for continuous nonlinear functions.

The paper does not provide a discrete or binary PSO formulation for problems such as:

- feature selection,
- scheduling,
- routing,
- or combinatorial assignment.

Such problems require later adaptations.

Global-Best Topology

Every particle follows the same global-best solution.

This can spread useful information quickly, but it also risks premature convergence when the global best is located at a local optimum.

No Formal Convergence Proof

The paper does not provide a mathematical proof that the swarm converges to the global optimum.

The conclusions are based mainly on conceptual reasoning and empirical results.

Parameter Selection

The coefficient value of 2 is selected through experimentation.

The authors explicitly state that further research is needed to determine:

- whether 2 is optimal,
- whether it should vary across problems,
- or whether it should evolve during the search.

Ambiguous Stopping Conditions

The paper discusses task-specific criteria but does not establish one general stopping rule for PSO.

Possible stopping conditions include:

- reaching a target fitness,
- completing a maximum number of iterations,
- or observing no meaningful improvement.

Future Research

The authors state that much more research is needed.

Open questions include:

- determining suitable coefficient values,
- adapting parameters to particular problems,
- understanding why the method works,
- expanding benchmark testing,
- and applying PSO to additional scientific and engineering problems.

The authors' design goals are to keep the method:

- simple,
- robust,
- elegant,
- and versatile.

Conclusion

The paper concludes that Particle Swarm Optimization is a simple and promising method for optimizing nonlinear functions.

Its defining idea is that candidate solutions move through a multidimensional search space while accelerating toward:

- their own best-known positions,
- and the swarm's best-known position.

The final velocity update combines:

previous velocity

+ randomized attraction toward personal best

+ randomized attraction toward global best

This produces a balance among:

- individual experience,
- social information,
- exploitation of good regions,

- and exploration through stochastic overshooting.

The early experiments indicate that PSO can:

- optimize nonlinear benchmark functions,
- train neural-network weights,
- locate global optima in multimodal landscapes,
- and achieve performance comparable with established optimization methods.

The authors present PSO as a compact optimization paradigm in which useful collective behavior emerges from simple interactions among particles.

Feature Selection Using Tabu Search Method

Hongbin Zhang and Guangyu Sun

Summary

This paper applies **Tabu Search** to feature selection for pattern-classification systems.

Feature selection attempts to choose a useful subset from a larger collection of available features. The selected subset should preserve or improve classification performance while reducing:

- dimensionality,
- feature-collection cost,
- classifier complexity,
- and computational requirements.

The authors examine two forms of feature selection:

1. Select a fixed number of features that produces the lowest classification error.
2. Select the smallest possible feature subset whose classification error remains below a specified threshold.

Both are combinatorial optimization problems. Because the number of possible feature subsets grows exponentially, exhaustive search becomes impractical when the original feature set is large.

The paper proposes Tabu Search as a method that balances:

- high feature-subset quality,
- reasonable computation cost,
- resistance to local optima,
- and applicability to medium- and large-scale feature-selection problems.

The method is compared with:

- Sequential Forward Selection,
- Sequential Backward Selection,
- generalized sequential methods,
- Plus-I Take Away-r methods,
- Sequential Forward Floating Selection,
- Sequential Backward Floating Selection,

- Genetic Algorithms,
- and Branch and Bound.

The experiments use synthetic feature-selection problems containing:

- 20 features,
- 30 features,
- 60 features,
- and 100 features.

The results indicate that Tabu Search frequently produces optimal or near-optimal feature subsets and often requires fewer objective-function evaluations than Genetic Algorithms, floating search methods, and Branch and Bound.

Importance of Feature Selection

A pattern classifier may initially receive many possible input features.

Using every feature is not always beneficial because:

- collecting features may be expensive,
- training may become slower,
- the available sample size may be too small relative to dimensionality,
- irrelevant or redundant features may reduce practical classifier performance,
- and the classifier may suffer from the curse of dimensionality.

From an ideal Bayesian perspective, removing information cannot improve the perfect Bayes classifier.

However, real classifiers are designed using:

- limited training samples,
- imperfect statistical assumptions,
- noisy data,
- and non-ideal learning procedures.

Under these practical conditions, removing unnecessary features may improve classifier performance.

Two Forms of Feature Selection

The paper distinguishes two optimization formulations.

Fixed-Size Feature Selection

The first form seeks the best subset containing exactly m features.

The objective can be written in copy-safe form as:

Minimize $J(S)$ = classification error of subset S

Subject to:

S is a subset of the full feature set

Number of features in S = m

The subset size is fixed, and the goal is to minimize classification error or maximize another quality measure.

Minimum-Size Feature Selection

The second form seeks the smallest subset whose error remains below a tolerable threshold.

The objective is:

Minimize $J(S)$ = number of features in subset S

Subject to:

Classification error of S < tolerated error threshold

In this form, classification performance becomes a constraint, while dimensionality is the main objective.

Computational Difficulty

For n original features, there are:

2^n possible feature subsets

Both feature-selection formulations are NP-hard.

Exhaustive search can guarantee an optimal solution, but it is practical only for small feature sets.

Branch and Bound can eliminate many candidate subsets, but its performance depends on the objective function satisfying a monotonicity condition.

Even when Branch and Bound eliminates most candidates, its worst-case complexity remains exponential.

The authors therefore focus on suboptimal methods that seek high-quality solutions without evaluating the complete subset space.

Earlier Sequential Methods

Sequential Forward Selection

Sequential Forward Selection, or SFS, begins with no features and repeatedly adds the feature producing the greatest improvement.

Its main weakness is that a feature added earlier cannot later be removed.

Sequential Backward Selection

Sequential Backward Selection, or SBS, begins with the full feature set and repeatedly removes one feature.

A feature removed earlier cannot later be restored.

Nesting Effect

This inability to reverse earlier decisions is called the **nesting effect**.

A locally attractive early decision may prevent the algorithm from reaching the globally best subset.

Generalized Sequential Methods

Generalized forward and backward methods evaluate combinations of several features at once.

They may produce better subsets than basic SFS or SBS.

However, computation cost increases rapidly as the number of simultaneously considered features grows.

The paper notes that the increase can become exponential.

Plus-I Take Away-r Methods

The Plus-I Take Away-r method alternates between:

- adding l features,
- and removing r features.

This partly addresses the nesting effect.

However, there is no clear theoretical rule for selecting suitable values of l and r .

Sequential Floating Search

Sequential Forward Floating Selection and Sequential Backward Floating Selection dynamically determine how many backward or forward corrections should be made.

These methods generally produce strong solutions and are more flexible than fixed Plus-I Take Away-r procedures.

However, the authors demonstrate that floating search may still become trapped in a local optimum.

The diagram on page 6 illustrates a small four-feature search space. The bold path shows SFFS following the same sequence as ordinary forward selection and reaching a subset that is only the third-best solution at its size.

Genetic Algorithms

Previous research showed that Genetic Algorithms could obtain good feature subsets.

However, the paper identifies several limitations:

- premature convergence,
- parameter sensitivity,
- need to tune population size,
- crossover rate,
- mutation rate,
- and number of generations.

Genetic Algorithms also evaluate many chromosomes, which can become expensive when classifier evaluation is the dominant cost.

Basic Tabu Search

Tabu Search begins with one current solution and explores neighboring solutions.

Unlike ordinary local search, it can move to a worse solution.

This allows it to escape a local optimum and continue searching.

Its principal components are:

- current solution,
- neighborhood,
- candidate set,
- tabu list,
- aspiration criterion,
- best-known solution,
- and termination rule.

Tabu List

The tabu list is a short-term memory containing recently selected solutions or moves.

A recently visited move remains prohibited for a specified number of iterations.

This reduces the chance that the search will immediately reverse itself or repeat a previous cycle.

The maximum number of remembered items is the:

`tabu-list length`

When the list becomes full:

- the newest item is added,
- and the oldest item is removed.

Aspiration Criterion

A tabu solution may still be accepted when it produces an exceptionally good result.

The paper uses a common aspiration rule:

Allow a tabu solution when it is better than the best solution found so far.

This prevents tabu restrictions from blocking a new best solution.

General Tabu Search Procedure

The paper describes the process as follows:

1. Generate an initial solution.
2. Record it as the current and best-known solution.
3. Generate candidate solutions from the current neighborhood.
4. Select the best candidate.
5. Reject it when it is tabu and does not satisfy aspiration.
6. Otherwise, move to the candidate.
7. Update the best-known solution when appropriate.
8. Add the selected solution to the tabu list.
9. Remove the oldest list item when necessary.
10. Repeat until the termination condition is reached.

The final output is the best solution discovered during the complete run, not necessarily the last current solution.

Converting the Minimum-Size Problem

The second feature-selection form contains a classification-error constraint.

The authors convert it into an unconstrained objective using a penalty function.

The resulting objective is:

Minimize $J(S) = \text{number of selected features} + \text{penalty}(\text{error of } S)$

The penalty depends on:

- actual classification error,
- tolerated error threshold,
- and a scale parameter.

The penalty is designed so that:

- it is small or negative when error is below the threshold,
- it equals zero at the threshold,
- and it rises rapidly when error exceeds the threshold.

The selected experimental settings are:

Tolerated error threshold = 0.20

Penalty scale parameter = 0.01

This encourages the search to find small subsets while strongly discouraging solutions with excessive error.

Synthetic Error-Rate Model

Extensive testing on many real datasets would require repeatedly designing and evaluating classifiers.

The authors therefore use a synthetic error model adopted from earlier feature-selection research.

Each feature subset is represented by a binary vector:

- 1 means the feature is selected.
- 0 means the feature is excluded.

The synthetic error function includes:

- a base classification error,
- effects caused by excluding individual or interacting features,
- training noise,
- and local defects associated with particular feature combinations.

The model is intended to simulate:

- feature interactions,
- imperfect classifier training,
- redundancy,
- and local irregularities in the search landscape.

Tabu Representation for Variable-Size Selection

For the minimum-size feature-selection problem:

- a solution is represented as a binary string,
- and the initial solution is randomly generated.

For example:

1, 0, 1, 0, 1

means that Features 1, 3, and 5 are selected.

Neighborhood for Variable-Size Selection

A neighboring solution is generated by changing the status of one feature.

The move may be:

- add one excluded feature,
- or remove one included feature.

This is called the **Type A neighborhood** later in the paper.

For n features:

Type A neighborhood size = n

Every neighbor differs from the current subset in exactly one bit.

Candidate Set

The algorithm may evaluate:

- the full neighborhood,
- or a randomly selected portion of it.

The candidate with the best objective value is considered for the next move, subject to the tabu and aspiration rules.

Termination

The main stopping criterion is:

A predefined maximum number of iterations

The best feature subset found during those iterations is returned.

Genetic Algorithm Used for Comparison

The comparison Genetic Algorithm uses:

- randomly generated binary chromosomes,
- proportional fitness-based selection,
- one-point crossover,
- bit mutation,
- and elitist population replacement.

The fitness transformation ensures that even the weakest chromosome has a positive probability of reproduction.

Thirty-Feature Experiment

The first major experiment uses a synthetic feature-selection problem containing:

30 original features

Twenty independent runs are conducted for both Tabu Search and the Genetic Algorithm.

The principal comparison results are:

Algorithm	Best criterion value	Mean criterion value	Worst criterion value	Mean error	Selected features	Evaluation cost
Genetic Algorithm	12.41	12.88	13.24	0.198	12–13	3,000

Tabu Search	12.22	12.69	13.11	0.201	12–13	1,800
-------------	-------	-------	-------	-------	-------	-------

Lower criterion values are better.

Tabu Search produces:

- the best result among all tested algorithms,
- a better average result than the Genetic Algorithm,
- and a lower computation cost.

Comparison with Sequential Methods

Basic SFS and SBS use far fewer evaluations.

However, their selected subsets have relatively poor objective values.

Generalized versions improve quality but require substantially more evaluations.

For example, some fourth-order generalized methods require more than:

54,000 objective evaluations

while Tabu Search uses:

1,800 evaluations

The floating methods SFFS and SBFS produce reasonably strong solutions but still perform slightly worse than the best Tabu Search outcomes.

Thirty-Feature Search Behavior

The chart on page 6 shows the progress of one Tabu Search run across 60 iterations.

Three quantities are plotted:

- criterion-function value,
- classification-error rate,
- and number of selected features.

The search initially changes rapidly.

Within approximately the first ten iterations:

- the number of selected features rises toward 14 or 15,
- the error falls from roughly 0.34 toward 0.20,
- and the objective value improves sharply.

Later iterations fluctuate around:

- 13 to 15 selected features,
- error rates of approximately 0.18 to 0.22,
- and criterion values near the best region.

The authors interpret this as concentration near the boundary between feasible and infeasible solutions, where strong subsets are most likely to occur.

Sixty-Feature Experiment

The next experiment contains:

60 original features

The selected parameter settings are:

- tabu-list length = 120
- candidate-set size = 60
- iterations = 120

Twenty runs are conducted.

The main results include:

Algorithm	Best criterion value	Mean criterion value	Worst criterion value	Mean error	Selected features	Evaluation cost
Genetic Algorithm	18.73	19.25	20.08	0.198	19–20	7,200

Tabu Search	18.19	19.09	19.55	0.200	18–20	7,200
-------------	-------	-------	-------	-------	-------	-------

With equal evaluation cost, Tabu Search achieves:

- a better best result,
- a better mean result,
- and a better worst result.

One-Hundred-Feature Experiment

The largest variable-size experiment contains:

100 original features

The selected Tabu Search parameters are:

- tabu-list length = 200
- candidate-set size = 100
- iterations = 200

The principal results are:

Algorithm	Best criterion value	Mean criterion value	Worst criterion value	Mean error	Selected features	Evaluation cost
Genetic Algorithm	38.61	39.73	41.75	0.199	39–42	20,000
Tabu Search	38.24	39.39	40.10	0.200	38–40	20,000

Tabu Search again produces better:

- best-case quality,
- average quality,

- and worst-case quality.

The authors conclude that Tabu Search remains effective as dimensionality increases.

Fixed-Size Selection with Mahalanobis Distance

The second major experimental section studies fixed-size feature selection.

Instead of directly using classification error, the authors use **Mahalanobis distance** as the subset-quality measure.

The test problem contains:

- 20 original features,
- two Gaussian classes,
- different class means,
- and a shared covariance matrix.

For Gaussian classes with equal covariance, larger Mahalanobis distance generally indicates better class separation and lower classification error.

Mahalanobis Distance

The measure can be written in copy-safe form as:

`Mahalanobis distance = (mean 1 - mean 2)' × inverse covariance × (mean 1 - mean 2)`

Where the prime mark indicates vector transpose.

Larger values are better.

The fixed-size objective is therefore:

`Maximize Mahalanobis distance`

Subject to:

`Number of selected features = required subset size`

Fixed-Size Neighborhood

For fixed-size selection, adding or removing only one feature would change the required subset size.

The neighborhood is therefore modified.

A move performs two actions simultaneously:

1. Remove one selected feature.
2. Add one unselected feature.

The total number of selected features remains constant.

Genetic Algorithm Modification

The Genetic Algorithm is also modified for fixed-size selection.

Its chromosomes must always contain the required number of selected features.

Crossover selects the required number of features from the union of the two parent subsets.

Mutation simultaneously:

- removes one feature,
- and adds another.

Selecting 10 Features

The first fixed-size experiment selects:

10 features from 20

The exact optimal Mahalanobis distance obtained through Branch and Bound is:

38.16

Results include:

Algorithm	Mahalanobis distance
Branch and Bound	38.16
SFS	29.72
SBS	30.00
SFFS	38.16
SBFS	34.51
Genetic Algorithm mean	38.06
Tabu Search	38.16

Tabu Search reaches the global optimum in:

20 out of 20 runs

The Genetic Algorithm reaches it in:

18 out of 20 runs

Selecting 12 Features

The second fixed-size experiment selects:

12 features from 20

The optimal Mahalanobis distance is:

67.23

Results include:

Algorithm	Mahalanobis distance
Branch and Bound	67.23
SFS	42.86
SBS	41.39
SFFS	54.33
SBFS	62.92
Genetic Algorithm mean	56.48
Tabu Search mean	58.22

Both Tabu Search and the Genetic Algorithm reach the optimum in:

7 out of 20 runs

However, the worst Tabu Search result is noticeably stronger than the worst Genetic Algorithm result.

The authors therefore conclude that Tabu Search has:

- a higher mean result,

- better worst-case reliability,
- and slightly better computational efficiency.

Distribution of Twelve-Feature Solutions

The graph on page 9 shows the distribution of Mahalanobis distances for all feasible 12-feature subsets.

Most subsets cluster at much lower Mahalanobis distances.

Only a very small number of subsets reach the extreme high-quality region.

The marked algorithm positions show that:

- SFS and SBS remain far from the optimum,
- floating methods move closer,
- Genetic Algorithm occasionally reaches the optimum,
- and Tabu Search also reaches the optimal extreme.

This illustrates how small the high-quality region is relative to the complete subset space.

Effect of Tabu-List Length

The authors test tabu-list lengths from:

2 to 90

The experiment uses:

- 30 original features,
- 90 maximum iterations,
- and 21 different initial solutions.

Results improve as the tabu-list length grows from 2 toward approximately 16.

Mean criterion values include:

Tabu-list length	Mean criterion value
---------------------	----------------------

2	13.06
4	13.00
6	12.92
8	12.81
10	12.74
12	12.74
14	12.74
16–90	12.66

After the length reaches approximately 16, further increases produce no additional improvement in this experiment.

The authors observe that 16 is approximately half of the original 30-feature dimension.

Recommended Tabu-List Range

The paper suggests a broad practical range:

Tabu-list length between $n / 2$ and $3n$

Where:

n = number of original features

This is not presented as a universal optimum.

The appropriate length should also consider:

- candidate-set size,
- neighborhood structure,
- and the distribution of feasible solutions.

Interpretation of Tabu-List Length

A shorter tabu list promotes:

- stronger local intensification,
- greater freedom to revisit nearby regions,
- but increased risk of cycling.

A longer tabu list promotes:

- greater diversification,
- movement farther from recent states,
- but possible restriction of useful moves.

The experiments suggest that continually increasing the list eventually stops improving performance.

Effect of Candidate-Set Size

The authors compare candidate-set sizes of:

- 30,
- 15,
- 8,
- and 4.

The number of iterations is increased for smaller candidate sets so that total evaluation cost remains approximately comparable.

Results are:

Candidate-set size	Iterations	Mean criterion	Best	Worst
30	60	12.69	12.22	13.11
15	120	12.73	12.33	13.05
8	225	12.87	12.12	13.65
4	450	13.13	12.61	13.76

Larger candidate sets generally produce better mean performance.

Why Larger Candidate Sets Help

With a larger candidate set:

- the algorithm has more alternatives,
- it is more likely to identify a strong admissible move,
- and the tabu list affects decisions more frequently.

With a very small candidate set:

- recent tabu restrictions rarely matter,
- moves become more arbitrary,
- and the search begins to resemble random search.

The experiments therefore favor evaluating a relatively large portion of the local neighborhood.

Effect of Initial Solution

The authors compare:

- randomly generated initial subsets,

- and solutions produced by other near-optimal algorithms.

The better initial solutions produce an average final criterion value of approximately:

12.56

Random initial solutions produce an average of approximately:

12.69

The difference is relatively small.

The authors conclude that Tabu Search is not highly sensitive to the initial solution in these experiments.

When the starting subset is poor, the early search behaves similarly to forward or backward sequential selection until it reaches a stronger region.

Neighborhood Type A

Type A changes exactly one feature state:

- add one feature,
- or remove one feature.

For n features:

Neighborhood size = n

This is the primary neighborhood used in the variable-size experiments.

Neighborhood Type B

Type B permits changing one or two feature states.

Possible moves include:

1. Add one feature.
2. Remove one feature.
3. Add two features.
4. Remove two features.
5. Add one feature and remove another.

For n features:

Type B neighborhood size = $n \times (n + 1) / 2$

This is much larger than Type A.

Type A Versus Type B Results

The comparison results are:

Neighborhood	Mean criterion	Best	Worst	Mean tabu activations
Type A	12.69	12.22	13.11	33.8
Type B	13.02	12.32	13.64	2.2

Type A performs better on average.

With the much larger Type B neighborhood, only 30 random candidates are examined per iteration.

This gives the search a very low neighborhood-access ratio.

The tabu list rarely affects the selected moves, and the search becomes more random.

The authors argue that a larger theoretical neighborhood is not necessarily beneficial when only a tiny portion can be evaluated.

Main Findings

The paper's principal findings are:

- Feature selection is a difficult combinatorial optimization problem.
- Exhaustive search and Branch and Bound become impractical as dimensionality grows.
- Greedy sequential methods are efficient but often produce weak subsets.

- Sequential floating methods improve flexibility but can still become trapped in local optima.
- Genetic Algorithms produce strong subsets but require more evaluations and careful parameter tuning.
- Tabu Search can accept worsening moves and escape local optima.
- A short-term tabu list helps guide the search and prevent cycling.
- Aspiration criteria allow exceptionally strong tabu moves.
- Tabu Search produces the best result in the 30-feature variable-size experiment.
- It outperforms the Genetic Algorithm in mean and worst-case quality on 60- and 100-feature problems.
- It reaches the exact 10-feature optimum in every fixed-size run.
- It is more reliable than the Genetic Algorithm in the 12-feature experiment.
- Performance improves as tabu-list length increases up to a useful threshold.
- Larger candidate sets generally improve average performance.
- The method is relatively insensitive to its initial solution.
- A simple one-feature neighborhood performs better than a much larger neighborhood sampled only sparsely.
- Search quality depends on the interaction among tabu length, candidate size, and neighborhood structure.

Important Limitations

Synthetic Data

The experiments use synthetic error models and simulated Gaussian feature distributions.

The study does not test the method on real classification datasets.

The authors explicitly identify real-data validation as future work.

Early Feature-Selection Context

The paper was published in 2002.

Its definitions of medium- and large-scale feature selection reflect the computing conditions of that period.

Problems containing 30 to 100 features were considered challenging, whereas modern applications may contain:

- thousands,
- tens of thousands,

- or millions of features.

Classifier Evaluation Is Simulated

The variable-size experiments use a synthetic error-rate model rather than repeatedly training an actual classifier.

This improves experimental control but may not represent every practical classifier.

Limited Realistic Noise Analysis

The error model includes training defects and feature interactions, but its behavior depends on manually designed synthetic parameters.

The conclusions may change with different feature dependencies or noise structures.

Limited Objectives

The paper mainly optimizes:

- classification error,
- feature count,
- or Mahalanobis distance.

It does not explicitly consider:

- feature-acquisition cost,
- measurement time,
- missing-value risk,
- privacy cost,
- interpretability,
- or classifier latency.

Comparison Parameters

The comparison algorithms use parameter settings selected by experimentation.

Different tuning effort could alter the rankings.

No Formal Statistical Tests

The paper reports:

- best,
- mean,
- worst,
- and success frequencies.

However, it does not conduct formal significance testing between the stochastic methods.

Full Solutions Stored in the Tabu List

The basic description places selected solutions in the tabu list.

For much larger feature spaces, storing complete binary subsets could consume more memory than storing move attributes.

Recommended Tabu Range Is Empirical

The suggested range from $n / 2$ to $3n$ is derived from the tested synthetic problems.

It should not be treated as a universal theoretical rule.

Fixed Candidate Evaluation

The method evaluates a fixed number of candidate subsets each iteration.

Its efficiency and behavior may change when:

- classifier evaluation is extremely expensive,
- the feature count is extremely large,
- or neighborhood sampling becomes very sparse.

Future Work

The authors recommend evaluating Tabu Search on real datasets.

They specifically suggest comparing it with:

- Genetic Algorithms,
- Sequential Forward Floating Selection,
- and Sequential Backward Floating Selection

using practical classification problems.

Further research would also be needed to determine:

- better parameter-selection rules,
- scalable neighborhood sampling,
- and behavior on substantially higher-dimensional datasets.

Conclusion

The paper concludes that Tabu Search is a promising method for optimal and near-optimal feature selection.

Its main advantage is the ability to explore beyond the first local optimum while using memory to prevent repeated cycling.

Across the synthetic experiments, Tabu Search generally produces:

- high-quality feature subsets,
- strong average performance,
- better worst-case outcomes,
- and competitive or lower computational cost.

It performs particularly well when:

- the tabu list is sufficiently large to prevent reversal,
- the candidate set covers a meaningful portion of the neighborhood,
- and the neighborhood remains simple enough for the tabu mechanism to guide the search effectively.

The authors conclude that Tabu Search offers a useful balance between the exactness of exhaustive methods and the efficiency of greedy selection methods.

Evolutionary Algorithms, Simulated Annealing and Tabu Search: A Comparative Study

Habib Youssef, Sadiq M. Sait, and Hakim Adiche

Summary

This paper experimentally compares three general-purpose metaheuristic optimization methods:

- Genetic Algorithm, or GA
- Simulated Annealing, or SA
- Tabu Search, or TS

The algorithms are tested on the same **Very Large-Scale Integration circuit floorplanning problem** using the same:

- solution representation,
- multi-objective evaluation method,
- neighborhood operations,
- and general experimental conditions.

The objective is not merely to compare the best final result. The authors also examine:

- how quickly each algorithm improves,
- how its search progresses,
- how often it explores high- or low-quality solutions,
- and how much implementation and parameter-tuning effort it requires.

The floorplanning problem simultaneously considers:

- occupied circuit area,
- total wire length,
- and circuit delay.

The authors combine these conflicting objectives using a fuzzy-logic evaluation function. Higher fuzzy membership values represent better floorplans.

Across all five tested circuits, Tabu Search performs best overall. Genetic Algorithm ranks second, while Simulated Annealing performs worst in its original implementation. The authors nevertheless stress that this ranking applies to the tested floorplanning problem and should not automatically be generalized to all optimization problems.

Purpose of the Study

The paper has two main purposes.

First, it explains the structural similarities and differences among GA, SA, and TS.

Second, it compares the three methods experimentally under a shared optimization setting.

The authors evaluate them according to four criteria:

1. Quality of the best solution found
2. Progress from the initial solution until termination
3. Improvement of the best solution over time or evaluations
4. Distribution of the quality of all solutions examined

This fourth criterion is important because two algorithms may find similar final solutions while spending their computation in very different regions of the search space.

Common Characteristics of the Three Algorithms

The authors identify several shared properties.

Approximation Methods

GA, SA, and TS are heuristic or approximate algorithms.

They do not normally guarantee that the final solution is globally optimal.

Blindness to Optimality

The algorithms do not automatically know when they have reached the true optimum.

They therefore require stopping conditions such as:

- maximum iterations,
- maximum evaluations,
- maximum runtime,
- or lack of further improvement.

Acceptance of Worse Moves

All three methods may accept solutions that are worse than the current or previous solution.

This allows them to escape local optima.

However, they use different mechanisms:

- GA uses population evolution and stochastic operators.
- SA accepts worse solutions according to temperature.
- TS chooses strong neighborhood moves while using memory to prevent reversal.

General Applicability

All three methods can be adapted to many combinatorial optimization problems when the following are available:

- a solution representation,
- an evaluation function,
- and a mechanism for generating alternative solutions.

Asymptotic Convergence

Under appropriate theoretical conditions, the algorithms may approach an optimal solution as computation continues indefinitely.

However, practical implementations stop after limited time or evaluations and therefore do not rely on those theoretical limits.

Comparing Iterative Metaheuristics

The authors explain that comparing iterative heuristics is more difficult than comparing ordinary constructive algorithms.

For iterative algorithms, computation depends on:

- the number of iterations,
- the number of objective evaluations,
- the cost of the evaluation function,
- the neighborhood or perturbation operators,
- and implementation details.

The most meaningful comparisons therefore concern:

- final solution quality,
- improvement rate,
- reliability,
- and the quality of the search region explored.

For example, Algorithm A may be considered better than Algorithm B when A consistently finds:

- lower-cost or higher-fitness solutions,
- more solutions above a chosen quality threshold,
- or strong solutions after fewer evaluations.

VLSI Floorplanning Problem

The benchmark problem is the floorplanning of VLSI circuits.

A floorplan determines:

- the position of circuit blocks,
- the shape and orientation of flexible blocks,
- the overall circuit dimensions,
- and the spatial relationships among connected components.

A valid floorplan must satisfy several constraints.

Block Placement

Every circuit block must be assigned a location.

No Overlap

No two blocks may occupy the same physical space.

Area Preservation

A flexible block may change shape, but its area must remain constant.

In copy-safe form:

`Block area = block width × block height`

Aspect-Ratio Constraints

The height-to-width ratio of each flexible block and the complete floorplan must remain within allowable limits.

Optimization Objectives

The floorplan should minimize:

- total occupied area,
- total estimated wire length,
- and circuit delay.

These objectives can conflict.

A compact floorplan may create longer wires or greater delay, while minimizing wire length may require more area.

Circuit Representation

The study restricts the layouts to **slicing floorplans**.

A slicing floorplan can be created by repeatedly dividing a rectangle using:

- horizontal cuts,
- and vertical cuts.

The layout can be represented as a binary slicing tree.

- Leaves represent circuit blocks.
- Internal nodes represent horizontal or vertical cuts.

Polish Expression Encoding

The slicing tree is converted into a Polish expression.

The expression contains:

- block numbers as operands,
- **H** as a horizontal-cut operator,
- and **V** as a vertical-cut operator.

For example:

12V34H5V

represents one possible sequence of block combinations and cuts.

The meanings of the operators are approximately:

- ijH means block or subfloorplan j is above i .
- ijV means i is to the left of j .

Every GA chromosome, SA state, and TS solution uses this same Polish-expression representation.

This helps make the comparison more consistent.

Floorplan Evaluation

Each solution is evaluated according to:

- area,
- wire length,
- and delay.

Area

Area is estimated using the smallest rectangle containing the complete floorplan.

Wire Length

Wire length is based on the estimated interconnections among circuit blocks.

A Steiner-tree approximation is used to estimate the required length of individual nets.

Delay

Delay is estimated along the longest signal path in the circuit.

It includes:

- circuit-block delay,
- and estimated interconnect delay.

Problem with Ordinary Weighted Sums

A common way to combine several objectives is:

Combined score = $w_1 \times \text{objective 1} + w_2 \times \text{objective 2} + \dots + w_n \times \text{objective n}$

However, this requires selecting fixed numerical weights.

The authors argue that appropriate weights can be difficult to determine, especially when the designer's preferences are qualitative.

They therefore use fuzzy logic.

Fuzzy-Logic Evaluation

The paper defines three linguistic objectives:

- small area,
- short wire length,
- low delay.

Each objective receives a membership function with values between 0 and 1.

Interpretation:

- **Membership = 1** means the objective is highly satisfactory.
- **Membership = 0** means it is highly unsatisfactory.

The original area, length, and delay measurements are normalized using lower and upper reference values.

Lower and Upper Reference Values

The lower bounds include:

- **A_{min}** for area,
- **L_{min}** for wire length,
- and **D_{min}** for delay.

The upper reference values are based on the initial floorplan:

- A_{max} ,
- L_{max} ,
- and D_{max} .

The normalized membership decreases as:

- area increases,
- wire length increases,
- or delay increases.

Thus, smaller objective values receive higher satisfaction.

Fuzzy Rule for Good Solutions

The principal linguistic rule is:

If area is small OR wire length is short OR delay is low, then the floorplan is good.

A strict fuzzy OR would use only the maximum membership value:

Goodness = maximum of area membership, length membership, and delay membership

However, this allows one strong objective to dominate the evaluation completely.

The authors instead use a compensatory Ordered Weighted Averaging operation.

In simplified copy-safe form:

Goodness = $b \times \text{maximum membership}$
 $+ (1 - b) \times \text{average membership}$

Where:

- $b = 1$ behaves like a strict maximum,
- $b = 0$ behaves like an average.

This allows all objectives to influence the result.

Relative Importance of Objectives

The authors treat area as more important than wire length and delay.

They modify the fuzzy memberships using exponents.

In simplified form:

Adjusted area membership = square root of area membership

Adjusted wire membership = wire membership squared

Adjusted delay membership = delay membership squared

Taking the square root emphasizes area, while squaring the other memberships makes them more demanding.

The complete fuzzy score remains between 0 and 1.

A higher score means a better floorplan.

General Requirements of Each Metaheuristic

The paper states that each iterative optimization implementation requires:

- solution encoding,
- initial-solution generation,
- evaluation function,
- perturbation or transition operators,
- parameter settings,
- and a stopping criterion.

Even when two researchers use the same general algorithm, different choices for these components can produce different performance.

Therefore, the study compares particular implementations rather than claiming to compare every possible form of GA, SA, and TS.

Genetic Algorithm

GA maintains a population of possible floorplans.

Each floorplan is represented as a Polish-expression chromosome.

The initial population is randomly generated.

Every chromosome receives a fitness equal to its fuzzy floorplan membership:

$\text{Fitness} = \text{fuzzy goodness of the floorplan}$

Higher values are better.

Genetic Selection

Parents are chosen using roulette-wheel selection.

A chromosome's probability of selection increases with its fitness.

Thus, stronger floorplans are more likely to contribute genetic material to the next generation.

Genetic Crossover

The study uses three crossover methods.

Operand-Inheritance Crossover

The operands or circuit blocks are inherited from one parent.

The operators are taken from the other parent.

This preserves the block ordering of one solution while adopting cut information from the other.

Slicing-Inheritance Crossover

The horizontal and vertical cut structure is inherited from one parent.

The blocks are taken from the other.

This preserves the broad slicing structure of one floorplan.

Partially Mapped Crossover

Partially Mapped Crossover is applied to the block operands.

It exchanges part of the parent permutations while maintaining valid block placement.

The diagrams on page 8 illustrate how these crossover operations produce valid offspring floorplans.

Genetic Mutation

Three mutation operators are used:

1. Reverse a selected sequence of operators.
2. Swap two randomly selected block operands.
3. Swap a block with an adjacent operator when the result remains valid.

Mutation introduces new structural information that may not be present in the current population.

Genetic Inversion

The inversion operation reverses the ordering of the operands in the chromosome.

For example:

268HH7H5V4H13VV

may become:

314HH5H7V8H62VV

This changes the block order while retaining the operator structure.

Genetic Algorithm Parameters

The selected GA settings are:

- population size = 10
- crossover probability = 80%
- mutation probability = 5%
- inversion probability = 30%
- number of offspring per generation = population size

The algorithm stops after a fixed number of generations or after reaching the shared evaluation budget.

The paper later concludes that GA requires the most implementation and tuning effort because it needs:

- population design,
- parent selection,
- several crossover methods,
- mutation,
- inversion,
- and survivor selection.

Simulated Annealing

SA begins with one current floorplan and repeatedly generates a neighboring floorplan.

It accepts every improving move.

A worse move may also be accepted.

The acceptance probability is controlled by temperature.

The search behaves differently at different temperatures:

- high temperature: almost random,
- moderate temperature: balanced exploration and improvement,
- low temperature: mostly greedy,
- zero temperature: fully greedy.

Metropolis Procedure

At a fixed temperature, SA performs a sequence of neighborhood transitions using the Metropolis criterion.

The fitness difference is:

$\Delta \text{fitness} = \text{new fitness} - \text{current fitness}$

When:

$\Delta \text{fitness} > 0$

the new floorplan is better and is accepted.

When:

$\Delta \text{fitness} \leq 0$

the worse solution may be accepted with probability:

$\text{Acceptance probability} = \exp(\Delta \text{fitness} / \text{temperature})$

Because $\Delta \text{fitness}$ is negative for a worse move, the probability remains between 0 and 1.

The candidate is accepted when:

$\text{Random number between 0 and 1} < \text{acceptance probability}$

Cooling Schedule

The temperature is reduced geometrically:

$\text{New temperature} = \alpha \times \text{current temperature}$

The study uses:

$\alpha = 0.85$

The number of transitions performed at each temperature can also be updated as:

$\text{New transition count} = \beta \times \text{current transition count}$

The authors use:

$\beta = 1$

Therefore, the number of transitions does not increase in their final implementation.

Initial Temperature

The initial temperature should be high enough to accept almost all early moves.

However, a temperature that is excessively high wastes computation.

The authors estimate the initial temperature by:

1. Generating a fixed number of preliminary moves.
2. Measuring the average fitness decrease of inferior moves.

3. Selecting an initial acceptance probability close to 1.
4. Solving the Metropolis relationship for temperature.

In copy-safe form:

$$\text{Initial temperature} = \text{average inferior fitness change} / \ln(\text{initial acceptance probability})$$

The selected initial probability is approximately:

0.999

Because the fitness difference is negative and the logarithm is also negative, the resulting temperature is positive.

Simulated Annealing Stopping Conditions

SA stops when either:

$$\text{Temperature} < 0.001$$

or:

$$\text{Accepted moves} / \text{attempted moves} < 0.05$$

The second condition indicates that the search has effectively frozen and accepts very few new states.

Simulated Annealing Perturbations

SA uses neighborhood transformations similar to the GA mutation operators:

- reversing operator sequences,
- swapping two blocks,
- and swapping a block with an adjacent operator.

Unlike GA, SA applies them to one current solution rather than a population.

Tabu Search

TS begins with one current feasible floorplan.

At every iteration, it generates several neighbors and selects the best candidate that is not prohibited by tabu restrictions.

The selected candidate does not need to improve the current solution.

This allows TS to leave a local optimum while still choosing the strongest available direction.

Candidate List

The candidate list contains a limited number of neighboring floorplans.

The selected size is:

`Candidate-list size = 20`

The authors describe candidate-list size as a trade-off.

- A larger list provides more chances to find a strong move.
- A smaller list requires fewer evaluations.

Tabu Restrictions

A recently performed move receives a tabu attribute.

A future move that would reverse it is temporarily forbidden.

This prevents the search from immediately undoing recent decisions.

The tabu list can be understood as a sliding window over recent moves.

Aspiration Criterion

A tabu move may be accepted when it produces a solution better than the best one found so far.

This is called the:

`Best-cost aspiration criterion`

It prevents memory restrictions from blocking an outstanding improvement.

Tabu Search Parameters

The TS implementation uses:

- 250 iterations,
- candidate-list size of 20,
- and tabu-list size equal to approximately one-fourth of the number of circuit blocks.

In copy-safe form:

`Tabu-list size = number of blocks / 4`

The paper uses three separate tabu lists because it uses three neighborhood operators.

Stored Tabu Attributes

The stored attribute depends on the move.

Operator-Sequence Inversion

The stored attribute is the position of the first operator in the reversed sequence.

Swapping Two Blocks

The stored attribute is one of the involved block operands, specifically the left operand.

Swapping a Block and Operator

The stored attribute is the block operand involved in the swap.

All three lists have the same size.

Only short-term memory is implemented. The study does not use more advanced TS mechanisms such as:

- long-term diversification,
- intensification memory,
- or path relinking.

Experimental Circuits

The algorithms are tested on five circuits containing between:

15 and 141 circuit blocks

The circuits are identified as:

- Ckt-1
- Ckt-2
- Ckt-3
- Ckt-4
- Ckt-5

The number of blocks is:

Circuit	Number of blocks
---------	---------------------

Ckt-1	15
-------	----

Ckt-2	30
-------	----

Ckt-3	39
-------	----

Ckt-4	45
-------	----

Ckt-5	141
-------	-----

The paper also reports lower-bound or reference values for:

- area,
- wire length,
- and timing.

Shared Evaluation Budget

For the primary comparisons, each algorithm is forced to stop after:

5,000 floorplan evaluations

This provides a common objective-evaluation budget.

The use of a shared number of evaluations is more informative than comparing raw iteration counts because:

- one GA generation evaluates several individuals,
- one SA transition evaluates one neighbor,
- and one TS iteration evaluates a neighborhood of 20 solutions.

Best-Solution Progress

Figure 11 compares the best fuzzy membership found as the number of evaluated solutions increases.

The same ranking appears across all five circuits:

1. Tabu Search
2. Genetic Algorithm
3. Simulated Annealing

TS finds a strong floorplan rapidly and then continues making smaller improvements.

GA improves more gradually.

SA produces the weakest best-solution curve in the original implementation.

Results for Ckt-3 and Ckt-4

The figure gives detailed examples for:

- Ckt-3 with 39 blocks,
- and Ckt-4 with 45 blocks.

For both circuits:

- TS rises rapidly toward a high fuzzy-membership value.

- GA reaches a moderate or strong value more slowly.
- SA remains below the other two for most of the evaluation budget.

For Ckt-4, the final TS fuzzy value appears close to approximately 0.78, while GA remains near 0.71 and the original SA implementation remains lower.

The precise values are presented graphically rather than in a consolidated results table.

Search-Path Behavior

Figure 12 compares the internal search patterns.

Genetic Algorithm Behavior

GA's average population fitness rises rapidly during the early generations.

The population later reaches a plateau and changes only slightly.

Occasional decreases occur, but they are small.

Simulated Annealing Behavior

SA shows large and frequent changes throughout the run.

Even after thousands of iterations, it continues accepting solutions substantially worse than its better states.

This demonstrates the strongest stochastic or exploratory behavior among the three methods.

Tabu Search Behavior

TS rapidly reaches a good region.

After reaching that region, its worsening moves are relatively small.

Its selected solution remains near the high-quality area.

Greediness Ranking

The authors rank the methods according to how greedily they search.

From most greedy to least greedy:

1. Tabu Search
2. Genetic Algorithm
3. Simulated Annealing

TS is most greedy because it selects the best admissible candidate from a neighborhood.

GA is intermediate because population selection favors strong individuals but mutation and crossover introduce variation.

SA is least greedy because it can continue accepting substantial worsening moves according to temperature.

Problem with SA's Normalized Fuzzy Cost

The fuzzy evaluation lies between:

0 and 1

The authors find that this narrow scale interacts poorly with the Metropolis criterion.

For example, reducing fitness from:

0.6 to 0.4

is a significant deterioration in the fuzzy evaluation.

However, at:

Temperature = 0.1

the acceptance probability is:

$\exp(-0.2 / 0.1) = \exp(-2)$

which is approximately:

0.14

Thus, a substantially worse floorplan still has around a 14% probability of acceptance.

The authors consider this too high during the colder phase of the search.

SA Cost-Inflation Experiment

To reduce the acceptance of large worsening moves, the authors multiply the fuzzy cost difference by:

`Inflation factor = 100`

This effectively changes the cost scale from:

`0 to 1`

into:

`0 to 100`

The acceptance probability becomes much smaller for meaningful deteriorations.

The paper reports that this modification substantially improves SA.

It results in:

- better best solutions,
- a stronger searched subspace,
- and smaller accepted worsening moves.

Meaning of the Inflation Result

This experiment demonstrates that SA performance depends not only on the ranking of objective values.

It also depends on their numerical scale.

A monotonic rescaling leaves the ordering of floorplans unchanged, but it changes:

`exp(change in fitness / temperature)`

and therefore changes the search behavior.

The SA temperature schedule must consequently be calibrated to the magnitude of the evaluation function.

Distribution of Searched Solutions

Figure 14 divides fuzzy solution quality into intervals and counts how many evaluated floorplans fall into each interval.

Higher fuzzy values indicate stronger floorplans.

Tabu Search Distribution

TS concentrates most evaluations in the highest-quality intervals.

This indicates strong intensification around promising floorplans.

Genetic Algorithm Distribution

GA also spends much of its search in relatively good regions.

However, its concentration is generally less extreme than TS.

Simulated Annealing Distribution

The original SA implementation spends most of its evaluations in poor- or medium-quality regions.

It examines relatively few of the strongest floorplans.

This helps explain its weaker best-solution performance under the shared evaluation budget.

Overall Experimental Ranking

For all five tested circuits, the overall ranking is:

1. Tabu Search
2. Genetic Algorithm
3. Simulated Annealing

The authors describe GA as a close second.

SA is described as a distant third in the original floorplanning experiments.

The modified cost-inflation version improves SA, but the paper still favors TS for this application.

Why Tabu Search Performs Well

The authors attribute TS performance to several characteristics.

Best-Candidate Selection

TS evaluates several neighbors and chooses the best admissible one.

This gives it a strong immediate search direction.

Explicit Memory

The tabu lists prevent reversal and repeated cycling.

Controlled Nonimproving Moves

TS can leave a local optimum without relying entirely on random acceptance.

Concentration in Good Regions

The experimental histograms show that TS spends most of its evaluations in high-quality regions.

Limited Worsening After Improvement

After TS reaches a strong region, accepted worsening moves remain relatively small.

Why Simulated Annealing Performs Poorly Here

The original SA implementation is weakened by:

- a cost scale poorly matched to temperature,
- continued acceptance of large worsening moves,
- random neighbor generation,
- and excessive time spent in low-quality regions.

The authors stress that SA's performance depends heavily on:

- its cooling schedule,
- its initial temperature,

- the objective-function scale,
- and its problem-specific perturbation design.

The inflation experiment shows that some of the weakness is implementation-related rather than an unavoidable property of SA.

Domain-Specific Knowledge

The paper discusses where each method uses problem-specific knowledge.

Simulated Annealing

SA incorporates most domain information in:

- the solution encoding,
- cost function,
- and perturbation operators.

The choice among possible moves is largely random, and acceptance is governed by the Metropolis rule.

Genetic Algorithm

GA uses domain knowledge throughout:

- representation,
- fitness,
- parent selection,
- crossover,
- mutation,
- inversion,
- and survivor selection.

Tabu Search

TS uses domain knowledge in:

- neighborhood construction,
- candidate generation,
- tabu attributes,
- tabu-list size,
- and aspiration rules.

Its explicit memory also influences future search direction.

Implementation and Tuning Effort

The authors judge GA as requiring the most engineering effort.

GA needs decisions involving:

- population size,
- parent selection,
- crossover probability,
- several crossover operators,
- mutation probability,
- inversion probability,
- offspring generation,
- and survivor selection.

SA and TS require similar and lower implementation effort than GA in this study.

However, both still need careful parameter choices.

SA requires:

- initial temperature,
- cooling rate,
- transitions per temperature,
- stopping rule,
- and properly scaled costs.

TS requires:

- neighborhood design,
- candidate-list size,
- tabu attributes,
- tabu-list length,
- aspiration criterion,
- and iteration limit.

Main Findings

The paper's principal findings are:

- GA, SA, and TS are general iterative heuristics for difficult combinatorial problems.

- All three can accept nonimproving changes and escape local optima.
- Their primary difference is how they search and how they use memory or randomness.
- A common solution representation and evaluation function can support a controlled comparison.
- Fuzzy logic can combine area, wire length, and delay into one floorplan score.
- GA performs a population-based parallel search.
- SA performs a temperature-controlled stochastic trajectory.
- TS performs a memory-guided neighborhood trajectory.
- TS finds strong floorplans faster than GA and SA in the tested circuits.
- GA improves quickly but often reaches a plateau.
- Original SA continues accepting relatively poor solutions even at later stages.
- TS is the most greedy method.
- SA is the least greedy method.
- TS and GA spend most of their evaluations in high-quality search regions.
- SA spends more time in weak regions.
- Rescaling the fuzzy cost substantially improves SA.
- The numerical scale of the objective function therefore matters for SA.
- TS provides the best solution quality and search-subspace quality across all five circuits.
- GA is a close second.
- SA ranks third.
- GA requires the greatest implementation and tuning effort.
- SA and TS require similar, lower effort in this study.

Important Limitations

One Problem Category

The comparison uses only VLSI floorplanning.

The results do not establish that TS is universally superior to GA or SA.

The authors explicitly warn against generalizing the ranking to all optimization problems.

Particular Implementations

The study compares one engineered version of each algorithm.

Different:

- cooling schedules,
- population sizes,
- crossover operators,

- tabu structures,
- local searches,
- or hybrid mechanisms

could change the results.

Only Short-Term Tabu Search

The TS implementation uses only short-term memory.

It does not test:

- long-term diversification,
- intensification memory,
- frequency memory,
- strategic oscillation,
- or path relinking.

A more advanced TS could perform differently, but it would also complicate the comparison.

Limited Circuit Set

Only five circuits are tested.

Although they range from 15 to 141 blocks, this remains a relatively small benchmark collection.

Graph-Based Results

Many numerical conclusions are based on charts rather than consolidated result tables.

Exact final fuzzy-membership values and run-to-run variation are not fully reported.

No Statistical Significance Tests

The paper does not provide formal significance testing or confidence intervals.

It reports consistent rankings but does not quantify uncertainty across repeated runs.

Unclear Repetition Counts

The paper does not clearly provide a full repeated-run protocol for every circuit and method.

Because the methods are stochastic, multiple independent runs would be necessary to assess reliability.

Objective-Function Dependence

The fuzzy evaluation depends on:

- membership-function shapes,
- normalization limits,
- objective priorities,
- and the Ordered Weighted Averaging parameter.

Different fuzzy-rule settings could alter the algorithm rankings.

SA Initially Disadvantaged by Cost Scaling

The original SA implementation uses a fuzzy score in the range from 0 to 1 without sufficient adjustment to the temperature scale.

The later inflation experiment substantially improves it.

This means the original comparison partly reflects an initially mismatched SA cost scale.

Different Search Structures

A common evaluation budget of 5,000 solutions improves fairness, but the algorithms still use fundamentally different information.

- GA maintains a population.
- SA evaluates one stochastic trajectory.
- TS evaluates candidate neighborhoods and remembers move attributes.

Equal evaluation counts do not make their other computational overhead identical.

No Exact Optimum

The study compares the algorithms with one another rather than with proven globally optimal floorplans.

It therefore establishes relative performance among the tested implementations, not absolute optimality.

No Free Lunch Discussion

The authors refer to the No Free Lunch theorem.

Its general implication is that when performance is averaged over every possible optimization problem, no one algorithm is universally superior.

An algorithm that performs well on one class must perform less well on another when all possible functions are considered.

However, the authors point out that practical algorithms incorporate domain-specific knowledge.

A well-engineered method designed for one problem category can therefore outperform a poorly matched general method on that category.

Their claim is not:

Tabu Search is always best.

Their narrower conclusion is:

For the tested VLSI floorplanning formulation and implementations, Tabu Search behaves best.

Conclusion

The paper concludes that GA, SA, and TS are all useful and robust optimization frameworks, but they use fundamentally different search strategies.

Genetic Algorithm searches through a population using:

- selection,
- crossover,
- mutation,
- and inversion.

Simulated Annealing follows one trajectory using:

- randomized neighbors,
- temperature,
- and probabilistic acceptance.

Tabu Search follows one trajectory using:

- candidate neighborhoods,
- best-admissible movement,
- short-term memory,
- and aspiration criteria.

For the tested VLSI floorplanning problems, Tabu Search produces:

- the strongest best solutions,
- the fastest early improvement,
- the greatest concentration in high-quality search regions,
- and the best overall behavior.

Genetic Algorithm performs nearly as well but requires more implementation and parameter-tuning effort.

Simulated Annealing performs worst in its original form, largely because it spends too much computation in low-quality regions and continues accepting large worsening moves. Rescaling the fuzzy cost improves its performance substantially.

The authors ultimately recommend Tabu Search for this floorplanning application while emphasizing that further comparisons across other problem categories are necessary before drawing broader conclusions.

References:

- Alharkan, I., Saleh, M., Ghaleb, M. A., Kaid, H., Farhan, A., & Almarfadi, A. (2020). Tabu search and particle swarm optimization algorithms for two identical parallel machines scheduling problem with a single server. *Journal of King Saud University–Engineering Sciences*, 32(5), 330–338. <https://doi.org/10.1016/j.jksues.2019.03.006>
- Allvi, M. W., Hasan, M., Rayan, L., Shahabuddin, M., Khan, M. M., & Ibrahim, M. (2020). Feature selection for learning-to-rank using simulated annealing. *International Journal of Advanced Computer Science and Applications*, 11(3). <https://doi.org/10.14569/IJACSA.2020.0110387>
- Glover, F. (1989). Tabu search—Part I. *ORSA Journal on Computing*, 1(3), 190–206. <https://doi.org/10.1287/ijoc.1.3.190>
- Grabusts, P., Musatovs, J., & Golenkov, V. (2019). The application of simulated annealing method for optimal route detection between objects. *Procedia Computer Science*, 149, 95–101. <https://doi.org/10.1016/j.procs.2019.01.112>
- Huang, S., Tian, N., Wang, Y., & Ji, Z. (2016). Multi-objective flexible job-shop scheduling problem using modified discrete particle swarm optimization. *SpringerPlus*, 5, Article 1432. <https://doi.org/10.1186/s40064-016-3054-z>
- Jwo, J.-S., Lee, C.-H., Chen, J.-T., Lin, C.-S., Lin, C.-Y., Cheng, W.-K., Chang, C.-H., & King, J.-K. (2023). Application of tabu search for job shop scheduling based on manufacturing order swapping. *Engineering Proceedings*, 55(1), Article 51. <https://doi.org/10.3390/engproc2023055051>
- Kennedy, J., & Eberhart, R. (1995). Particle swarm optimization. In *Proceedings of ICNN'95—International Conference on Neural Networks* (Vol. 4, pp. 1942–1948). IEEE. <https://doi.org/10.1109/ICNN.1995.488968>
- Kirkpatrick, S., Gelatt, C. D., Jr., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science*, 220(4598), 671–680. <https://doi.org/10.1126/science.220.4598.671>
- Mhamdi, B., Grayaa, K., & Aguilu, T. (2011). Hybrid of particle swarm optimization, simulated annealing and tabu search for the reconstruction of two-dimensional targets from laboratory-controlled data. *Progress in Electromagnetics Research B*, 28, 1–18. <https://doi.org/10.2528/PIERB10112902>
- Niño, E. D., Ardila, C. J., & Chinchilla, A. (2012). A novel, evolutionary, simulated annealing inspired algorithm for the multi-objective optimization of combinatorial problems. *Procedia Computer Science*, 9, 1992–1998. <https://doi.org/10.1016/j.procs.2012.04.218>
- Pirim, H., Bayraktar, E., & Eksioğlu, B. (2008). Tabu search: A comparative study. In W. Jaziri (Ed.), *Tabu search*. IntechOpen. <https://doi.org/10.5772/5637>
- Ru, S. (2024). Vehicle logistics intermodal route optimization based on tabu search algorithm. *Scientific Reports*, 14, Article 11859. <https://doi.org/10.1038/s41598-024-60361-7>
- Sengupta, S., Basak, S., & Peters, R. A., II. (2019). Particle swarm optimization: A survey of historical and recent developments with hybridization perspectives. *Machine*

Learning and Knowledge Extraction, 1(1), 157–191.

<https://doi.org/10.3390/make1010010>

- Watson, J.-P., Whitley, L. D., & Howe, A. E. (2005). Linking search space structure, run-time dynamics, and problem difficulty: A step toward demystifying tabu search. *Journal of Artificial Intelligence Research*, 24, 221–261.
- Xie, H., Zhang, L., Lim, C. P., Yu, Y., & Liu, H. (2021). Feature selection using enhanced particle swarm optimisation for classification models. *Sensors*, 21(5), Article 1816. <https://doi.org/10.3390/s21051816>
- Youssef, H., Sait, S. M., & Adiche, H. (2001). Evolutionary algorithms, simulated annealing and tabu search: A comparative study. *Engineering Applications of Artificial Intelligence*, 14(2), 167–181. [https://doi.org/10.1016/S0952-1976\(00\)00065-8](https://doi.org/10.1016/S0952-1976(00)00065-8)
- Zhan, S.-H., Lin, J., Zhang, Z.-J., & Zhong, Y.-W. (2016). List-based simulated annealing algorithm for traveling salesman problem. *Computational Intelligence and Neuroscience*, 2016, Article 1712630. <https://doi.org/10.1155/2016/1712630>
- Zhang, H., & Sun, G. (2002). Feature selection using tabu search method. *Pattern Recognition*, 35(3), 701–711. [https://doi.org/10.1016/S0031-3203\(01\)00046-2](https://doi.org/10.1016/S0031-3203(01)00046-2)
- Zhang, W., & Nicholson, C. D. (2018). *Empirical analysis of metaheuristic search techniques for the parameterized dynamic slope scaling procedure* [Preprint]. arXiv. <https://arxiv.org/abs/1808.10264>
- Zhang, Y., Gong, D.-W., Sun, X.-Y., & Guo, Y.-N. (2017). A PSO-based multi-objective multi-label feature selection method in classification. *Scientific Reports*, 7, Article 376. <https://doi.org/10.1038/s41598-017-00416-0>